MELLOW · APTRUN PLACEMENT SANDBOX

AptRun
Technical Codebase Guide

Roles, controllers, frontend pages, database and deployment — a field manual for making safe changes in production.



7 ROLES · ONE API · ONE STATIC FRONTEND

REPOSITORY	D:\Dev\AptRun (branch main)
SNAPSHOT	commit 6d03e41 "code before vps deploy" · 2026-10-12
BACKEND	Laravel 12 · PHP 8.2 · Sanctum tokens · MySQL
FRONTEND	Next.js 14 static export · React 18 · TypeScript · Tailwind
SCALE	439 API routes · 84 controllers · 73 models · 85 tables · 104 pages
GENERATED	12 October 2026

Internal engineering document. Derived from the source tree; environment values and credentials are intentionally omitted.

Contents

How to use this guide . . . 5

Three ways to read it . . . 5
The five facts that carry most of the understanding . . . 5
Conventions used in this document . . . 5
Map of this document . . . 6

Part I — Orientation . . . 7

1.1 What AptRun is . . . 7
1.2 Technology stack . . . 7
1.3 Repository map . . . 8
1.4 Architecture at a glance . . . 9
1.5 One request, end to end . . . 10
1.6 Running it locally . . . 10

Part II — Roles, permissions & access control . . . 11

2.1 The seven roles . . . 11
2.2 Granular permissions for staff accounts . . . 12
2.3 Three layers of enforcement . . . 13
2.4 Who can reach which part of the API . . . 14
2.5 Content ownership types (contests, interviews, soft skills) . . . 14
2.6 How each tenant is kept apart . . . 15
2.7 Who creates whom — the account lifecycle . . . 16
2.8 Login surfaces . . . 17
2.9 Role quick map — controllers, pages and tables per role . . . 18

Part III — Authentication & session . . . 20

3.1 Endpoints . . . 20
3.2 The token model . . . 20
3.3 What the frontend does with it . . . 21
3.4 Password reset, forced reset and phone verification . . . 21

Part IV — Backend reference . . . 22

4.1 Folder map . . . 22
4.2 Routing conventions you must know . . . 22
4.3 Controller directory — which controller serves which role . . . 23
4.4 The mirror pattern — why one feature appears 2–3 times . . . 28
4.5 Services — the “one write path” layer . . . 29
4.6 Queued jobs and mail . . . 31

4.7 Scheduler and artisan commands . . . 31
4.8 Rate limiters . . . 32
4.9 Configuration files . . . 32

Part V — Feature domains . . . 34

5.1 Colleges, roster & section coordinators . . . 34
5.2 Placement drives & the application pipeline . . . 35
5.3 Hiring-partner (company) portal . . . 36
5.4 Practice Arena & the code judge . . . 37
5.5 Contests . . . 38
5.6 AI interviews & Final Interview tracks . . . 39
5.7 Soft Skills assessments . . . 40
5.8 Proctoring . . . 40
5.9 Talent Pool marketplace . . . 41
5.10 Learning Centre — articles, speaking, listening, vocabulary . . . 42
5.11 Project Lab & browser workspaces . . . 43
5.12 Hack Arena . . . 44
5.13 Plans, subscriptions & entitlements . . . 45
5.14 Marketing funnel, leads & customers . . . 46
5.15 Superadmin & platform operations . . . 46
5.16 Reports, analytics & exports . . . 47

Part VI — Database . . . 48

6.1 Connection and conventions . . . 48
6.2 How the tables fit together . . . 49
6.3 Table inventory . . . 50
6.4 Status and type vocabularies . . . 53
6.5 Seeding — what each seeder is, and what is safe in production . . . 53
6.6 Changing the schema safely . . . 54
6.7 Recent migration history (themes) . . . 54

Part VII — Frontend reference . . . 55

7.1 The static-export model — and what it forces . . . 55
7.2 Source layout . . . 55
7.3 Shell, navigation and the multi-view /admin page . . . 56
7.4 One console, three roles — the basePath pattern . . . 56
7.5 Talking to the API . . . 57
7.6 Domain libraries (src/lib/) . . . 57
7.7 Static data, types and legacy files . . . 58
7.8 Styling and design system . . . 58

Part VIII — Infrastructure, configuration & deployment . . . 59

8.1 Environment variables by purpose . . . 59

8.2 What must be running in production . . . 60

8.3 Packaging and shipping a release . . . 60

8.4 Observability . . . 61

8.5 Things to watch — verified observations . . . 61

Part IX — Production change playbook . . . 62

9.1 "I need to change X" cheat sheet . . . 62

9.2 Recipes . . . 63

9.3 Symptom → likely cause → where to look . . . 65

9.4 Pre-deploy checklist . . . 65

9.5 Danger zones — where mistakes are expensive . . . 66

9.6 Command cheat sheet . . . 66

9.7 A short path from 5% to 75%, and a self-check . . . 67

Appendix A — Complete API route reference . . . 68

Appendix B — Frontend page → API map . . . 80

Appendix C — Database column reference . . . 91

Appendix D — Models and relationships . . . 98

Appendix E — Migration timeline . . . 101

Appendix F — Glossary . . . 103

START HERE

How to use this guide

A map for finding the right file, predicting the blast radius of a change, and shipping it without breaking a role you were not thinking about.

Three ways to read it

You want to...	Go to	What you get
Find where something lives ("which controller serves the TPO's student list?")	§2.9 role quick map, §9.1 cheat sheet, then Appendix A (all 439 routes) and Appendix B (all 104 pages)	File names, route, guard, role
Understand a feature end to end ("how does Run/Submit work?")	Part V — one section per feature domain	Roles, flow, controllers, services, tables, pages, gotchas
Ship a change safely ("add a field to the drive form")	Part IX recipes + Part VIII deployment	Step lists, the 'mirror' rule, a pre-deploy checklist, rollback
Learn the foundations first	Parts I → III in order	Architecture, the 7 roles, authentication

The five facts that carry most of the understanding

1. **One Laravel API, one static Next.js site.** Everything the browser does is an HTTP call to `/api/*` with a Bearer token. There is no server-side rendering and no cookie session — the frontend is plain files you can host anywhere.
2. **Seven roles, and the route group in `routes/api.php` is the permission model.** Each role family owns a URL prefix (`/admin`, `/tpo`, `/company`, `/coordinator`, `/marketing`, `/superadmin`; students use the API root and `/arena`). If you want to know who can call something, read the `Route::middleware('role:…')` group it sits in.
3. **Features are mirrored per role.** Interviews, contests, soft-skills, tracks and proctoring each exist as an `Admin*`, a `Tpo*` and a `Company*` controller, and the frontend reuses one console component with a `basePath` prop. A behaviour change usually has to be made two or three times (Part IV §4).
4. **Tenancy is enforced in controllers, not by the database.** A TPO is confined to their college because the controller reads `$request->user()->college_id`, never an id from the request. Break that rule and one college can see another's students.
5. **Slow or expensive work is queued.** Code judging, emails, bulk imports and AI interview scoring run in queue workers; reminders, contest scoring and workspace reaping run from a once-a-minute scheduler. In production, **workers and the cron entry must be running** or those features silently stop (Part VIII).

Conventions used in this document

- `Controller@method` names a PHP handler; paths such as `app/Http/Controllers/Api/AdminController.php` are relative to `backend/`, and `src/…` paths are relative to `frontend/`.
- Role keys are written exactly as stored in `users.role` and coloured: `user` `admin_tpo` `section_coordinator` `admin_company` `admin_internal` `admin_marketing` `superadmin`.
- Route URIs are shown without the `/api` prefix (the router adds it): `/admin/colleges` means `https://<api-host>/api/admin/colleges`.
- `{contest}` style parameters are Laravel route-model bindings. Seven models bind by **slug** instead of id: `Article`, `ArticleTopic`, `Contest`, `Interview`, `InterviewTrack`, `Problem`, `SoftSkillAssessment`.
- "Mellow" is the company that runs the platform (hence **Mellow Ops**, **Mellow Marketing**, **Mellow Internal**); **AptRun** is the product.

> **NOTE How this guide was produced — and what it does not cover**
> The route table comes from `php artisan route:list`, the schema from running all 125 migrations against a throw-away SQLite file, and the model, controller and page maps from parsing the source. Nothing was run against your MySQL database or any production server, so row counts, current data and the live server configuration are **not** described. Where a statement is an interpretation rather than something read directly from code it is worded as such ("appears to", "verify before relying"). Environment values are omitted on purpose: only variable **names** appear.

Map of this document

Part	What is inside	Open it when you need to...
I Orientation	What AptRun is, stack, repository map, architecture figure, request lifecycle, running locally	get your bearings
II Roles & access	The 7 roles, staff permissions, the three enforcement layers, access matrix, content ownership types, tenancy rules, account lifecycle, **role quick map**	decide who can do what, or find a role's code
III Authentication	Endpoints, token model, frontend session, password / OTP / phone flows	touch login, sessions or password rules
IV Backend	Folder map, routing rules, **controller directory by role**, the mirror pattern, services, jobs, scheduler, rate limits, config	change API behaviour
V Feature domains	16 chapters: roles, flow, controllers, services, tables, pages, gotchas for each product area	understand or modify one feature end to end
VI Database	Connection, table families and diagrams, status vocabularies, seeding, safe schema changes	change data or schema
VII Frontend	Static-export model, shell and navigation, the `basePath` pattern, API client, libraries, styling	change the UI
VIII Infrastructure	Env variables, what must run in production, packaging and deploy, observability, **things to watch**	deploy or operate
IX Playbook	"I need to change X" sheet, recipes, symptom table, checklists, danger zones, learning path	ship a change safely
Appendices A–F	All routes, page → API map, every column, models, migrations, glossary	look something up

PART I

Orientation

What AptRun is, what it is built from, and how one request travels through it.

1.1 What AptRun is

AptRun is a campus-placement and hiring platform. Colleges onboard their students, run practice and proctored assessments, and manage recruiter drives; hiring partners post openings, assess candidates and search a shared talent pool; students practise coding, interviews, soft skills and projects; Mellow staff run the platform and its sales funnel.

Audience	Role key	What they do in the product
Students / candidates	`user`	Practice Arena (DSA judge), contests, AI interviews, soft-skills tests, Learning Centre (speaking / listening / vocabulary / articles), Project Lab (browser VS Code), Hack Arena, placement drives, Talent Pool opt-in, readiness reports.
College placement officer (TPO)	`admin_tpo`	Imports the student roster, maps recruiter drives to the college, tracks each student through a drive, runs private mock contests / interviews / soft-skills tests, reviews proctoring, downloads readiness and placement reports.
Section coordinator	`section_coordinator`	A narrow slice of the TPO's view, limited to one section of one college: roster, reports, proctoring.
Hiring partner (company)	`admin_company`	Posts job openings, invites or imports candidates, runs proctored assessments and AI interviews, searches the Talent Pool, hosts hackathons.
Mellow Ops staff	`admin_internal`	Curates the catalog (companies, drives, problems, contests, interviews, articles, projects, hackathons) and onboards colleges. Which sections they may open is decided per employee by granular permissions.
Mellow Marketing staff	`admin_marketing`	Works the sales funnel: self-registered “Mellow Direct” leads and “Talk to Our Team” contact requests.
Superadmin	`superadmin`	Trust tier above everyone: creates staff, grants permissions, reads the audit log, watches judge nodes, flips feature flags.

1.2 Technology stack

Layer	Technology	Where it is configured
API framework	Laravel 12 on PHP ≥ 8.2 (dev machine: PHP 8.2.12 / XAMPP)	`backend/composer.json`, `bootstrap/app.php`
Authentication	Laravel Sanctum 4 — personal access tokens used as **Bearer tokens**	`config/sanctum.php`, `AuthController`
Database	MySQL (db name `codegen_box` on the dev machine; code comments cite MySQL 8 behaviour); SQLite used only for throw-away verification	`config/database.php`, `.env`
Queue / cache / sessions	`database` driver by default; **Redis** (predis) in production for the judge queue, cache and result store	`config/queue.php`, `config/cache.php`, `config/judge.php`
PDF / Excel (server)	dompdf (SOW PDFs) and phpoffice/phpspreadsheet (weekly readiness Excel, exports)	`composer.json`
Code execution	Self-hosted **Piston** nodes in Docker (isolate sandbox), 19 languages	`config/piston.php`, `infra/judge/`
Project Lab runtime	Docker containers running OpenVSCode Server behind a Caddy wildcard gateway	`config/workspaces.php`, `infra/workspaces/`
AI	Google Gemini over HTTPS (interview/soft-skill question generation, interview + speaking scoring, listening/vocabulary generation)	`config/services.php` (`GEMINI_*`)
Phone verification	Firebase Auth SMS OTP in the browser; server verifies the ID token with `firebase/php-jwt`	`FirebasePhoneVerificationService`
Frontend	Next.js 14.2 (App Router, `output: "export"`), React 18, TypeScript 5.7, Tailwind CSS 3.4	`frontend/package.json`, `next.config.mjs`

Layer	Technology	Where it is configured
Frontend libraries	framer-motion, GSAP, lucide-react, Monaco editor, react-markdown, exceljs + jspdf (client-side reports), Firebase JS SDK	`frontend/package.json`
Tests	PHPUnit 11 (backend: Feature + Unit), Vitest + Testing Library (unit), Playwright (e2e)	`phpunit.xml`, `vitest.config.ts`, `playwright.config.ts`

1.3 Repository map

```
AptRun/
├─ backend/                      Laravel 12 API — the only server-side code
│  ├─ app/
│  │  ├─ Http/Controllers/Api/    84 controllers  (+ Concerns/: 3 shared traits)
│  │  ├─ Http/Middleware/         EnsureUserHasRole, EnsureUserHasPermission
│  │  ├─ Models/                  73 Eloquent models (+ Concerns/HasSubscription)
│  │  ├─ Services/                86 classes — the business rules
│  │  │    Judge/ Workspaces/ Hackathons/ ProjectLab/ ProblemCodeGenerator/
│  │  ├─ Jobs/                    22 queued jobs (mail, imports, judge, AI scoring)
│  │  ├─ Mail/                    19 mailables
│  │  ├─ Console/Commands/        9 artisan commands
│  │  ├─ Support/                 5 small pure helpers
│  │  └─ Providers/AppServiceProvider.php   rate limiters · runtime binding · User::created hook
│  ├─ routes/   api.php (the map) · console.php (scheduler) · web.php (stub + named login route)
│  ├─ config/   20 files — judge, piston, workspaces, proctoring, billing, contests, authorship …
│  ├─ database/ migrations (125) · seeders (30 PHP) · factories (UserFactory)
│  ├─ tests/    Feature/ · Unit/ · Concerns/ (fixtures)
│  └─ Dockerfile                  PHP-Apache image (written for Render)
├─ frontend/                     Next.js 14 app, exported as static files
│  └─ src/  app/ (104 pages) · components/ · lib/ · types/ · data/ · tests/
├─ infra/                        deploy/ judge/ redis/ workspaces/ loadtest/ local/
├─ docs/                         feature docs (judge scaling, Project Lab, Hack Arena …) + this guide
├─ dist/                         output of infra/deploy/make-package.ps1 (git-ignored)
└─ .env  README.md  backend.zip   (root .env and the zip are NOT part of the app — see Part VIII §8)
```

1.4 Architecture at a glance

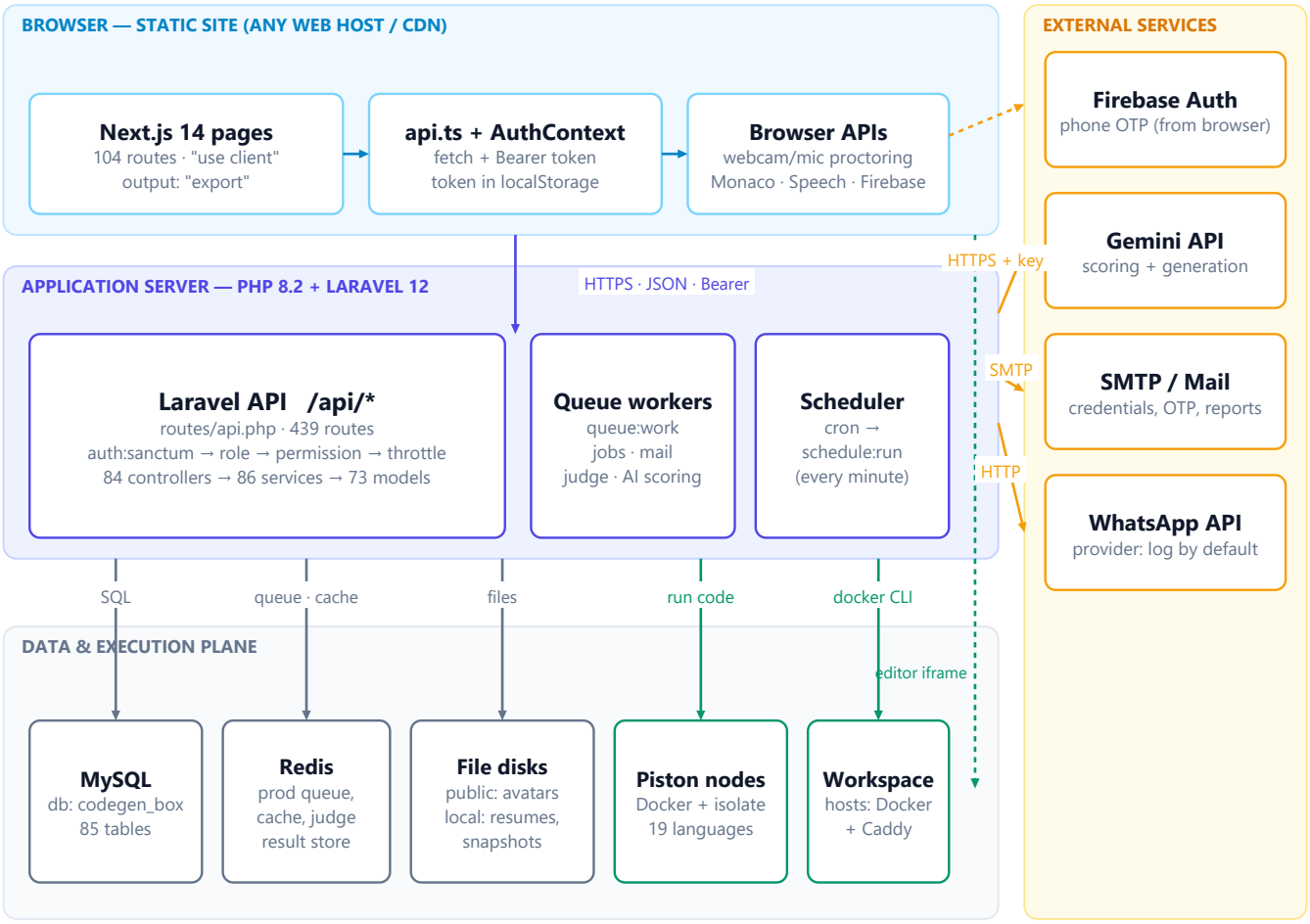


Figure 1 — Runtime components. Everything in the blue band is static files; the only dynamic server is the Laravel API.

Why this shape matters for production changes. The frontend is built once into plain HTML/JS (`frontend/out/`), so a frontend change means *rebuild and re-upload the static files*; the API address is baked in at build time (`NEXT_PUBLIC_API_URL`). A backend change means deploying PHP and — if you touched anything queued or scheduled — restarting workers (`php artisan queue:restart`). The two halves deploy independently, so a response-shape change must stay backward compatible until the new frontend is live.

Component	Responsibility	Fails how
Static frontend	Renders UI, holds the token, calls the API, records proctoring media **locally** (never uploaded)	Blank/stale pages if built with the wrong API URL
Laravel API	Auth, authorization, validation, business rules, persistence, enqueuing	403/401/422/500 JSON; 503 when judge queue is over depth
Queue workers	Run Jobs: emails, imports, judging, AI scoring, WhatsApp	Features look "stuck" (queued forever) if no worker runs
Scheduler (cron)	Weekly readiness mail, subscription expiry, contest finalize + daily contest, workspace reaper	Contests never score; idle workspaces keep running
Piston nodes	Execute student code in a sandbox, HTTP on port 2000 per node	Run/Submit report "execution engine unavailable" / retry
Workspace hosts	Docker + Caddy: one VS Code container per Project Lab student	Workspace stays "starting", then fails

1.5 One request, end to end

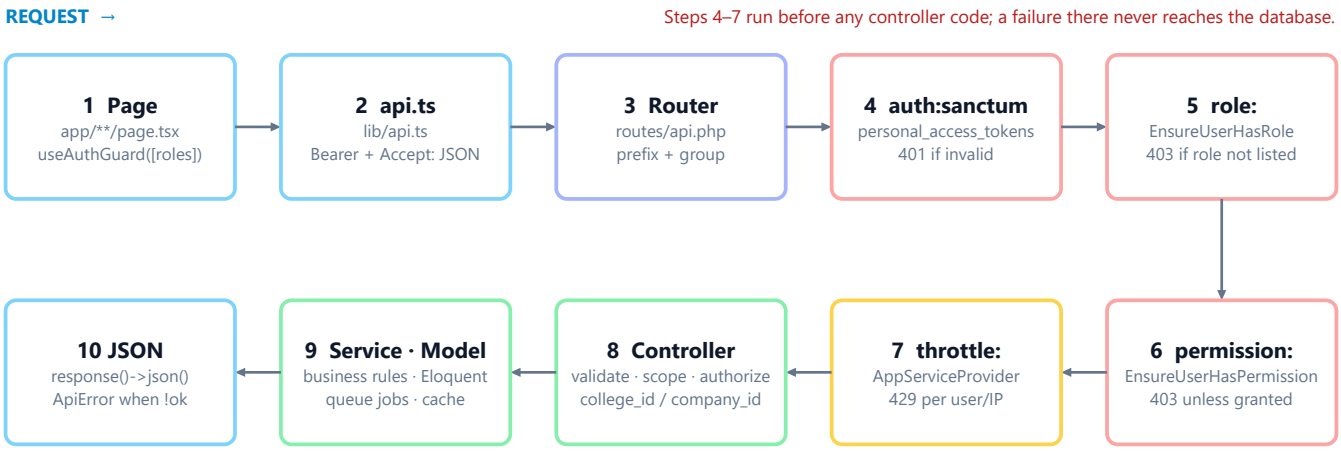


Figure 2 — The ten stops of every authenticated API call. Stops 4–7 are the security gate and live in routes/api.php and middleware.

Worked example — a student presses "Submit" on a practice problem

1. Page `src/app/dashboard/practice/view/page.tsx` (guarded by `useAuthGuard(["user"])`) calls `executeJudged()` in `src/lib/judge.ts`, which POSTs to `/problems/{slug}/submit` through `src/lib/api.ts` (adds `Authorization: Bearer <aptrun_token>`) and then polls with 400 ms → 2 s back-off.
2. Laravel matches the route inside the `role:user` group in `routes/api.php`, passing `auth:sanctum`, `EnsureUserHasRole:user` and the per-user `throttle:judge-submit` limiter.
3. `SubmissionController@submit` validates the code (max 64 KB), builds a `JudgeRequest` and hands it to `JudgeQueue`. It never executes code itself.
4. `JudgeQueue` short-circuits on a cache hit or a duplicate in-flight submission, enforces 3 active jobs per user and a queue-depth ceiling, then enqueues `JudgeSubmissionJob` and answers **202 + token**.
5. A queue worker runs `JudgeService` → `HarnessGenerator` (wraps the student's function with a generated test driver) → `CodeExecutionService` → `PistonPool` → a Piston node.
6. The verdict is persisted idempotently into `submissions` and cached in `JudgeResultStore`; the browser polls `GET /judge/{token}` until it reads `done`.

1.6 Running it locally

Task	Command (from the folder shown)	Notes
API	`php artisan serve` in `backend/` → http://127.0.0.1:8000	XAMPP's PHP ships with OPcache off, which makes the dev server slow; starting PHP with `-d zend_extension=opcache -d opcache.enable_cli=1` was measured ~4× faster on this machine.
Frontend	`npm run dev` in `frontend/` → http://localhost:3000	Reads `NEXT_PUBLIC_API_URL`; defaults to http://127.0.0.1:8000/api if unset.
Queue worker	`php artisan queue:listen --tries=1 --timeout=0`	Locally the judge runs **inline** (`JUDGE_QUEUE_CONNECTION` defaults to `sync` when `APP_ENV=local`).
Judge stack (prod-shaped)	`infra/local/judge-stack.ps1 up` (needs Docker)	Starts Redis + Piston nodes + workers; run `infra/judge/install-runtimes.sh` once.
Migrate / seed	`php artisan migrate` then seed **selectively**	Do not run bare `db:seed` anywhere real — see Part VIII §8.
Backend tests	`php artisan test` (PHPUnit 11)	Feature tests cover judge, Project Lab, Hack Arena, soft skills, interviews, billing.
Frontend tests	`npm test` (Vitest) · `npm run test:e2e` (Playwright)	e2e specs live in `src/tests/e2e/`.
Check & build	`npm run type-check` · `npm run lint` · `npm run build` → `frontend/out/`	`infra/deploy/make-package.ps1` wraps the build and verifies the API URL.

PART II

Roles, permissions & access control

Who exists, what each can reach, how that is enforced, and how tenants are kept apart. This is the single most important part to internalise before touching production.

2.1 The seven roles

A role is a string in `users.role` (declared as constants on `App\Models\User`). A user has **exactly one** role. The last column counts API routes that role can call, including the shared self-service ones.

Role key	Who / how created	Lands on after login	Frontend area	API prefix	Scoped by	Routes
`user`	Student. Self-registers (`POST /register`) **or** is created by a TPO (roster import / add one) **or** invited by a company.	`/dashboard`	`/dashboard/*`, `/arena/*`, `/settings`	API root (`/problems`, `/contests`, `/interviews` …) + `/arena`	`user_id`; `college_id` only affects *what is visible*	108
`admin_tpo`	College placement officer. Created together with the college by Mellow Ops (`AdminController::storeCollege`).	`/admin?view=tpo`	`/admin/*` (TPO pages) — Command Center, drives, students, coordinators, mock-*, analytics, reports, proctoring	`/tpo/*`	`college_id`	90
`section_coordinator`	Created by the college's TPO (`TpoCoordinatorController::store`) or by Ops on a college's behalf.	`/coordinator`	`/coordinator/*`	`/coordinator/*`	`college_id` **and** `section`	25
`admin_company`	Hiring partner's admin. Ops creates the company and its login in one step (`AdminCompanyController::store`).	`/admin?view=company`	`/admin/company/*`	`/company/*`	`company_id`	112
`admin_internal`	Mellow Ops employee. Created by a superadmin with a chosen set of permissions.	`/admin?view=mellow`	`/admin/*` (Ops pages), tabs of `/admin`	`/admin/*`	none (platform-wide), narrowed by `permissions[]`	156
`admin_marketing`	Mellow Marketing employee. Created by a superadmin.	`/marketing`	`/marketing`, `/marketing/inquiries`	`/marketing/*`	`assigned_marketing_id` on leads	25
`superadmin`	Only created by `php artisan superadmin:sync` from env vars — the API refuses to mint a superadmin.	`/superadmin`	`/superadmin?tab=…` + may use `/admin` and `/marketing` pages	`/superadmin/*` and all of `/admin/*`, `/marketing/*`	none	176

KEY IDEA **"admin" means three different things in the frontend**
In the **frontend** the folder `src/app/admin/` is shared: the same `/admin` page renders the Mellow Ops view, the TPO Command Center or the Company Hiring Command Center depending on `user.role`, and `/admin/company/*` is the hiring-partner area. In the **API**, `/admin/*` means Mellow Ops staff only; a TPO calls `/tpo/*`, a company calls `/company/*`. Never infer a role from a URL on one side and apply it to the other.

2.2 Granular permissions for staff accounts

`admin_internal` and `admin_marketing` accounts additionally carry a `permissions` JSON array chosen by a superadmin. Each key maps to exactly one dashboard section and is enforced by the `permission:` route middleware. **Superadmin always passes** (`User::hasPermission()` short-circuits), so a superadmin's own `permissions` column is irrelevant.

Permission key	Role	Routes	Controllers it unlocks	Frontend entry
`colleges`	`admin_internal`	14	Admin, StudentImport	`/admin?view=mellow&tab=colleges` — PartnerCollegesPanel
`platform_users`	`admin_internal`	6	Admin, AdminStudentReport	`/admin?view=mellow&tab=users` — PlatformUsersPanel
`problem_bank`	`admin_internal`	3	AdminProblem	`/admin?view=mellow&tab=problems` — ProblemBankPanel
`placements`	`admin_internal`	17	AdminCompany, AdminPlacementDrive	`/admin/placements`
`contests`	`admin_internal`	10	AdminContest	`/admin/contests`
`interviews`	`admin_internal`	25	AdminInterview, AdminInterviewQuestionBank, AdminInterviewRoleTemplate, AdminInterviewTrack	`/admin/interviews` (+ `/question-bank`, `/tracks`)
`customers`	`admin_internal`	3	InternalCustomer	`/admin/customers`
`articles`	`admin_internal`	9	AdminArticle, AdminArticleTopic	`/admin/articles` (+ `/new`, `/edit`)
`talent_pool`	`admin_internal`	3	AdminTalentPool	`/admin/talent-pool`
`soft_skills`	`admin_internal`	12	AdminSoftSkill, AdminSoftSkillQuestionBank	`/admin/soft-skills`
`project_lab`	`admin_internal`	9	AdminProjectLab	`/admin/projects` (+ `/edit`, `/review`)
`hackathons`	`admin_internal`	20	AdminHackathon	`/admin/hackathons` (+ `/edit`, `/manage`)
`leads`	`admin_marketing`	8	MarketingContactRequest, MarketingLead	`/marketing`, `/marketing/inquiries`
`lead_outreach`	`admin_marketing`	1	MarketingLead	`/marketing` — notify action

> **CAUTION** **Adding a permission is a data change as well as a code change**
> Existing staff only get a new key if a migration grants it. Look at `2026_09_24_…backfill_interviews_permission_for_full_access_staff`, `…backfill_articles_…`, `…backfill_soft_skills_…`, `…backfill_project_lab_…`, `…backfill_hackathons_…`: each one appends the new key to every employee who already held *all* previous permissions. Copy that pattern or nobody will see your new section. `GET /admin/overview` needs **no** permission — every `admin_internal` account can always see the platform-wide counts.

2.3 Three layers of enforcement

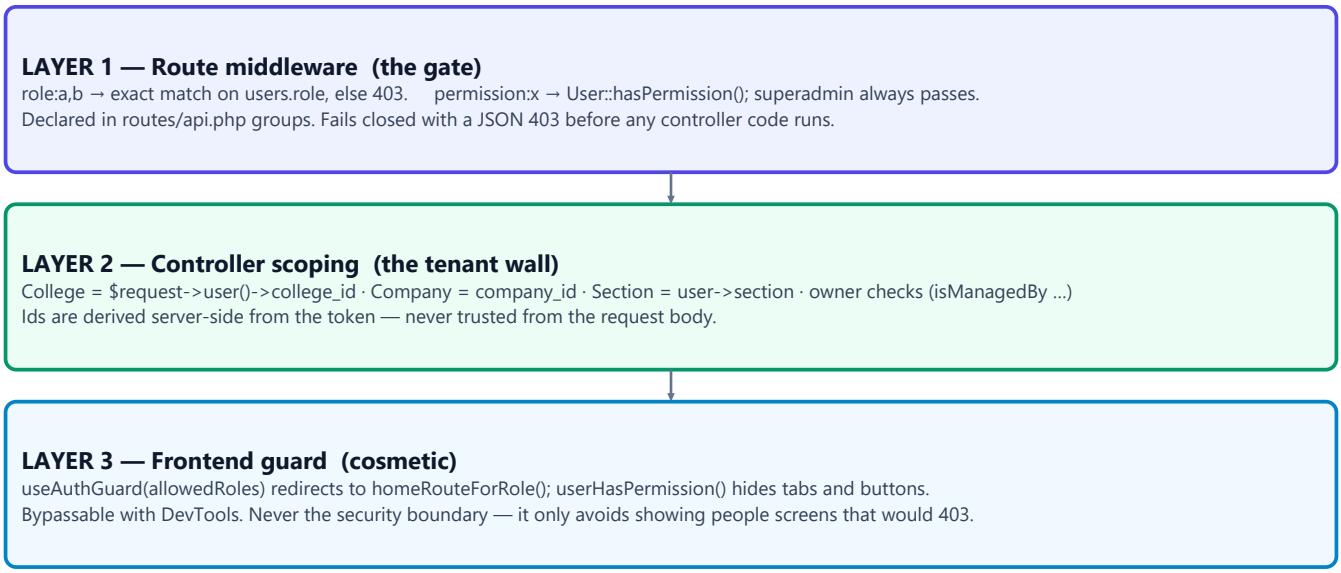


Figure 3 — Security lives in layers 1 and 2. Layer 3 only keeps people from walking into screens that would 403.

Layer 1 — the two middleware (app/Http/Middleware/)

```
// bootstrap/app.php  — aliases
'role'       => EnsureUserHasRole::class,        // role:admin_internal,superadmin
'permission' => EnsureUserHasPermission::class,  // permission:contests

// EnsureUserHasRole::handle()
if (! $user || ! in_array($user->role, $roles, true)) return 403 {"message": "…"}

// EnsureUserHasPermission::handle()   (passes if the user holds ANY listed permission)
if (! $user || ! collect($permissions)->contains(fn ($p) => $user->hasPermission($p))) return 403

// User::hasPermission()
if ($this->isSuperAdmin()) return true;
return in_array($key, $this->permissions ?? [], true);
```

Layer 3 — the frontend mirror (src/lib/auth.ts, src/lib/useAuthGuard.ts)

```
useAuthGuard(["admin_internal", "superadmin"])   // redirects anyone else to homeRouteForRole(user.role)
userHasPermission(user, "contests")               // superadmin → true, else user.permissions.includes(key)
```

If you add a section for staff, change **both** the route group (Layer 1) and the page's `useAuthGuard` / tab list (Layer 3); a missing Layer 3 entry shows a dead tab, a missing Layer 1 entry is a security hole.

2.4 Who can reach which part of the API

Counts of API routes by area (rows) and by the role that may call them (columns). The matrix is computed from the live route table, so it is the ground truth for "can role X call area Y at all?". A dot means none.

API area	Total	user	admin_tpo	section_coordinator	admin_company	admin_internal	admin_marketing	superadmin
Public (no token)	9	·	·	·	·	·	·	·
Self-service (/me/*, /logout)	16	16	16	16	16	16	16	16
Student & shared reads at the API root	82	82	1	·	1	1	·	1
/arena (Hack Arena, students)	10	10	·	·	5	5	·	5
/tpo (College TPO)	71	·	71	·	·	·	·	·
/coordinator (Section Coordinator)	9	·	·	9	·	·	·	·
/company (Hiring partner)	88	·	·	·	88	·	·	·
/admin (Mellow Ops)	132	·	·	·	·	132	·	132
/marketing (Marketing)	9	·	·	·	·	·	9	9
/superadmin	11	·	·	·	·	·	·	11
Shared staff reads (/interview-question-bank/*)	2	·	2	·	2	2	·	2

- **A TPO cannot call anything under** `/admin`**,** `/company`**,** `/marketing` **or** `/superadmin` (403), and cannot enter the Hack Arena.
- **Superadmin can call everything Ops and Marketing can** — it is listed in every `role:` group except `/tpo`, `/company`, `/coordinator` (those derive their scope from a `college_id` / `company_id` that staff accounts do not have).
- `/arena` reads are open to students and, as a *preview*, to Ops, hiring partners and superadmin; arena writes are students only.
- `GET /problems` is deliberately open to students, Ops, TPOs, companies and superadmin because every contest author needs to browse the problem bank.

2.5 Content ownership types (contests, interviews, soft skills)

Contests, AI interviews and soft-skills assessments all share one ownership model: a `*_type` column says who owns the item, and `owning_college_id` / `owning_company_id` / `placement_drive_id` say which tenant. **The type, not the controller, decides who sees it.** Each type is created and edited by exactly one controller family.

Type	Created by (controller)	Visible to	Rated / notes
`general`	Ops — `AdminContestController`, `AdminInterviewController`, `AdminSoftSkillController`	Everyone (platform-wide)	Contests may be rated (Elo-style) once finalized
`daily` (contest only)	Nobody — `contests:generate-daily` command	Everyone	One per calendar day; 1 easy + 1 medium + 1 hard; unique `daily_key`
`company`	Ops — same Admin controllers (contests, interviews); `CompanySoftSkillController` for soft-skills	Only colleges the linked drive is mapped **and approved** to (soft skills: colleges with an approved drive mapping to the company)	Tied to one company/drive
`tpo_mock`	College TPO — `TpoContestController`, `TpoInterviewController`, `TpoSoftSkillController`	Only that college (`owning_college_id`)	Never rated; invisible to other colleges
`company_hiring`	Hiring partner — `CompanyContestController`, `CompanyInterviewController`	**Invite-only** candidates of that company (`owning_company_id`)	Always tied to one job opening; never rated

Type	Created by (controller)	Visible to	Rated / notes
`talent_pool`	Ops — `AdminContestController` / `AdminInterviewController`	By `audience_scope`: `all`, `college` (via pivot table) or `direct` (college-less users)	Qualification test feeding the Talent Pool; passing score = `qualifying_score_percent`

The visibility rules are written once per model — `Contest::isVisibleToCollege()` / `isVisibleToUser()`, `Interview::isVisibleToUser()`, `SoftSkillAssessment::isVisibleToUser()` — and every student-facing list filters through them. **When you add a new type or audience, change the model method; do not add a filter in a controller.**

2.6 How each tenant is kept apart

Tenant	Where the scope comes from	Typical enforcement in code	Mirror controllers
College (TPO)	`$request->user()->college_id` — never from the request body	`where('owning_college_id', $collegeId)`; students `where('college_id', …)`; drive mappings keyed `(placement_drive_id, college_id)`	`Tpo*Controller`
Section	`college_id` + `users.section`	Same queries as the TPO's plus `where('section', $coordinator->section)`	`Coordinator*Controller`
Company	`$request->user()->company_id`	`where('owning_company_id', …)`; drives `where('company_id', …)` and `source = company_direct`; hackathons via `Hackathon::isManagedBy()`	`Company*Controller`
Student	`$request->user()->id`	Own rows only (`user_id`); `/judge/{token}` answers 404 for another user's token	`role:user` routes
Marketing	`assigned_marketing_id = auth user` (superadmin unscoped)	Leads and contact requests are least-loaded assigned by `LeadAssignmentService`	`Marketing*Controller`
Ops customers	`assigned_internal_id = auth user` (superadmin unscoped)	Converted leads handed off from Marketing on purchase	`InternalCustomerController`

DANGER The #1 way to break production isolation
Writing a query that takes `college_id` (or `company_id`, or a student id) from `$request->input()` instead of the authenticated user, or forgetting the scope on a *new* method of an existing controller. Route-model binding (`{contest}`, `{student}`) fetches **any** row by id/slug — the controller must still check the row belongs to the caller. Review every new route against 2.6 before merging.

2.7 Who creates whom — the account lifecycle

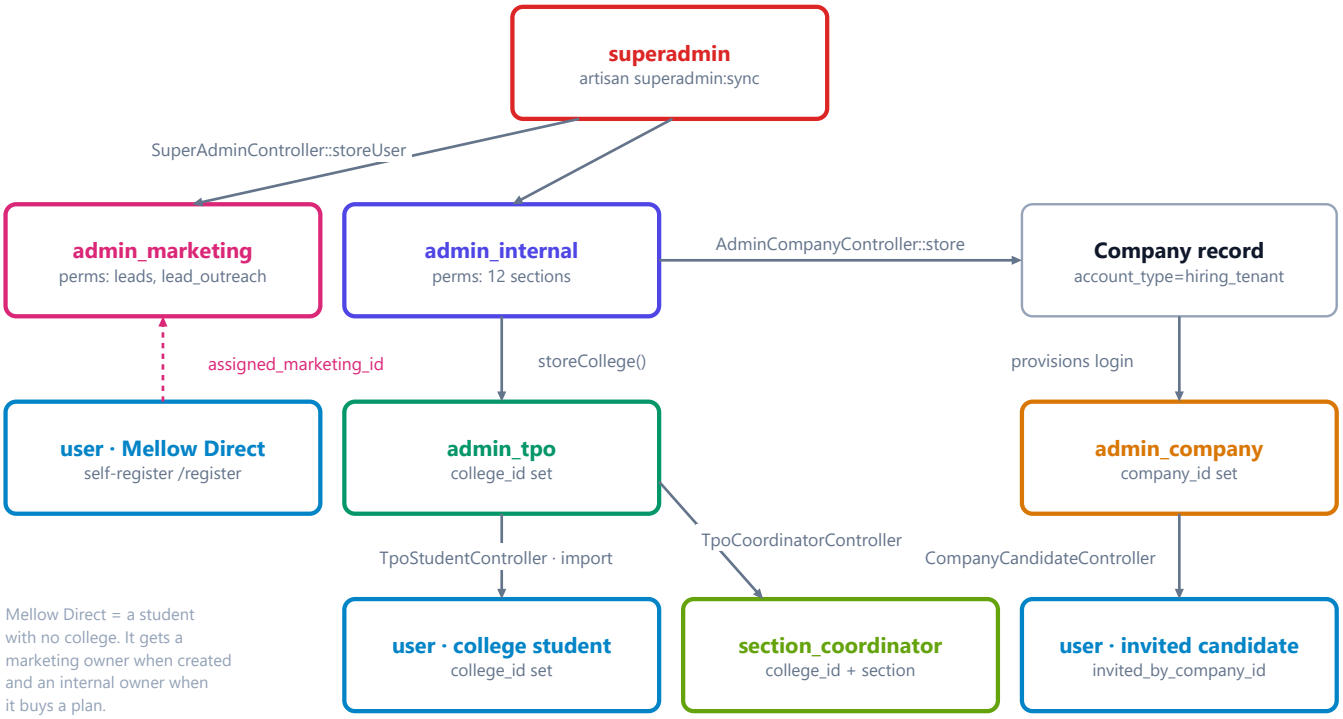


Figure 4 — Account creation paths. Solid = API action; dashed = automatic hook.

Account	Created via	First login behaviour
superadmin	`php artisan superadmin:sync` (reads `SUPERADMIN_EMAIL` / `SUPERADMIN_PASSWORD`); `--update-password` to rotate	Normal
admin_internal / admin_marketing	`POST /superadmin/users` with a `permissions[]` selection	Password is a one-time credential → forced reset
Company + admin_company	`POST /admin/companies` (needs `placements`)	Forced reset
College + admin_tpo	`POST /admin/colleges` (needs `colleges`) — also assigns the college's plan	Forced reset
section_coordinator	`POST /tpo/coordinators` (TPO) or `POST /admin/colleges/{college}/coordinators` (Ops)	Forced reset
Student (college)	TPO: `POST /tpo/students` or CSV import; Ops on a college's behalf: `POST /admin/colleges/{college}/students/import`	Forced reset; welcome email via queued `SendAccountCredentialsEmail`
Student (Mellow Direct)	`POST /register` (public) — role forced to `user`, no college	Token issued immediately; `SendWelcomeEmail`
Candidate (invited)	Company single add / CSV import (`CompanyCandidateController`)	Invite email; `invited_by_company_id` set

Forced reset (`must_change_password`). A password generated by an admin is only proof that you received the email, so `POST /login` returns `{must_change_password: true}` **without a token**. The frontend routes this into `ForcePasswordResetModal`, which reuses the forgot-password OTP endpoints (`/password/forgot` → `/password/otp/verify` → `/password/reset`) and then signs the user in.

Automatic hooks. `User::created` (in `AppServiceProvider`) attaches the free **Coder** plan to every new student and — if the new user is a college-less "Mellow Direct" lead — assigns a marketing owner via `LeadAssignmentService`. This runs for *every* creation path including seeders and imports.

2.8 Login surfaces

URL	For	Notes
`/login`	`user`, `admin_tpo`, `admin_company` (three tabs)	Customer-facing. Never offers staff roles. After login → `homeRouteForRole(role)`.
`/signup`	New students	Calls `POST /register`; role is always `user`.
`/mellow-internal`	`superadmin`, `admin_internal`, `admin_marketing`	Unlisted: `noindex` meta + `robots.txt` disallow. Not an access boundary — the same `/login` API endpoint authenticates; the page just rejects other roles client-side.
Forgot password	Everyone except superadmin	6-digit OTP, 10-minute lifetime, 5 attempts, then a one-time reset token; all existing tokens are revoked on reset.

> **CAUTION** **Dev demo logins in the frontend**
> `src/app/mellow-internal/page.tsx` contains a `DEMO_LOGINS` block that is only compiled in when `NODE_ENV !== "production"`. A `next build` is a production build, so it is not shipped — but it is a reason never to point a local dev build at a production API.

2.9 Role quick map — controllers, pages and tables per role

One block per role: the controllers written for it (routes in its own API prefix), the frontend pages guarded for it, the tables it mostly reads and writes, and where it signs in. This is the fastest way to answer "where do I change this for TPOs?". Controller names are shown without the `Controller` suffix.

`user` — 108 API routes reachable · 25 controllers in its own prefix

Controllers	HackathonArena 10 · Interview 8 · ProjectLab 8 · SoftSkill 7 · ProjectWorkspace 6 · Contest 5 · Vocabulary 5 · Article 4 · ListeningLab 4 · SpeakingPractice 4 · StudentTalentPool 4 · ContestSubmission 3 · MySubmissions 3 · ContestProctoring 2 · InterviewProctoring 2 · InterviewTrack 2 · Problem 2 · SoftSkillProctoring 2 · StudentDrive 2 · StudentStats 2 · Submission 2 · VocabularyProgress 2 · JudgeResult 1 · Leaderboard 1 · LearningCentre 1
Frontend pages	/arena/my · /dashboard · /dashboard/articles · /dashboard/articles/topic · /dashboard/articles/view · /dashboard/billing · /dashboard/contests · /dashboard/contests/problem · /dashboard/contests/view · /dashboard/drives · /dashboard/interview-tracks · /dashboard/interviews · /dashboard/interviews/session · /dashboard/interviews/view · /dashboard/learning-centre · /dashboard/learning-centre/listening · /dashboard/learning-centre/listening/session · /dashboard/learning-centre/speaking · /dashboard/learning-centre/speaking/session · /dashboard/learning-centre/vocabulary · /dashboard/learning-centre/vocabulary/session · /dashboard/learning-centre/vocabulary/words · /dashboard/practice · /dashboard/practice/view · /dashboard/projects · /dashboard/projects/view · /dashboard/projects/workspace · /dashboard/reports · /dashboard/soft-skills · /dashboard/soft-skills/session · /dashboard/soft-skills/view · /dashboard/talent-pool — also shared: /arena · /arena/hackathon · /dashboard/leaderboard · /settings
Key tables	users · submissions · contest_participants / contest_submissions · interview_sessions / interview_responses · soft_skill_sessions / soft_skill_responses · project_enrollments / project_workspaces / project_submissions · hackathon_participants / _teams / _submissions · vocabulary_* · speaking_attempts · listening_attempts · article_reads · talent_pool_candidates · subscriptions
Sign-in	/login (tab Student) or /signup → /dashboard

`admin_tpo` — 90 API routes reachable · 14 controllers in its own prefix

Controllers	TpoInterview 13 · TpoContest 9 · TpoDrive 9 · TpoSoftSkill 8 · StudentImport 4 · TpoCoordinator 4 · TpoDriveApplication 4 · TpoInterviewTrack 4 · TpoInterviewRoleTemplate 3 · TpoProctoring 3 · TpoStudent 3 · TpoStudentReport 3 · AdminSoftSkillQuestionBank 2 · TpoReports 2
Frontend pages	/admin · /admin/analytics · /admin/coordinators · /admin/drives · /admin/mock-contests · /admin/mock-interviews · /admin/mock-soft-skills · /admin/proctoring · /admin/reports · /admin/students · /admin/students/report — also shared: /arena · /arena/hackathon · /dashboard/leaderboard · /settings
Key tables	colleges · users (own college) · student_imports · drive_college_mappings · placement_drives (own) · drive_applications · contests / interviews / soft_skill_assessments (type tpo_mock) · interview_tracks · proctoring_*
Sign-in	/login (tab TPO) → /admin?view=tpo

`section_coordinator` — 25 API routes reachable · 3 controllers in its own prefix

Controllers	CoordinatorProctoring 3 · CoordinatorStudent 3 · CoordinatorStudentReport 3
Frontend pages	/admin/students/report · /coordinator · /coordinator/proctoring · /coordinator/reports
Key tables	users (own college + section) · proctoring_sessions / proctoring_violations · reads submissions and contest_submissions through reports
Sign-in	/login → /coordinator

`admin_company` — 112 API routes reachable · 13 controllers in its own prefix

Controllers	CompanyHackathon 20 · CompanyInterview 14 · CompanyContest 11 · CompanySoftSkill 8 · CompanyTalentPool 8 · CompanyCandidate 7 · CompanyDrive 6 · CompanyInterviewTrack 4 · CompanyInterviewRoleTemplate 3 · CompanyProctoring 3 · AdminSoftSkillQuestionBank 2 · CompanyCollege 1 · CompanyReports 1
Frontend pages	/admin · /admin/company/assessments · /admin/company/candidates · /admin/company/colleges · /admin/company/drives · /admin/company/hackathons · /admin/company/hackathons/edit · /admin/company/hackathons/manage · /admin/company/interviews · /admin/company/proctoring · /admin/company/reports · /admin/company/soft-skills · /admin/company/talent-pool — also shared: /arena · /arena/hackathon · /dashboard/leaderboard · /settings
Key tables	companies · placement_drives (company_direct) · drive_applications · candidate_imports · contests / interviews / soft_skill_assessments (type company_hiring / company) · interview_tracks · talent_pool_inquiries / _messages · hackathons

Sign-in	`/login` (tab Hiring Partner) → `/admin?view=company`

`admin_internal` — 156 API routes reachable · 19 controllers in its own prefix

Controllers	`AdminHackathon` 20 · `Admin` 14 · `AdminInterview` 14 · `AdminCompany` 11 · `AdminContest` 10 · `AdminProjectLab` 9 · `AdminSoftSkill` 8 · `AdminPlacementDrive` 6 · `AdminArticle` 5 · `AdminInterviewTrack` 5 · `AdminArticleTopic` 4 · `AdminSoftSkillQuestionBank` 4 · `StudentImport` 4 · `AdminInterviewQuestionBank` 3 · `AdminInterviewRoleTemplate` 3 · `AdminProblem` 3 · `AdminStudentReport` 3 · `AdminTalentPool` 3 · `InternalCustomer` 3
Frontend pages	`/admin` · `/admin/articles` · `/admin/articles/edit` · `/admin/articles/new` · `/admin/contests` · `/admin/customers` · `/admin/hackathons` · `/admin/hackathons/edit` · `/admin/hackathons/manage` · `/admin/interviews` · `/admin/interviews/question-bank` · `/admin/interviews/tracks` · `/admin/placements` · `/admin/projects` · `/admin/projects/edit` · `/admin/projects/review` · `/admin/soft-skills` · `/admin/students/report` · `/admin/talent-pool`
Key tables	`colleges` · `companies` · `placement_drives` · `drive_college_mappings` · `problems` / `problem_test_cases` · `contests` · `interviews` · `interview_question_banks` · `soft_skill_*` · `articles` / `article_topics` · `project_topics` / `project_submissions` · `hackathons` · `talent_pool_candidates` · `users` · `subscriptions` · `activity_logs`
Sign-in	`/mellow-internal` → `/admin?view=mellow`

`admin_marketing` — 25 API routes reachable · 2 controllers in its own prefix

Controllers	`MarketingLead` 5 · `MarketingContactRequest` 4
Frontend pages	`/marketing` · `/marketing/inquiries`
Key tables	`users` (lead columns) · `lead_notes` · `contact_requests` · `contact_request_notes`
Sign-in	`/mellow-internal` → `/marketing`

`superadmin` — 176 API routes reachable · 1 controller in its own prefix

Controllers	`SuperAdmin` 11 — plus every `admin_internal` and `admin_marketing` controller (it is in those role lists)
Frontend pages	`/superadmin` — plus every Ops and Marketing page
Key tables	everything Ops uses, plus `activity_logs` · `feature_flags` · `users` of every role
Sign-in	`/mellow-internal` → `/superadmin`

PART III

Authentication & session

Tokens, storage, the guard hooks, and the password flows — everything between "user types a password" and "every request is authorised".

3.1 Endpoints

Method	URI	Handler	Guard	Throttle
POST	/login	Auth@login	(public)	login
POST	/logout	Auth@logout	(auth only)	—
GET	/me	Auth@me	(auth only)	—
POST	/me/avatar	Auth@updateAvatar	(auth only)	—
DELETE	/me/avatar	Auth@deleteAvatar	(auth only)	—
PUT	/me/password	Auth@updatePassword	(auth only)	—
POST	/me/phone/verify	PhoneVerification@verify	(auth only)	phone-verify
PUT	/me/profile	Auth@updateProfile	(auth only)	—
POST	/me/resume	StudentProfile@uploadResume	(auth only)	—
DELETE	/me/resume	StudentProfile@deleteResume	(auth only)	—
GET	/me/resume	StudentProfile@downloadOwnResume	(auth only)	—
GET	/me/sessions	Auth@sessions	(auth only)	—
DELETE	/me/sessions/{tokenId}	Auth@revokeSession	(auth only)	—
PUT	/me/student-profile	StudentProfile@update	(auth only)	—
POST	/password/forgot	PasswordReset@forgot	(public)	password-reset-request
POST	/password/otp/verify	PasswordReset@verifyOtp	(public)	password-reset-verify
POST	/password/reset	PasswordReset@reset	(public)	password-reset-verify
POST	/register	Auth@register	(public)	—

Throttle names are defined in `AppServiceProvider::boot()`. `login`: 5 / min per email **and** 20 / min per IP. `password-reset-request`: 3 / min per email, 10 / min per IP. `password-reset-verify`: 10 / min per email, 20 / min per IP. `phone-verify`: 10 / min per user.

3.2 The token model

- `POST /login` verifies the password (`Hash::check`), rejects blocked accounts (`users.is_blocked`), then calls `$user->createToken('auth')->plainTextToken`.
- Tokens live in `personal_access_tokens` (Sanctum). `config/sanctum.php` **sets** `expiration` **to** `null`**, so tokens never expire on their own** — they end on logout (`currentAccessToken()->delete()`), on `DELETE /me/sessions/{id}`, or when a password reset deletes *all* of a user's tokens.
- Every protected request sends `Authorization: Bearer <token>`. There are **no cookies and no CSRF** involved; `config/cors.php` has `supports_credentials = false`.
- **Blocking is enforced only at login.** `is_blocked` is checked once, in `AuthController@login`. The four block toggles (`AdminController`, `SuperAdminController`, `AdminCompanyController`, `CoordinatorStudentController`) set the flag and write an audit row but do **not** delete the user's tokens, and no middleware re-checks the flag (confirmed by searching `app/` for `is_blocked` and `tokens()`). Since tokens never expire, a user blocked while signed in keeps working until they log out. The comment in `AuthContext.tsx` assumes the server answers 403 for blocked accounts — nothing does today. If blocking must bite immediately, call `$user->tokens()->delete()` inside the toggle and/or add the check to a middleware.
- `GET /me/sessions` lists a user's tokens (device sessions) and `DELETE /me/sessions/{tokenId}` revokes one — both in `AuthController`.

3.3 What the frontend does with it

Piece	File	Behaviour
Storage	`src/lib/auth.ts`	`localStorage` keys `aptrun_token` and `aptrun_user` (a cached copy of the user for instant paint).
HTTP client	`src/lib/api.ts`	`api.get/post/put/delete/postFormData/getFile`; adds Bearer + `Accept: application/json`; throws `ApiError(message, status, errors, code)`; de-duplicates identical in-flight GETs.
Session provider	`src/lib/AuthContext.tsx`	Paints from the cached user, then verifies with `GET /me` once per app load and again when the tab becomes visible. Only **401/403** sign the user out; a timeout or 5xx keeps the cached session (fixed 2026-10-10).
Per-page gate	`src/lib/useAuthGuard.ts`	`useAuthGuard(allowedRoles?)` → `{user, status}`; redirects signed-out users to `/login` and wrong-role users to `homeRouteForRole(role)`.
Home routes	`homeRouteForRole()` in `auth.ts`	superadmin `/superadmin` · admin_internal `/admin?view=mellow` · admin_tpo `/admin?view=tpo` · admin_marketing `/marketing` · section_coordinator `/coordinator` · admin_company `/admin?view=company` · user `/dashboard`.
Permission helper	`userHasPermission()`	Used by `/admin` and `/marketing` to hide tabs the employee cannot open.

3.4 Password reset, forced reset and phone verification

- **Forgot password** (`PasswordResetService`): `requestOtp` deletes any previous unconsumed row for the user, stores a *hashed* 6-digit code (`password_reset_otps.code_hash`) valid 10 minutes with a 5-attempt cap, and queues `SendPasswordResetOtpEmail` (an encrypted job, so the code never sits in plaintext in the `jobs` table). Verifying returns a one-time reset token (also stored hashed); `reset` sets the password, clears `must_change_password` and **deletes every Sanctum token**. The endpoints never reveal whether an email exists, and superadmin accounts are a silent no-op.
- **Forced reset** reuses the same three endpoints; the only difference is the login response described in 2.7.
- **Phone verification** (`PhoneVerificationController@verify`): the browser performs the SMS OTP directly against Firebase; the API receives the resulting **ID token** and `FirebasePhoneVerificationService` verifies its signature against Google's public JWKS (cached 1 h) and checks issuer/audience — no Firebase Admin SDK or service account needed. Only then is `phone_verified_at` set (`forceFill`, never mass-assignable).

> **TIP Where to change auth behaviour**
> Token lifetime → `config/sanctum.php expiration` (minutes) · login throttle → `AppServiceProvider RateLimiter::for('login')` · password rules → `AuthController@register/updatePassword` validation · post-login landing page → `homeRouteForRole()` in `src/lib/auth.ts` · add a role → constants on `User`, `ROLES`, `homeRouteForRole`, the `Role` type, the sidebar in `DashboardSidebar.tsx`, a `role:` route group, and the DB enum (see the `widen_users_role_enum_*` migrations — they alter a MySQL ENUM).

PART IV

Backend reference

The Laravel application: where each kind of code lives, which controller serves which role, and the shared machinery (services, jobs, scheduler, limits).

4.1 Folder map

Path (under `backend/`)	Count	What belongs here — and what does not
`routes/api.php`	439 routes	The access-control map. Only wiring: no logic. Closure routes: none in the API.
`app/Http/Controllers/Api/`	84	HTTP edge: validate input, authorize/scope, call a service or model, return JSON. Keep business rules out of here.
`app/Http/Controllers/Api/Concerns/`	3 traits	`ManagesHackathons`, `ManagesSoftSkillQuestions`, `EnforcesLearningCentreAiLimit` — logic shared by sibling controllers.
`app/Http/Middleware/`	2	`EnsureUserHasRole`, `EnsureUserHasPermission` (aliases `role`, `permission`).
`app/Models/`	73	Eloquent models: relations, constants (statuses/types), visibility methods (`isVisibleToUser`), casts.
`app/Services/`	86	Business rules, each "the one write path" for a concept (see 4.5). Subfolders: Judge, Workspaces, Hackathons, ProjectLab, ProblemCodeGenerator.
`app/Jobs/` · `app/Mail/`	22 · 19	Queued work and the mailables it sends (see 4.6).
`app/Console/Commands/`	9	Artisan commands, several scheduled (see 4.7).
`app/Support/`	5	Pure helpers with no framework dependencies (`ActivityStreak`, `SpeechAlignment`, `VocabularyText` …).
`app/Providers/AppServiceProvider.php`	1	Rate limiters, the workspace-runtime container binding, and the `User::created` hook (default plan + lead assignment).
`config/`	20	Per-feature settings read from env (see 4.9).
`database/migrations/`	125	Schema history (Part VI). Never edit an applied migration.
`database/seeders/`	30 PHP	Catalog/demo content. `DatabaseSeeder` also creates demo accounts — see the warning in Part VIII.
`tests/`	—	`Feature/` (HTTP + DB), `Unit/`, `Concerns/` (fixture builders for Project Lab, Hackathons, listening, vocabulary).

4.2 Routing conventions you must know

- **All API routes are in one file**, grouped by guard. The nesting is `auth:sanctum` → `role:` (→ `permission:` for staff) → optional `throttle:`. A new route placed outside the right group is silently exposed to the wrong roles.
- **Declaration order matters for wildcards.** `/hackathons/colleges` and `/soft-skill-question-bank/generate` are declared *before* `/hackathons/{hackathon}` and `/soft-skill-question-bank/{softSkillQuestion}`; the reverse order would swallow the literal as an id and 404. The comments in `api.php` flag each case — keep them.
- **Slug vs id.** `Article`, `ArticleTopic`, `Contest`, `Interview`, `InterviewTrack`, `Problem`, `SoftSkillAssessment` override `getRouteKeyName()` to `slug`; everything else binds by numeric id. Arena routes bind hackathons explicitly by slug (`{hackathon:slug}`) while host routes use the id. The frontend therefore sends `contest.slug` for contests and `hackathon.id` for host screens.
- **Static export means ids travel in the query string on the frontend** (`/dashboard/contests/view?slug=…`), but are normal path segments in the API.
- **Errors are JSON.** `routes/web.php` defines a `login` route that returns a JSON 401 so Laravel's default redirect never throws `RouteNotFoundException`; the frontend always sends `Accept: application/json`.
- **Health check:** `GET /up` (registered in `bootstrap/app.php`, outside `/api`).

4.3 Controller directory — which controller serves which role

Every controller below is in `app/Http/Controllers/Api/`; the class name is shown **without its `Controller` suffix** (e.g. `AdminCompany` is `AdminCompanyController`). "Routes" counts the endpoints it serves **in that area** (a few controllers are mounted in several areas). "Frontend pages" lists the pages whose code (including the components they import) calls it directly; pages that reach a controller only through a shared component with a variable `basePath` are not listed here — Appendix B shows every page's endpoints.

Public (no token) — 9 routes, 5 controllers

Controller	Routes	What it does	Frontend pages	Models · services
`PasswordReset`	3	PUBLIC forgot-password OTP flow	`/login`	User; PasswordResetService
`Auth`	2	register, login, logout, me, profile, avatar, password, active sessions	`/login`, `/mellow-internal`, `/settings` +1	User
`Public`	2	PUBLIC landing stats + sample problems	`/`, `/login`, `/signup`	Problem, ProblemTestCase
`ContactRequest`	1	PUBLIC 'Talk to Our Team' form submit	`/`, `/colleges`, `/contact` +13	ContactRequest; ContactRequestService
`Newsletter`	1	PUBLIC newsletter signup	—	NewsletterSubscriber

Self-service for any signed-in user (/me/*, /logout) — 16 routes, 4 controllers

Controller	Routes	What it does	Frontend pages	Models · services
`Auth`	8	register, login, logout, me, profile, avatar, password, active sessions	`/login`, `/mellow-internal`, `/settings` +1	User
`StudentProfile`	4	Student recruiter-ready profile + resume upload/download	`/settings`	—
`Subscription`	3	Billing: own coverage, price quote, subscribe	`/admin`, `/dashboard/billing`, `/dashboard/interviews` +5	Interview, InterviewSession, Plan, Submission …; ProjectLabAccess, PurchaseQuoteService, SubscriptionService
`PhoneVerification`	1	Verify phone via Firebase ID token	`/settings`	User; FirebasePhoneVerificationService

`user` Student routes at the API root (and shared reads) — 82 routes, 24 controllers

Controller	Routes	What it does	Frontend pages	Models · services
`Interview`	8	Student: browse, start, answer (audio+transcript), complete, result	`/dashboard/interviews`, `/dashboard/interviews/session`, `/dashboard/interviews/view` +1	Interview, InterviewQuestion, InterviewResponse, InterviewSession; StudentReportService
`ProjectLab`	8	Student Project Lab: catalog, start, SOW PDF, milestones	`/dashboard/projects`, `/dashboard/projects/view`, `/dashboard/projects/workspace`	ActivityLog, ProjectEnrollment, ProjectMilestone, ProjectMilestoneProgress …; ProjectLabAccess, ProjectLabPresenter, SowPdfService …
`SoftSkill`	7	Student: soft-skills browse, take, autosave, submit, view	`/dashboard/reports`, `/dashboard/soft-skills`, `/dashboard/soft-skills/session` +1	SoftSkillAssessment, SoftSkillResponse, SoftSkillSession; SoftSkillGradingService, StudentReportService
`ProjectWorkspace`	6	Student workspace: start/poll/heartbeat/integrity/stop/submit	`/dashboard/projects/workspace`	ProjectEnrollment, ProjectWorkspace; ProjectLabAccess, ProjectLabPresenter, ProjectSubmissionService …
`Contest`	5	Student: browse, register/unregister, live leaderboard	`/dashboard`, `/dashboard/contests`, `/dashboard/contests/view`	Contest, ContestParticipant
`Vocabulary`	5	Student: Vocabulary Sprint sessions, answers, AI quiz	`/dashboard/learning-centre/vocabulary`, `/dashboard/learning-centre/vocabulary/session`	VocabularyAttempt, VocabularyWord; GeminiVocabularyQuizService, VocabularyAnswerService, VocabularyProgressService …

Controller	Routes	What it does	Frontend pages	Models · services
Article	4	Student: read published articles/topics, mark read	/dashboard/articles, /dashboard/articles/topic, /dashboard/articles/view	Article, ArticleRead, ArticleTopic
ListeningLab	4	Student: Listening Lab lessons, AI-generate, attempts	/dashboard/learning-centre/listening, /dashboard/learning-centre/listening/session	ListeningAttempt, ListeningLesson; GeminiListeningLessonService, GeminiSpeakingPassageService, ListeningProgressService
SpeakingPractice	4	Student: Speaking Practice (Gemini-scored from audio)	/dashboard/learning-centre/speaking, /dashboard/learning-centre/speaking/session	SpeakingAttempt, SpeakingPrompt; GeminiSpeakingPassageService, GeminiSpeakingScoringService, NoSpeechDetectedException
StudentTalentPool	4	Student: Talent Pool consent + respond to HR invites	/dashboard/talent-pool	TalentPoolCandidate, TalentPoolInquiry
ContestSubmission	3	Student: fetch/run/submit one contest problem	—	Contest, ContestParticipant, ContestProblem, ProblemTestCase; JudgeService, JudgeQueue, JudgeRequest …
MySubmissions	3	Student: own submission history + code	/dashboard/reports	ContestSubmission, Submission; StudentReportService
ContestProctoring	2	Student: contest proctoring start + violation reports	—	Contest, ContestParticipant, ContestProblem, ProctoringSession …; JudgeService, ProctoringService
InterviewProctoring	2	Student: interview proctoring start + violations	—	Interview, InterviewProctoringViolation, InterviewQuestion, InterviewSession; InterviewProctoringService
InterviewTrack	2	Student: Final Interview pipeline state (locked/unlocked/scored)	/dashboard/interview-tracks, /dashboard/interviews	Interview, InterviewSession, InterviewTrack
Problem	2	Practice Arena problem catalog + detail	/admin/company/assessments, /admin/contests, /admin/mock-contests +3	Problem, ProblemTestCase, Submission; StarterCodeGenerator
SoftSkillProctoring	2	Student: soft-skills proctoring	—	SoftSkillProctoringSession, SoftSkillProctoringViolation, SoftSkillSession; SoftSkillProctoringService
StudentDrive	2	Student: drives mapped to own college + eligibility	/dashboard, /dashboard/drives, /dashboard/reports	DriveApplication, DriveCollegeMapping, PlacementDrive, Problem …; EligibilityService
StudentStats	2	Student dashboard stats + rating history	/dashboard, /dashboard/practice, /dashboard/reports	ContestParticipant; StudentStatsService
Submission	2	Practice Run/Submit (enqueue on judge)	—	Problem, Submission; JudgeService, JudgeQueue, JudgeRequest
VocabularyProgress	2	Student: vocabulary overview + word bank	/dashboard/learning-centre/vocabulary, /dashboard/learning-centre/vocabulary/words	VocabularyWordProgress; VocabularyProgressService
JudgeResult	1	Poll a queued Run/Submit verdict by token	/dashboard/contests/problem, /dashboard/practice/view	JudgeQueue
Leaderboard	1	Global leaderboard (solved score / contest rating)	/dashboard, /dashboard/leaderboard	User
LearningCentre	1	Learning Centre hub stats (one aggregate call)	/dashboard/learning-centre	ListeningAttempt, SpeakingAttempt, VocabularyAnswer, VocabularyAttempt …; ListeningProgressService

user /arena — Hack Arena — 10 routes, 1 controllers

Controller	Routes	What it does	Frontend pages	Models · services
HackathonArena	10	Student Hack Arena: browse, register, team, save/submit project	/admin/company/hackathons, /admin/company/hackathons/edit, /admin/company/hackathons/manage +7	Hackathon, HackathonParticipant, HackathonSubmission, User; HackathonPresenter, HackathonRegistrationService, HackathonRuleException ...

admin_tpo /tpo/* — 71 routes, 14 controllers

Controller	Routes	What it does	Frontend pages	Models · services
TpoInterview	13	TPO private mock AI interviews	/admin/mock-interviews	Interview, InterviewQuestion, InterviewResponse, InterviewSession; InterviewProctoringService, InterviewTrackAdvancementService
TpoContest	9	TPO private mock contests (tpo_mock)	/admin/mock-contests	Contest, ContestProblem, ContestSubmission, User; ContestFinalizeService, ContestReportService
TpoDrive	9	TPO: map/unmap catalog drives, respond to proposals, create campus drive	/admin, /admin/drives	ActivityLog, Company, DriveCollegeMapping, PlacementDrive
TpoSoftSkill	8	TPO private soft-skills tests	/admin/mock-soft-skills	SoftSkillAssessment
StudentImport	4	Bulk student CSV import (TPO self-serve + Ops on behalf)	—	College, StudentImport; StudentImportService
TpoCoordinator	4	TPO manages Section Coordinators	/admin, /admin/coordinators, /admin/reports +1	ActivityLog, User; SectionCoordinatorService
TpoDriveApplication	4	TPO records each student's progress through a drive	/admin/drives	DriveApplication, PlacementDrive, User; DrivePipelineService
TpoInterviewTrack	4	TPO private 'Final Interview' tracks	—	InterviewRoleTemplate, InterviewTrack
TpoInterviewRoleTemplate	3	TPO-private role templates	—	InterviewRoleTemplate
TpoProctoring	3	College-wide proctoring review + reinstate	/admin/proctoring	ProctoringSession, ProctoringViolation; ProctoringService
TpoStudent	3	Student cohort: roster, add student, bulk notify	/admin/drives, /admin/students	User; StudentBulkNotificationService, StudentCohortService
TpoStudentReport	3	Per-student report + code viewer (own college)	—	ContestSubmission, Submission, User; StudentReportService
AdminSoftSkillQuestionBank	2	Soft-skill question bank incl. Gemini generation (also mounted for TPO/company)	—	SoftSkillQuestion; GeminiSoftSkillQuestionGeneratorService
TpoReports	2	Placement reports data + placement target	/admin, /admin/analytics, /admin/reports	College, DriveCollegeMapping, PlacementDrive, User; EligibilityService, PlacementReportService

section_coordinator /coordinator/* — 9 routes, 3 controllers

Controller	Routes	What it does	Frontend pages	Models · services
CoordinatorProctoring	3	Section-scoped proctoring review	/coordinator/proctoring	ProctoringSession, ProctoringViolation; ProctoringService
CoordinatorStudent	3	Section roster: list, block/unblock, bulk notify	/coordinator, /coordinator/reports	User; StudentBulkNotificationService, StudentCohortService
CoordinatorStudentReport	3	Section-scoped student report + code viewer	—	ContestSubmission, Submission, User; StudentReportService

admin_company /company/* — 88 routes, 13 controllers

Controller	Routes	What it does	Frontend pages	Models · services
CompanyHackathon	20	Company-hosted hackathons (ManagesHackathons trait)	/admin/company/hackathons, /admin/company/hackathons/edit, /admin/company/hackathons/manage	Hackathon
CompanyInterview	14	Company AI interviews: CRUD, invite, review/score, finalize	/admin/company/interviews	DriveApplication, Interview, InterviewQuestion, InterviewResponse ...; InterviewProctoringService, InterviewTrackAdvancementService
CompanyContest	11	Company hiring assessments: CRUD, problems, invite, results, import qualifiers	/admin/company/assessments	Contest, ContestParticipant, ContestProblem, ContestSubmission ...; ContestFinalizeService
CompanySoftSkill	8	Company soft-skills tests	/admin/company/soft-skills	SoftSkillAssessment
CompanyTalentPool	8	Talent Pool marketplace: search, interest, interview, hire, notify	/admin/company/talent-pool	TalentPoolCandidate, TalentPoolInquiry, TalentPoolMessage
CompanyCandidate	7	Candidate pipeline per job opening; single add + CSV import	/admin/company/assessments, /admin/company/candidates, /admin/company/interviews	CandidateImport, DriveApplication, PlacementDrive; CandidateImportService, DrivePipelineService
CompanyDrive	6	Company job openings; propose an opening to colleges	/admin, /admin/company/assessments, /admin/company/candidates +4	DriveApplication, PlacementDrive; DriveCollegeProposalService
CompanyInterviewTrack	4	Company 'Final Interview' tracks (invite-only)	—	InterviewRoleTemplate, InterviewTrack, PlacementDrive
CompanyInterviewRoleTemplate	3	Company-private role templates	—	InterviewRoleTemplate
CompanyProctoring	3	Review flagged proctoring across the company's assessments	/admin/company/proctoring	ProctoringSession, ProctoringViolation; ProctoringService
AdminSoftSkillQuestionBank	2	Soft-skill question bank incl. Gemini generation (also mounted for TPO/company)	—	SoftSkillQuestion; GeminiSoftSkillQuestionGeneratorService
CompanyCollege	1	Browse partner colleges (aggregate figures only)	/admin/company/colleges, /admin/company/drives, /admin/company/talent-pool	College, User
CompanyReports	1	Hiring Command Center + Hiring Reports data (one call)	/admin, /admin/company/reports	Company; HiringReportService

admin_internal /admin/* (Mellow Ops — superadmin may call these too) — 132 routes, 19 controllers

Controller	Routes	What it does	Frontend pages	Models · services
AdminHackathon	20	Host/judge hackathons (all logic in the ManagesHackathons trait)	/admin/hackathons, /admin/hackathons/edit, /admin/hackathons/manage	Hackathon
Admin	14	Platform ops: overview, colleges + TPO onboarding, platform users, coordinators, seats & plans	/admin, /superadmin	ActivityLog, Article, College, Company ...; SectionCoordinatorService, SubscriptionService
AdminInterview	14	Platform AI interviews: CRUD, questions, review/score sessions, finalize	/admin/interviews, /admin/talent-pool	College, CompanyRecommendedInterviewQuestion, DriveCollegeMapping, Interview ...; InterviewProctoringService, InterviewTrackAdvancementService
AdminCompany	11	Company catalog, prep Q&A, recommended problems; provisions a company's admin login + plan	/admin/contests, /admin/interviews, /admin/interviews/tracks +1	ActivityLog, Company, CompanyPrepQuestion, CompanyRecommendedProblem ...; SubscriptionService

Controller	Routes	What it does	Frontend pages	Models · services
AdminContest	10	Platform contests: CRUD, problem set, college audience, finalize, participant report	/admin/contests, /admin/talent-pool	College, CompanyRecommendedProblem, Contest, ContestProblem ...; ContestFinalizeService, ContestReportService
AdminProjectLab	9	Project Lab topics/SOW/milestones + student submission review	/admin/projects, /admin/projects/edit, /admin/projects/review	ActivityLog, ProjectEnrollment, ProjectSubmission, ProjectTopic; SowPdfService
AdminSoftSkill	8	Platform soft-skills tests and their questions	/admin/soft-skills	SoftSkillAssessment
AdminPlacementDrive	6	Catalog placement drives; propose/map drives to colleges	/admin/contests, /admin/interviews, /admin/interviews/tracks +1	College, PlacementDrive; DriveCollegeProposalService
AdminArticle	5	Knowledge-base articles (markdown) — CRUD	/admin/articles, /admin/articles/edit, /admin/articles/new	Article, ArticleTopic
AdminInterviewTrack	5	'Final Interview' 3-round pipeline container (platform-owned)	—	College, DriveCollegeMapping, InterviewRoleTemplate, InterviewTrack ...
AdminArticleTopic	4	Article topics / reading paths — CRUD	/admin/articles, /admin/articles/edit, /admin/articles/new	ArticleTopic
AdminSoftSkillQuestionBank	4	Soft-skill question bank incl. Gemini generation (also mounted for TPO/company)	—	SoftSkillQuestion; GeminiSoftSkillQuestionGeneratorService
StudentImport	4	Bulk student CSV import (TPO self-serve + Ops on behalf)	—	College, StudentImport; StudentImportService
AdminInterviewQuestionBank	3	Interview question bank — authoring	/admin/interviews/question-bank	InterviewQuestionBank
AdminInterviewRoleTemplate	3	Role templates driving the 3-round Final Interview tracks	—	InterviewRoleTemplate
AdminProblem	3	Problem bank authoring (function signature + test cases → generated harness)	/admin	ActivityLog, Problem, Submission; JudgeService, ProblemType
AdminStudentReport	3	Any student's full activity report (platform-wide)	—	ContestSubmission, Submission, User; StudentReportService
AdminTalentPool	3	Talent Pool marketplace oversight (visibility)	/admin/talent-pool	ActivityLog, College, TalentPoolCandidate
InternalCustomer	3	Mellow Internal: converted-customer book + notes	/admin/customers	ActivityLog, LeadNote, User; StudentStatsService

admin_marketing /marketing/* — 9 routes, 2 controllers

Controller	Routes	What it does	Frontend pages	Models · services
MarketingLead	5	Marketing: Mellow Direct leads — status, notes, notify	/marketing, /superadmin	LeadNote, User; MarketingLeadService, StudentStatsService
MarketingContactRequest	4	Marketing: 'Talk to Our Team' request queue	/marketing/inquiries	ContactRequest, ContactRequestNote, User; ContactRequestService

superadmin /superadmin/* — 11 routes, 1 controllers

Controller	Routes	What it does	Frontend pages	Models · services
SuperAdmin	11	Superadmin: staff accounts, permissions, audit log, judge nodes, feature flags	/superadmin	ActivityLog, College, Company, ContactRequest ...; JudgeStatusService, LeadAssignmentService, StudentStatsService

Shared by Ops, TPO, company and superadmin — 2 routes, 2 controllers

Controller	Routes	What it does	Frontend pages	Models · services
`InterviewQuestionBank`	1	Read-only bank browse (Ops/TPO/Company/Superadmin)	`/admin/company/interviews`, `/admin/interviews`, `/admin/mock-interviews` +1	InterviewQuestionBank
`InterviewQuestionGeneration`	1	Gemini question generation into the bank (Ops/TPO/Company)	`/admin/company/interviews`, `/admin/interviews`, `/admin/interviews/tracks` +1	CompanyRecommendedInterviewQuestion, InterviewQuestionBank; GeminiQuestionGeneratorService

4.4 The mirror pattern — why one feature appears 2–3 times

A feature that several roles can author is implemented as a **family of near-identical controllers**, one per role, each hard-wired to its own scope. The frontend mostly reuses **one console component** and switches the API prefix with a `basePath` prop. This keeps each role's security rules local and obvious, at the price that **a behavioural change must be repeated in every sibling**.

Feature	Ops (`/admin`)	TPO (`/tpo`)	Company (`/company`)	Shared frontend piece
Contests	AdminContestController	TpoContestController	CompanyContestController	per-role pages; `ContestParticipantsModal`
AI interviews	AdminInterviewController	TpoInterviewController	CompanyInterviewController	`ReviewSessionsModal`, `ManageInterviewQuestionsModal` (prop `basePath`)
Interview tracks	AdminInterviewTrackController	TpoInterviewTrackController	CompanyInterviewTrackController	`CreateInterviewTrackModal`, `RoundConfigEditor`
Role templates	AdminInterviewRoleTemplateController	TpoInterviewRoleTemplateController	CompanyInterviewRoleTemplateController	`RoleTemplateModal`
Soft skills	AdminSoftSkillController	TpoSoftSkillController	CompanySoftSkillController	`SoftSkillAdminConsole` (props `basePath`, `bankBasePath`, `allowedRoles`); trait `ManagesSoftSkillQuestions`
Hackathons	AdminHackathonController	— (TPOs cannot host)	CompanyHackathonController	`components/hackathons/host/*` via `hostApi(scope)`; trait `ManagesHackathons`
Proctoring review	—	TpoProctoringController (college) · CoordinatorProctoringController (section)	CompanyProctoringController	`ProctoringSessionList` (prop `apiBasePath`)
Student report	AdminStudentReportController	TpoStudentReportController · CoordinatorStudentReportController	—	`/admin/students/report` page (one page, all 4 roles)
Student roster	AdminController::users	TpoStudentController	— (Coordinator: CoordinatorStudentController)	`StudentProfileDrawer`, `BulkNotifyModal`
Question banks	AdminInterviewQuestionBankController, AdminSoftSkillQuestionBankController	same controllers, read/generate routes mounted under `/tpo`	same controllers mounted under `/company`	`GenerateQuestionsPanel`

> **CAUTION Checklist when you change a mirrored feature**
> 1) Find all siblings in the table above. 2) Change the shared service/trait/model method first if one exists. 3) Apply the change in each controller (the scope filter is the part that differs). 4) Find the frontend consumers (Appendix B shows which pages call which controller) and check the `basePath` they pass. 5) Run the sibling Feature tests: `AdminProblemControllerTest`, `InterviewTrackCreationTest`, `SoftSkillVisibilityTest`, `HackathonHostingTest` …

4.5 Services — the “one write path” layer

A service owns one business concept and is the only code allowed to write it (the file headers say so repeatedly: “the one write path for …”). When you need to change a rule — pricing, eligibility, lock thresholds, scoring — **change the service, not a controller**, so every caller stays consistent.

Accounts, roster & leads

Service	LOC	Responsibility
`PasswordResetService`	113	Forgot-password OTP: hashed 6-digit code, 10-minute life, 5 attempts, one-time reset token; revokes all tokens on reset
`FirebasePhoneVerificationService`	105	Verifies a Firebase ID token against Google's cached JWKS (issuer + audience) — no Admin SDK
`SectionCoordinatorService`	184	Create / update / block Section Coordinators — one rule shared by TPO and Ops callers
`StudentImportService`	64	Bulk roster CSV import (with `ProcessStudentImportJob`); one implementation for TPO self-serve and Ops-on-behalf
`StudentCohortService`	70	Builds one cohort-table row: readiness score/tier, 7-day consistency, real per-drive eligibility
`StudentBulkNotificationService`	70	Fans an email or WhatsApp notification out to a chosen set of students (one queued job each)
`LeadAssignmentService`	223	Least-loaded assignment of marketing owner (and internal owner on conversion) for Mellow Direct leads
`MarketingLeadService`	95	Lead status changes, notes and outreach for the marketing team
`ContactRequestService`	84	Write path of the public “Talk to Our Team” form (+ assignment, confirmation/notification mail)

Placement & hiring

Service	LOC	Responsibility
`DriveCollegeProposalService`	90	“Propose a drive to these colleges, pending their approval” — shared by Ops and companies; skips already-mapped colleges
`DrivePipelineService`	116	The one write path for a drive application's stage (registered → … → offer_accepted / rejected / withdrawn)
`EligibilityService`	84	One student vs one drive's requirements (CGPA, backlogs, branch …) → verdict used by student, TPO and report views
`PlacementReportService`	261	TPO report numbers derived from real `drive_applications` rows (funnel, packages, branches)
`HiringReportService`	218	Same idea scoped to one company's direct-hire drives
`CandidateImportService`	74	The one place a company candidate account is found-or-created and put in a job opening's pipeline (+ `ProcessCandidateImportJob`)
`WeeklyReadinessReportService`	104	Builds the weekly “students below 60% readiness” Excel and mails it to a college's TPOs

Analytics

Service	LOC	Responsibility
`ReadinessScoreService`	194	Readiness Score v2 — interview 35 %, contest rating 20 %, practice depth 20 %, consistency 15 %, academic 10 %
`StudentStatsService`	254	Dashboard / practice numbers: solved counts, streaks, topic mastery, heatmap — all real queries
`StudentReportService`	298	A student's full activity picture (contests, interviews, drives, calendar, code) for TPO / coordinator / Ops

Practice, judge & contests

Service	LOC	Responsibility
`JudgeService`	286	Single problem-judging pipeline: generate harness → run → compare per case → classify verdict (hidden cases revealed pass/fail only)
`CodeExecutionService`	166	Thin client around the Piston pool; knows nothing about problems or test cases
`ContestScoringService`	62	Recomputes a participant's score/penalty from their whole submission history (idempotent)
`ContestFinalizeService`	186	Closes a contest: final scores, ranks, rating changes, talent-pool qualification; idempotent via `finalized_at`
`ContestRatingService`	77	Simplified Elo-style rating delta for rated contests
`ContestReportService`	83	Participant / per-problem report a contest's creator sees
`TalentPoolQualificationService`	89	Decides whether a talent_pool contest score qualifies a candidate and creates/updates their marketplace row

Interviews, soft skills & proctoring

Service	LOC	Responsibility
`GeminiQuestionGeneratorService`	223	Gemini-written interview questions persisted into the bank
`GeminiInterviewScoringService`	176	One batched Gemini call scores a completed session (run by `ScoreInterviewSessionJob`)
`InterviewTrackAdvancementService`	196	Decides a track round's outcome and auto-invites passers to the next round
`SoftSkillGradingService`	73	Exact-match grading of an attempt against the bank's `correct_index` (never AI)
`GeminiSoftSkillQuestionGeneratorService`	146	Gemini-written soft-skill questions for the bank
`ProctoringService`	211	Contest proctoring authority: consent, violations, lock/auto-submit, reinstate, flagged-session queries
`InterviewProctoringService`	125	Same bookkeeping for interviews (lock ends the attempt; nothing to auto-submit)
`SoftSkillProctoringService`	133	Same bookkeeping for soft-skills attempts

Learning Centre

Service	LOC	Responsibility
`GeminiSpeakingPassageService`	170	Generates a read-aloud passage around a student's topic and level
`GeminiSpeakingScoringService`	452	Scores one recording from the audio itself (never the browser transcript) + deterministic alignment
`GeminiListeningLessonService`	379	Writes a listening lesson/dialogue + questions; grading stays deterministic
`ListeningProgressService`	225	All Listening Lab progress numbers in one place
`GeminiVocabularyQuizService`	228	On-demand vocabulary-in-context quiz for any topic
`VocabularySessionService`	282	Chooses what a sprint contains (due reviews + a few new words)
`VocabularyQuestionFactory`	292	Turns a word into a question; harder types as memory strengthens (no AI)
`VocabularyAnswerService`	346	Grades one answer at a time and reveals the key only after commitment
`VocabularySrsService`	121	Six-box Leitner schedule (1, 3, 7, 16, 35 days)
`VocabularyProgressService`	282	Vocabulary home, word-bank and deck totals computed fresh

Billing & messaging

Service	LOC	Responsibility
`SubscriptionService`	389	The one write path for subscription state: activate, renew, change plan, institution/custom plans, seat limits, demo trials
`PurchaseQuoteService`	133	The one place a price is decided (plan + add-on + bundle discount); the browser is never the source of a price
`WhatsAppService`	65	Provider-agnostic WhatsApp sender (`WHATSAPP_PROVIDER=log` writes to the log instead of sending)

Packages

Folder	Contents
`app/Services/Judge/ (7)`	JudgeQueue (entry point, caching, dedupe, depth limits), PistonPool (balance/breaker/failover), JudgeRequest, JudgeResultStore, JudgeOutcomePersister, JudgeStatusService, JudgeUnavailableException
`app/Services/ProblemCodeGenerator/ (4)`	HarnessGenerator (966 LOC — test-driver per language), StarterCodeGenerator, LiteralEmitter, ProblemType (closed set of signature types)
`app/Services/ProjectLab/ (4)`	ProjectLabAccess (402 gate), ProjectLabPresenter (preview vs full JSON), ProjectSubmissionService (snapshot + review queue), SowPdfService (dompdf, watermarked)
`app/Services/Workspaces/ (17)`	WorkspaceManager (543 LOC control plane), WorkspaceRuntime + DockerCliRuntime / FakeWorkspaceRuntime, WorkspaceHosts, SnapshotScript, GitHistoryScript, AuthorshipReport, WorkspaceIntegrity, WorkspaceStarterFiles, value objects
`app/Services/Hackathons/ (5)`	HackathonRegistrationService (race-safe seats/teams), HackathonSubmissionService, HackathonResultsService (weighted score → rank), HackathonPresenter (what each audience may see), HackathonRuleException

4.6 Queued jobs and mail

Jobs implement `ShouldQueue`; the mail jobs share a `RateLimited('transactional-emails')` middleware (60/min) to protect the SMTP provider during bulk fan-outs. Four jobs — `SendAccountCredentialsEmail`, `SendPasswordResetOtpEmail`, `SendWelcomeEmail` and `SendStudentBulkNotification` — are `ShouldBeEncrypted`, so their payload is not readable in the `jobs` table.

Job	Dispatched from	Sends / does
`JudgeSubmissionJob`	JudgeQueue	Judges one Run/Submit; queue `judge-contest` > `judge-submit` > `judge-run`; retries for 240 s on infrastructure failure
`ProcessCandidateImportJob`	CandidateImportService	Same for a company's candidate CSV
`ProcessStudentImportJob`	StudentImportService	Parses a roster CSV in the background (bcrypt for ~1,900 accounts is CPU-bound)
`ScoreInterviewSessionJob`	InterviewController	Gemini-scores a completed interview session
`SendAccountCredentialsEmail`	AdminCompany, Admin, CompanyCandidate, TpoStudent, SuperAdmin controllers; SectionCoordinatorService	AccountCredentialsMail
`SendCandidateInviteEmail`	CompanyCandidateController	CandidateInviteMail
`SendContactRequestConfirmation`	ContactRequestService	ContactRequestConfirmationMail
`SendContactRequestNotification`	ContactRequestService	ContactRequestNotificationMail
`SendDriveMappingProposalEmail`	DriveCollegeProposalService	DriveMappingProposalMail
`SendInterviewShortlistEmail`	CompanyInterviewController	InterviewShortlistMail
`SendNewsletterWelcomeEmail`	NewsletterController	NewsletterWelcomeMail
`SendPasswordResetOtpEmail`	PasswordResetService	PasswordResetOtpMail
`SendProctoringViolationReport`	ProctoringService, ContestFinalizeService	ProctoringViolationReportMail
`SendStudentBulkNotification`	StudentBulkNotificationService, MarketingLeadService	StudentBulkNotificationMail / PlacementDriveTerminationMail
`SendTalentPoolHireOfferEmail`	CompanyTalentPoolController	TalentPoolHireOfferMail
`SendTalentPoolInterestEmail`	CompanyTalentPoolController	TalentPoolInterestMail
`SendTalentPoolInterviewInviteEmail`	CompanyTalentPoolController	TalentPoolInterviewInviteMail
`SendTalentPoolInterviewResponseEmail`	StudentTalentPoolController	TalentPoolInterviewResponseMail
`SendTalentPoolOutreachEmail`	CompanyTalentPoolController	TalentPoolOutreachMail
`SendTalentPoolQualifiedEmail`	TalentPoolQualificationService	TalentPoolQualifiedMail
`SendWelcomeEmail`	AuthController@register	WelcomeMail
`SendWhatsAppNotification`	StudentBulkNotificationService	(WhatsAppService, no mailable)

Two mailables are not tied to their own job: `WeeklyReadinessReportMail` (sent by the weekly readiness report) and `PlacementDriveTerminationMail` (a template the bulk-notify job can choose). Locally `MAIL_MAILER` is `smtp`; set it to `log` to read mails in `storage/logs` instead of sending them.

4.7 Scheduler and artisan commands

`routes/console.php` registers the schedule. **Nothing fires unless the server runs** `* * * * * php artisan schedule:run` every minute.

Command / task	Schedule	What it does
`workspaces:reap`	every minute (no overlap)	Stops Project Lab workspaces that are idle (15 min), lapsed, out of hours, over the session/disk cap or whose container died; stops orphan containers
`contests:finalize`	every 5 minutes	Scores/rates every published contest whose `end_at` has passed (idempotent)
`contests:generate-daily`	daily 00:05 Asia/Kolkata, one server	Creates today's platform "Daily Challenge" (1 easy + 1 medium + 1 hard)
`subscriptions:expire`	daily	Flips subscriptions past `current_period_end` to `expired`
`authorship:prune` (closure)	daily	Deletes `project_workspace_integrity_events` older than `authorship.retention_days` (180)
`report:weekly-readiness`	weekly	Emails each college's TPOs an Excel list of students below 60 % readiness

Command / task	Schedule	What it does
`superadmin:sync`	manual	Creates/updates the superadmin from `SUPERADMIN_EMAIL` / `SUPERADMIN_PASSWORD`; `--update-password` to rotate
`judge:status [--json]`	manual / uptime probe	Pool + queue snapshot; exits non-zero when no execution node is healthy
`judge:loadtest-seed`	manual	Throw-away students + tokens for `infra/loadtest`; `--force` needed when `APP_ENV=production`
`speaking:purge-audio [--dry-run]`	manual, one-off	Deletes legacy speaking recordings stored before scoring became audio-first

4.8 Rate limiters

Defined in `AppServiceProvider::boot()`. Judge, proctoring, AI, lab and arena limits are **per user, never per IP** — a campus lab shares one NAT address, so an IP limit would throttle a whole class as one person. Each Project Lab route family has its **own named limiter** so that polling does not spend the Start button's allowance.

Limiter	Limit	Applied to
login	5/min per email + 20/min per IP	`POST /login`
password-reset-request	3/min per email + 10/min per IP	`POST /password/forgot`
password-reset-verify	10/min per email + 20/min per IP	`POST /password/otp/verify`, `/password/reset`
contact-request	5/hour per email + 10/hour per IP	`POST /contact-requests`
newsletter-subscribe	5/hour per email + 20/hour per IP	`POST /newsletter/subscribe`
judge-run / judge-submit / judge-poll	20 / 10 / 180 per min per **user** (from `config/judge.php`)	practice + contest run/submit, `GET /judge/{token}`
proctoring-event	30/min per user	all three `…/proctoring/violations` routes
ai-generation	5/min per user	Gemini generation: interview + soft-skill questions, speaking/listening/vocabulary generation, speaking attempts
phone-verify	10/min per user	`POST /me/phone/verify`
lab-topic-start · lab-sow · lab-purchase	20/min per user each	Project Lab start, SOW PDF, purchase
lab-milestone	60/min	milestone toggle
lab-workspace-start · -poll · -heartbeat · -stop	10 · 120 · 30 · 20 per min	workspace lifecycle (separate counters on purpose)
lab-integrity · lab-submit	30 · 5 per min	integrity reports, project submit
arena-action · arena-submit · arena-host	30 · 30 · 120 per min per user	Hack Arena writes / project save / host console
throttle:60,1 (unnamed)	60/min	`/public/*` (per IP), vocabulary session start; `throttle:240,1` on vocabulary answer
transactional-emails (job middleware)	60/min	all `Send*Email` jobs via `RateLimited`

4.9 Configuration files

File	What it controls
app.php / auth.php / session.php / logging.php / filesystems.php / mail.php	Laravel standard files; values come from `.env` (`APP_*`, `MAIL_*`, `FILESYSTEM_DISK`)
`sanctum.php`	Token auth. `expiration` is `null` (tokens never expire); guard `web`
`cors.php`	Allowed origins are **hard-coded** to `http://localhost:3000` and `http://127.0.0.1:3000`; `supports_credentials=false`
database.php / queue.php / cache.php	MySQL default in `.env`; queue + cache + sessions default to the `database` driver; Redis (predis) in production
`judge.php`	Queue names, 64 KB code cap, 3 active jobs/user, per-user rate limits (run 20 · submit 10 · poll 180 per min), backpressure depths (run 2,000 · submit 5,000 · contest 50,000), result TTL 15 min, job timeout 60 s
`piston.php`	Node pool (`PISTON_NODES=url|slots,…`), HTTP/connect timeouts, circuit breaker, run wall 7 s / CPU 3 s, compile 15 s / 10 s, language → Piston runtime/version/filename map (PHP uses a separate solution file)
`proctoring.php`	3 violations before lock, auto-submit on lock, soft-skills proctoring switch
`contests.php`	Daily contest: timezone Asia/Kolkata, starts 19:00, 60 min, points 100/200/300
`billing.php`	Add-on bundle discount 30 %, currency INR (prices themselves live on `plans` rows)

File	What it controls
`workspaces.php`	Docker runtime (`docker` vs `fake`), host list, image/network, per-container limits (2 GB, 1.5 CPU, 512 pids), idle 15 min, max session 480 min, snapshot limits
`authorship.php`	Paste-guard mode (`enforce` / `report` / `off`), allowed event kinds, 180-day retention, the rule thresholds behind the reviewer's authorship label
`services.php`	`gemini` (key, model `gemini-3.1-flash-lite`), `firebase` public config, mail provider keys
`whatsapp.php`	`WHATSAPP_PROVIDER` — `log` by default

> **CAUTION config/cors.php is hard-coded**
> `allowed_origins` lists only `localhost:3000` and `127.0.0.1:3000` and does not read an env variable. A browser on your production frontend origin will have its API calls blocked by CORS unless that origin is added (or the file reads an env var). Verify this before the first production cut-over.

PART V

Feature domains

Sixteen chapters, one per product area. Each lists the roles involved, the flow, the exact controllers / services / tables, the frontend pages and components, and the rules that are easy to break.

5.1 Colleges, roster & section coordinators

How a college enters the platform, how its students get accounts, and how day-to-day roster work is delegated.

Roles	`admin_internal` (permission `colleges`) · `superadmin` · `admin_tpo` · `section_coordinator`
Tables	colleges · users · student_imports · activity_logs · subscriptions

How it works

- **Onboard a college.** Ops submits `POST /admin/colleges`; one DB transaction creates the `colleges` row and its TPO user with a random 16-char password and `must_change_password = true`; then the chosen tier (`Standard`, `Pro Campus`, `Academic Enterprise`, or `Custom` seats/price) becomes the college's institution plan immediately. `is_demo` + `demo_days` (max 90) shortens the term for a trial. A credentials email is queued.
- **Seats.** `PUT /admin/colleges/{college}/seats` changes the seat limit without touching renewal dates (`SubscriptionService::adjustSeatLimit`). `College::studentLimit()` is read at every enforcement point; `null` means unlimited (Academic Enterprise).
- **Roster import.** TPO (`/tpo/students/import`) or Ops on the college's behalf (`/admin/colleges/{college}/students/import`) upload a CSV (template from `…/import-template`). `StudentImportService` validates and `ProcessStudentImportJob` creates/updates accounts in the background, then one `SendAccountCredentialsEmail` per new student. Import history: `student_imports` rows (`indexMine` / `showMine`).
- **Cohort.** `GET /tpo/students/cohort` returns each student's readiness score/tier, 7-day consistency and per-drive eligibility (`StudentCohortService`); `POST /tpo/students` adds one; `POST /tpo/students/bulk-notify` fans out email/WhatsApp.
- **Coordinators.** TPO creates a `section_coordinator` bound to one `section` (`/tpo/coordinators`); the coordinator sees only students whose `section` matches (`CoordinatorStudentController`), can block/unblock and notify, but cannot edit or add anyone.
- **Student report.** One drill-down (contest, interview and drive history, activity calendar, submission code) served by three role-scoped controllers: `TpoStudentReportController`, `CoordinatorStudentReportController`, `AdminStudentReportController`.

Backend map

Controllers	`AdminController` (colleges, tpos, coordinators, users, seats, subscription) · `StudentImportController` · `TpoStudentController` · `TpoCoordinatorController` · `CoordinatorStudentController` · `Tpo/Coordinator/AdminStudentReportController`
Services	`StudentImportService` · `SectionCoordinatorService` · `StudentCohortService` · `StudentBulkNotificationService` · `SubscriptionService` · `StudentReportService` · `EligibilityService`
Jobs / mail	`ProcessStudentImportJob` · `SendAccountCredentialsEmail` (encrypted, rate-limited) · `SendStudentBulkNotification` · `SendWhatsAppNotification`
Models / tables	`College` (colleges) · `User` (users: `college_id`, `section`, `roll_number`, `branch`, `cgpa`, `backlogs`) · `StudentImport` · `ActivityLog`

Frontend map

Ops UI	`/admin?view=mellow&tab=colleges` → `PartnerCollegesPanel` + `CollegeCoordinatorsModal` (same panel is reused in `/superadmin?tab=colleges`)
TPO UI	`/admin/students` (cohort; `StudentProfileDrawer`, `AddStudentModal`, `BulkImportStudentsPanel`, `BulkNotifyModal`) · `/admin/coordinators` (`AddCoordinatorModal`) · `/admin/students/report`

Coordinator UI	`/coordinator` (roster) · `/coordinator/reports`
Shared libs	`lib/generateStudentsExcel.ts`, `lib/generateCoordinatorReports.ts` (client-side Excel/PDF), `types/cohort.ts`

Rules and gotchas

- Student **sections** are free text set by the TPO; a coordinator's access is *exactly* `users.section == coordinator.section` within the same `college_id`.
- Academic fields (`roll_number`, `branch`, `cgpa`, `backlogs`) are the college's source of truth: `AuthController@updateProfile` deliberately does not let a student edit them.
- Bulk mail is throttled by the `transactional-emails` limiter (60/min); a 2,000-student import therefore takes ~30+ minutes to finish sending — workers must stay up.

5.2 Placement drives & the application pipeline

The recruiter-visit workflow: a shared company/drive catalog, per-college approval, and a TPO-maintained pipeline per student.

Roles	`admin_internal` (`placements`) · `admin_tpo` · `admin_company` · `user` (read-only)
Tables	companies · placement_drives · drive_college_mappings · drive_applications · company_prep_questions · company_recommended_*

How it works

- **Catalog.** Ops curates `companies` (+ prep Q&A and recommended problems) and schedules `placement_drives` (`source = catalog`, status draft → published → completed/cancelled).
- **Getting a drive to a college — three ways.** (a) the TPO self-maps a published drive (`POST /tpo/drives/{drive}/map`); (b) Ops pushes it (`POST /admin/placement-drives/{drive}/map-colleges`) which creates a **pending** `drive_college_mappings` row and emails the TPO (`SendDriveMappingProposalEmail`), who must approve or decline (`/tpo/drives/pending`, `…/respond`); (c) a company proposes its own opening (`POST /company/drives/{drive}/propose`) — same pending/approve mechanism via `DriveCollegeProposalService`.
- **Campus-only drives.** A TPO can create their own drive (`POST /tpo/drives`, `source = tpo_created`) for one college.
- **Students do not apply themselves.** `StudentDriveController` is read-only: a student sees published drives approved for their college, with a per-drive eligibility verdict (`EligibilityService`: CGPA, backlogs, branch …). The TPO registers students and moves them through stages.
- **Pipeline.** `drive_applications.stage`: `registered → online_test → technical_interview → hr_round → offer_extended → offer_accepted`, plus `rejected` / `withdrawn`. `DrivePipelineService` is deliberately **not** a strict state machine (real TPOs skip rounds and backfill outcomes).
- **Entitlement.** Individual (non-college) students on the free Coder plan have `drive_access = false` and see an empty list.

Backend map

Controllers	`AdminCompanyController` · `AdminPlacementDriveController` · `TpoDriveController` · `TpoDriveApplicationController` · `StudentDriveController` · `CompanyDriveController` · `TpoReportsController`
Services	`DriveCollegeProposalService` · `DrivePipelineService` · `EligibilityService` · `PlacementReportService`
Jobs / mail	`SendDriveMappingProposalEmail` → `DriveMappingProposalMail`; `PlacementDriveTerminationMail` via bulk notify
Models / tables	`Company` · `PlacementDrive` · `DriveCollegeMapping` (`pending/approved/declined`, `is_active`) · `DriveApplication` · `CompanyPrepQuestion` · `CompanyRecommendedProblem` · `CompanyRecommendedInterviewQuestion`

Frontend map

Ops UI	`/admin/placements` → `CompaniesPanel`, `DrivesPanel`, `MapCollegesModal`, `PrepQuestionsPanel`, `RecommendedProblemsPanel`
TPO UI	`/admin/drives` (`CreateDriveModal`, `DrivePipelineBar`, `AddApplicantModal`) · `/admin/reports` · `/admin/analytics`
Student UI	`/dashboard/drives?…` (detail, eligibility, prep questions) and the "Placement drives" section of `/dashboard`
Company UI	`/admin/company/drives` (`CreateJobOpeningModal`, `ProposeToCollegesModal`) · `/admin/company/colleges`

Rules and gotchas

- A mapping must be `approved` **and** `is_active` for dependent content (company contests, interviews) to become visible to that college — see `Contest::isVisibleToCollege()`.
- `drive_applications.college_id` is a **snapshot** taken at creation and is nullable (company-direct candidates have no college).
- TPO scope: every method derives `college_id` from the authenticated user; never accept it from the request.

5.3 Hiring-partner (company) portal

A company tenant's own workspace: job openings, candidate pipeline, proctored assessments, AI interviews, the Talent Pool and hackathons.

Roles	`admin_company` (provisioned by `admin_internal` with `placements`)
Tables	companies · placement_drives · drive_applications · candidate_imports · users

How it works

- **Tenant creation.** Ops adds a company with `POST /admin/companies`; this always provisions its `admin_company` login (`AdminCompanyController::store`) and can assign a company plan (`Hiring Starter` / `Hiring Growth`). `companies.account_type` = `hiring_tenant`.
- **Job opening** = `placement_drives` with `source = company_direct`, scoped by `company_id` (`CompanyDriveController`).
- **Candidates** are ordinary `users` (role `user`, `college_id` null, `invited_by_company_id` set) added one by one or by CSV (`CompanyCandidateController`, `CandidateImportService`, `ProcessCandidateImportJob`); each lands in the opening's `drive_applications` pipeline and receives an invite email.
- **Assess → interview → hire.** Typical flow: create a `company_hiring` contest, invite candidates, read score-ranked results, import qualifiers into the pipeline, create an AI interview and invite the same shortlist; `updateStage` moves people towards an offer (no separate "hire" endpoint).
- **Reports.** `GET /company/reports/data` returns everything for the Hiring Command Center and Hiring Reports in one call (`HiringReportService`).

Backend map

Controllers	`CompanyDriveController` · `CompanyCandidateController` · `CompanyContestController` · `CompanyInterviewController` · `CompanyInterviewTrack/RoleTemplateController` · `CompanyProctoringController` · `CompanyReportsController` · `CompanyCollegeController` · `CompanyTalentPoolController` · `CompanySoftSkillController` · `CompanyHackathonController`
Services	`CandidateImportService` · `DrivePipelineService` · `HiringReportService` · `DriveCollegeProposalService` · `ContestFinalizeService` · `InterviewTrackAdvancementService`
Jobs / mail	`SendCandidateInviteEmail` · `SendInterviewShortlistEmail` · `ProcessCandidateImportJob` · `SendAccountCredentialsEmail`
Scope rule	`company_id = $request->user()->company_id`; contests/interviews filtered by `contest_type|interview_type = company_hiring AND owning_company_id = company_id`

Frontend map

Pages	`/admin?view=company` (`CompanyCommandCenter`) · `/admin/company/drives` · `/candidates` · `/assessments` · `/interviews` · `/soft-skills` · `/hackathons` · `/talent-pool` · `/reports` · `/proctoring` · `/colleges`
Components	`components/dashboard/company/*` (`AddCandidateModal`, `BulkImportCandidatesPanel`, `InviteToAssessmentModal`, `InviteToInterviewModal`, `AssessmentResultsModal`), `components/portal/*` kits, `lib/generateHiringReports.ts`

Rules and gotchas

- Company contests/interviews are **invite-only**; a candidate cannot see them until `invited_at` is set, regardless of college.
- The Hiring Command Center and the candidate list share the pipeline stages with the TPO's — change `DriveApplication::STAGE_*` in one place only.

5.4 Practice Arena & the code judge

The LeetCode-style coding practice: a problem bank, a Monaco editor, and a queued, sandboxed judge. Contests reuse the same pipeline.

Roles	`user` (practice) · `admin_internal` (`problem_bank`) · `superadmin` (telemetry)

How it works

- **Problems** have a function signature (`function_name`, `params`, `return_type`, `comparison_mode`), tags, optional company tags and **test cases**: the first 3 are public samples, the rest hidden. The student's code is judged by wrapping it with a *generated harness* that calls the function once per case and prints canonical JSON (`HarnessGenerator`, 19 languages).
- **Authoring** goes through `POST /admin/problems`: a structured signature + test cases (no hand-written per-language code); starter code is generated (`StarterCodeGenerator`). Bulk content comes from seeders (`DsaHundredProblemSeeder` V1–V10 + JSON batches).
- **Run vs Submit.** Run judges only the public samples; Submit judges everything. Hidden-case results reveal only pass/fail and stop at the first failure.
- **Pipeline (Figure 5).** The API never executes code: `JudgeQueue` caches identical Run results, de-duplicates in-flight double-clicks, caps 3 active jobs per user and rejects with **503 + Retry-After** when a queue is too deep; workers run `JudgeSubmissionJob`; `PistonPool` spreads calls over nodes with a circuit breaker; the verdict is stored (`submissions`) and cached 15 minutes.
- **Daily limits.** The governing plan's `max_practice_problems_per_day` (Coder = 1) is checked in `SubmissionController`; null means unlimited.

Backend map

Controllers	`ProblemController` · `SubmissionController` · `JudgeResultController` · `AdminProblemController` · `MySubmissionsController` · `StudentStatsController` · `LeaderboardController` · `PublicController`
Services	`JudgeService` · `CodeExecutionService` · `Judge/*` (queue, pool, store, persister, status) · `ProblemCodeGenerator/*` · `StudentStatsService`
Config	`config/judge.php` (queues, limits, backpressure) · `config/piston.php` (nodes, timeouts, language map) · `.env PISTON_NODES=url|slots,…`
Tables	`problems` · `problem_test_cases` · `submissions` (unique per user, problem, day)
Infra	`infra/judge/` (compose, runtimes install, supervisor workers) · `infra/loadtest/` · `docs/scaling-the-judge.md` (capacity maths, measured results)

Frontend map

Student pages	`/dashboard/practice` (`ProblemExplorer`) · `/dashboard/practice/view?slug=…` (`CodeEditorPanel` Monaco, `ProblemDescriptionPanel`, `ConsolePanel`, `LanguagePicker`) · `/dashboard/leaderboard` · `/dashboard/reports`
Libs	`lib/judge.ts` (`executeJudged`: POST then poll with back-off) · `lib/judgeLanguages.ts` · `data/languages.ts` · `lib/useMyStats.ts`
Ops UI	`/admin?view=mellow&tab=problems` → `ProblemBankPanel` · superadmin → Infrastructure tab (`InfrastructurePanel` ← `GET /superadmin/judge-nodes`)

Rules and gotchas

- `JudgeService::SUPPORTED_LANGUAGES` (19) must stay in step with `config/piston.php` language map **and** the runtimes actually installed on each node (`infra/judge/install-runtimes.sh`).
- Worker count should be ≈ total Piston slots + 10 %. After every deploy run `php artisan queue:restart` or old workers keep running old code.
- A node outage is retried (up to 240 s) and never stored as the student's runtime error — important because a contest wrong attempt costs a penalty.
- Locally the judge runs **inline** (`sync`) because `APP_ENV=local`; production *must* use a real queue connection or Run/Submit will block web workers.

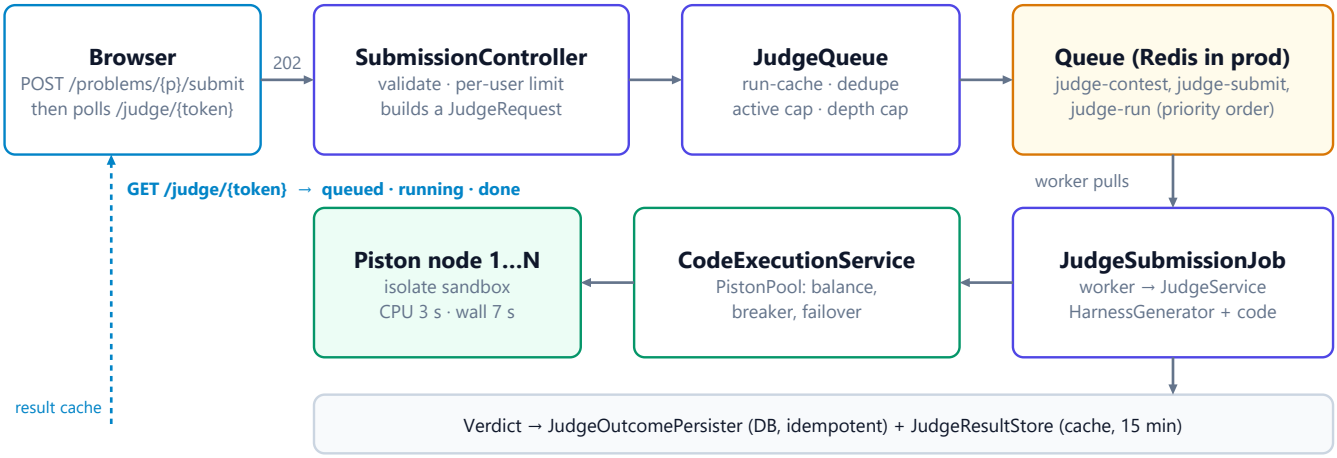


Figure 5 — Judge pipeline. Contest run/submit uses the same path on the highest-priority queue (judge-contest).

5.5 Contests

Timed, proctored coding events — platform-run, college-private, company hiring assessments, daily challenges and Talent Pool qualifiers.

Roles	`user` · `admin_internal` (`contests`) · `admin_tpo` · `admin_company`

How it works

- **Lifecycle.** `draft` → add problems (`contest_problems`, with `points`) → set audience → `published` → students **register** (`/contests/{c}/register`) → live between `start_at` and `end_at` → finalized.
- **Live participation.** `ContestSubmissionController` shows a problem, runs and submits through the same queued judge (queue `judge-contest`, so a live event is never starved). Scores are recomputed from the whole submission history (`ContestScoringService`), so a retried request cannot double count.
- **Finalize.** `contests:finalize` runs every 5 minutes and also `POST /admin|tpo|company/contests/{c}/finalize`; `ContestFinalizeService` freezes ranks, applies rating changes (`ContestRatingService`, only when `is_rated`), qualifies Talent Pool candidates and mails proctoring reports. It is idempotent via `finalized_at`.
- **Daily challenge.** `contests:generate-daily` creates one platform-wide contest every day at 00:05 IST that runs 19:00–20:00 (1 easy, 1 medium, 1 hard).
- **Ratings** live on `users.current_rating` / `rated_contests_count` (written only by `ContestFinalizeService` via `forceFill`); an unrated student is shown "Unrated", never 1200.

Backend map

Controllers	`ContestController` · `ContestSubmissionController` · `ContestProctoringController` · `AdminContestController` · `TpoContestController` · `CompanyContestController` · `AdminStudentReportController` …
Services	`ContestScoringService` · `ContestFinalizeService` · `ContestRatingService` · `ContestReportService` · `ProctoringService` · `TalentPoolQualificationService` · `JudgeQueue`
Command	`contests:finalize` (5 min) · `contests:generate-daily` (daily) · config `contests.php`
Tables	`contests` · `contest_problems` · `contest_participants` · `contest_submissions` (carries `judge_token`) · `contest_colleges` · `proctoring_sessions` · `proctoring_violations`

Frontend map

Student	`/dashboard/contests` · `/dashboard/contests/view?slug=…` (register, countdown, leaderboard) · `/dashboard/contests/problem?…` (editor + proctoring gate)
Ops / TPO / Company	`/admin/contests` (`DriveVisibilityPanel`) · `/admin/mock-contests` (TPO) · `/admin/company/assessments` (company)
Shared	`ContestParticipantsModal` (per-participant submissions + code), `CodeViewModal`, `lib/rating.ts`, `RatingBadge`, `RatingChart`

Rules and gotchas

- Contest routes bind by **slug**; the participant report route needs both `{contest}` and `{student}` — TPO/Admin/Company each have their own copy.
- Unique `daily_key` is the real idempotency guard for the daily contest; `onOneServer()` is only a courtesy.
- If the scheduler is not running, contests never finalize and ratings never move.

5.6 AI interviews & Final Interview tracks

Voice interviews a candidate takes in the browser, scored by Gemini and optionally reviewed by a human; three rounds can be chained into a pipeline.

Roles	`user` · `admin_internal` (`interviews`) · `admin_tpo` · `admin_company`

How it works

- **Taking one.** `POST /interviews/{slug}/start` → for each question the browser records audio (MediaRecorder) and a speech-to-text transcript and posts a multipart `answer` (`transcript_text` is mandatory, `audio` optional; the question id must equal the session's current expected question — replay/out-of-order is rejected) → `complete` → `result`.
- **Scoring.** When a session completes, `InterviewController` dispatches `ScoreInterviewSessionJob`, which makes **one batched Gemini call per session** (`GeminiInterviewScoringService`) so the candidate sees a real score moments later. A human reviewer (Ops / TPO / company, via `…/responses/{r}/score` and `…/sessions/{s}/finalize`) can add or override scores.
- **Tracks.** `InterviewTrack` is a container for three ordinary `Interview` rows (`interview_track_id`, `round_number`) created from an `InterviewRoleTemplate`'s `rounds_config` (snapshotted, never read live afterwards). When a reviewer finalizes a round, `InterviewTrackAdvancementService` computes the weighted composite and, if it clears the round's threshold, auto-invites the candidate to the next round.
- **Visibility** is per type (see 2.5); rounds 2 and 3 are always invite-only.
- **Daily limit.** `max_mock_interviews_per_day` of the governing plan is enforced in `InterviewController` (Coder 1, Expert 3, Grandmaster 10).
- **Question bank.** `interview_question_banks` is authored by Ops or generated by Gemini (`POST /interview-question-bank/generate`, 5/min per user, persisted straight into the bank); when no Gemini key is configured the endpoint reports "not configured" instead of failing.

Backend map

Controllers	`InterviewController` · `InterviewTrackController` · `InterviewProctoringController` · `Admin/Tpo/CompanyInterviewController` · `…InterviewTrackController` · `…InterviewRoleTemplateController` · `AdminInterviewQuestionBankController` · `InterviewQuestionBankController` · `InterviewQuestionGenerationController`
Services / jobs	`GeminiInterviewScoringService` · `GeminiQuestionGeneratorService` · `InterviewTrackAdvancementService` · `InterviewProctoringService` · `ScoreInterviewSessionJob` · `SendInterviewShortlistEmail`
Tables	`interviews` · `interview_questions` · `interview_question_banks` · `interview_sessions` · `interview_responses` · `interview_tracks` · `interview_role_templates` · `interview_colleges` · `interview_track_colleges` · `interview_proctoring_*`
Config	`GEMINI_API_KEY`, `GEMINI_MODEL` (`config/services.php`); rate limiter `ai-generation`

Frontend map

Student	`/dashboard/interviews` · `/view?slug=` · `/session?slug=` (recording, HUD, proctoring) · `/dashboard/interview-tracks`
Authoring / review	`/admin/interviews` (+ `/question-bank`, `/tracks`) · `/admin/mock-interviews` · `/admin/company/interviews`; components `components/dashboard/interviews/*` (`ReviewSessionsModal`, `ManageInterviewQuestionsModal`, `GenerateQuestionsPanel`, `CreateInterviewTrackModal`, `RoundConfigEditor`, `RoleTemplateModal`)
Libs	`lib/proctoring/useInterviewProctoring.ts`, `components/proctoring/InterviewSessionHud.tsx`, `lib/voice/*` (browser voice engine)

Rules and gotchas

- The candidate never scores themselves; Gemini output is stored on `interview_responses` / `interview_sessions` (`ai_*` columns), reviewer scores in separate columns — keep them separate.
- A round's outcome is decided in **one place** (`InterviewTrackAdvancementService`) called by all three finalize endpoints; never duplicate its logic in a controller.

5.7 Soft Skills assessments

Option-based aptitude / reasoning / English / workplace-judgment tests — the third assessment pillar beside contests and interviews.

Roles	`user` · `admin_internal` (`soft_skills`) · `admin_tpo` · `admin_company`

How it works

- A `SoftSkillAssessment` (type `general` / `tpo_mock` / `company`) picks questions from the shared bank `soft_skill_questions` (manual, Gemini-generated, or **auto-fill**).
- **Taking it.** `start` creates a session; `answer` autosaves one response by id in any order (like an exam); `submit` grades by exact `selected_index === correct_index` (`SoftSkillGradingService`) — never AI judgement. A proctored attempt can be ended by the server when violations accumulate (`SoftSkillProctoringService`).
- **Visibility** mirrors interviews: `SoftSkillAssessment::isVisibleToUser()`.

Backend map

Controllers	`SoftSkillController` · `SoftSkillProctoringController` · `Admin/Tpo/CompanySoftSkillController` (trait `ManagesSoftSkillQuestions`) · `AdminSoftSkillQuestionBankController`
Services	`SoftSkillGradingService` · `SoftSkillProctoringService` · `GeminiSoftSkillQuestionGeneratorService`
Tables	`soft_skill_questions` · `soft_skill_assessments` · `soft_skill_assessment_questions` · `soft_skill_sessions` · `soft_skill_responses` · `soft_skill_proctoring_*`

Frontend map

Student	`/dashboard/soft-skills` · `/view?…` · `/session?…`
Authoring	`SoftSkillAdminConsole` rendered by `/admin/soft-skills`, `/admin/mock-soft-skills`, `/admin/company/soft-skills` with different `allowedRoles`, `basePath`, `bankBasePath`; `CreateSoftSkillAssessmentModal`, `ManageSoftSkillQuestionsModal`

Rules and gotchas

- The bank routes `/…/soft-skill-question-bank/generate` must be declared before the `{softSkillQuestion}` wildcard (see 4.2).

5.8 Proctoring

Camera/microphone/tab-focus integrity for contests, interviews and soft-skills tests. The browser detects; the server decides.

Roles	`user` (subject) · `admin_tpo` · `section_coordinator` · `admin_company` (reviewers)

How it works

- **Client.** After a consent gate (`ProctoringConsentGate`), hooks in `src/lib/proctoring/` watch for tab switches, fullscreen exits and devtools and record video/audio **locally** — media is never uploaded for proctoring; only small JSON violation events are sent (`POST …/proctoring/violations`, 30/min per user).
- **Server.** `ProctoringService` / `InterviewProctoringService` / `SoftSkillProctoringService` keep consent and violation rows and decide the lock. Lock threshold `proctoring.max_violations_before_lock` = 3 (only violation types flagged as strikes count); a locked contest attempt **auto-submits** the current code (`auto_submit_on_lock`).

- **Review.** TPO (whole college), Coordinator (own section) and Company (own assessments) list flagged sessions and can **reinstate** a locked student (false-positive override). `SendProctoringViolationReport` emails the TPOs and the section coordinator.
- Practice is never proctored. Soft-skills proctoring can be switched off with `PROCTORING_SOFT_SKILLS=false`.

Backend map

Controllers	`ContestProctoringController` · `InterviewProctoringController` · `SoftSkillProctoringController` · `TpoProctoringController` · `CoordinatorProctoringController` · `CompanyProctoringController`
Config	`config/proctoring.php`: `PROCTORING_MAX_VIOLATIONS` (3), `PROCTORING_AUTO_SUBMIT_ON_LOCK` (true), `PROCTORING_SOFT_SKILLS` (true)
Tables	`proctoring_sessions/violations` (contest) · `interview_proctoring_*` · `soft_skill_proctoring_*`

Frontend map

Components	`components/proctoring/*`: `ProctoringConsentGate`, `ProctoringLockedScreen`, `ProctoringOverlay`, `InterviewProctoringConsentGate`, `InterviewProctoringLockedScreen`, `InterviewSessionHud`, `ProctoringSessionList` (prop `apiBasePath`), `proctoringCopy.ts`
Hooks	`lib/proctoring/useProctoring.ts`, `useInterviewProctoring.ts`, `recordingSink.ts`, `types.ts`
Review pages	`/admin/proctoring` (TPO) · `/coordinator/proctoring` · `/admin/company/proctoring`

Rules and gotchas

- The violation vocabulary lives in `ProctoringViolation`; the interview and soft-skills violation models deliberately **borrow** it — add new violation types there only.

5.9 Talent Pool marketplace

A consent-based marketplace where students who beat Mellow's scouting assessments can be discovered by hiring partners — all contact is mediated by the platform.

Roles	`admin_internal` (`talent_pool`, `contests`, `interviews`) · `user` · `admin_company`
Tables	talent_pool_candidates · talent_pool_inquiries · talent_pool_messages

How it works

- **Sourcing.** Ops creates a `talent_pool` contest (and optionally interview) with `qualifying_score_percent` and an `audience_scope` (`all` / `college` / `direct`). On finalize, `TalentPoolQualificationService` creates a `talent_pool_candidates` row in `pending_consent` and queues `SendTalentPoolQualifiedEmail`.
- **Consent.** The student opts in (`POST /me/talent-pool/opt-in`) → `visible`; or out → `hidden_by_candidate`. Ops can hide a profile (`hidden_by_mellow`).
- **Hiring partner actions** (`/company/talent-pool/*`): search (ranked by the student's recruiter-ready `profile_completion_percent`), view, download resume (authenticated), express interest, schedule an HR interview, hire, send a note. One `talent_pool_inquiries` row per (candidate, company) moves `interested → interview_scheduled → interview_completed → hired` (or declined/withdrawn).
- **No direct contact data** (email/phone) is ever returned to a company; emails go out through queued jobs from the platform.

Backend map

Controllers	`AdminTalentPoolController` · `CompanyTalentPoolController` · `StudentTalentPoolController` · `StudentProfileController`
Services / jobs	`TalentPoolQualificationService` · `SendTalentPool{Qualified,Interest,InterviewInvite,InterviewResponse,HireOffer,Outreach}Email`
Tables	`talent_pool_candidates` (`pending_consent/visible/hidden_by_candidate/hidden_by_mellow/hired`) · `talent_pool_inquiries` · `talent_pool_messages`

Frontend map

Pages	`/dashboard/talent-pool` (student) · `/admin/company/talent-pool` · `/admin/talent-pool` (Ops oversight)
Profile	`/settings` → `ProfileCompletionBar`, `SkillsTagInput`; score weights in `User::computeProfileCompletion()` (phone 15, avatar 10, bio 10, LinkedIn 15, GitHub 10, resume 25, ≥3 skills 15)

Rules and gotchas

- A resume lives on the **private** `local` disk and is only reachable through ownership-checked routes; `resume_path` is hidden from serialization.

5.10 Learning Centre — articles, speaking, listening, vocabulary

Self-study modules that sit beside placement prep. Three are AI-assisted; one is a markdown knowledge base.

Roles	`user` · `admin_internal` (`articles`)

How it works

- **Reading Hub (articles).** Ops writes markdown articles grouped in topics (`ArticleEditor`); students read them with next/previous navigation and automatic "read" tracking (`article_reads`).
- **Speaking.** Pick a read-aloud passage (shared library or AI-generated for the student) → record → `SpeakingPracticeController@submit`. `GeminiSpeakingScoringService` scores from the **audio itself** (the browser transcript is never trusted) plus a deterministic word alignment (`SpeechAlignment`). Pass mark 60; "no speech detected" is a free retry.
- **Listening.** Lessons are cut into sentences by one shared rule (`ListeningScript`) so server and client agree; the browser's text-to-speech reads one sentence at a time; grading is deterministic.
- **Vocabulary Sprint.** Six-box Leitner spaced repetition (box intervals 1, 3, 7, 16, 35 days), daily goal and streak, decks (constants in `VocabularyDecks`), an AI "quick quiz" on any topic. Answers are graded one at a time and the key is revealed only after the student commits.
- **AI quota.** Generation/scoring calls count against `max_learning_centre_ai_attempts_per_day` of the governing plan (`EnforcesLearningCentreAiLimit`) and the `ai-generation` limiter (5/min).

Backend map

Controllers	`ArticleController` · `AdminArticleController` · `AdminArticleTopicController` · `LearningCentreController` · `SpeakingPracticeController` · `ListeningLabController` · `VocabularyController` · `VocabularyProgressController`
Services	`GeminiSpeaking{Passage,Scoring}Service` · `GeminiListeningLessonService` · `ListeningProgressService` · `GeminiVocabularyQuizService` · `Vocabulary{Session,QuestionFactory,Answer,Progress,Srs}Service`
Support	`SpeechAlignment` · `ListeningScript` · `VocabularyDecks` · `VocabularyText` · `ActivityStreak`
Tables	`articles` · `article_topics` · `article_reads` · `speaking_prompts/attempts` · `listening_lessons/attempts` · `vocabulary_words/word_progress/attempts/answers`
Docs	`docs/speaking-practice.md` · `docs/vocabulary-sprint.md`

Frontend map

Pages	`/dashboard/learning-centre` (hub) · `/speaking` + `/speaking/session` · `/listening` + `/listening/session` · `/vocabulary` + `/session` + `/words` · `/dashboard/articles` (+ `/topic`, `/view`)
Components / libs	`components/listening/*`, `components/vocabulary/*`, `components/dashboard/articles/*`; `lib/listening/*` (player engine, voices), `lib/vocabulary/*`, `lib/voice/*`
Admin	`/admin/articles` (+ `/new`, `/edit`) → `ArticleEditor`, `MarkdownRenderer`

Rules and gotchas

- Speaking recordings are scored from memory and **not stored** (`speaking:purge-audio` removes legacy files).
- Vocabulary session questions are generated once with their answer key stored server-side; the browser only receives `questionsWithoutAnswers()`.

5.11 Project Lab & browser workspaces

SOW-guided, full-stack projects built inside a browser VS Code that runs in a per-student Docker container on separate hosts.

Roles	`user` (with the add-on) · `admin_internal` (`project_lab`)

How it works

- **Entitlement.** Anyone can browse the catalog; the SOW body, PDF, milestone detail and workspace need Project Lab access: the student's own **add-on** (`project-lab` plan, INR 499/month, 80 workspace-hours/month, 3 active projects) or a plan with `project_lab_access`. `User::projectLabCoverage()` resolves this; failures answer **402** with `code: project_lab_required`.
- **Flow.** start a project (`project_enrollments`, one per student+topic) → download the watermarked SOW PDF (`SowPdfService`, dompdf) → tick milestones → **start workspace** (`WorkspaceManager` picks a host, `DockerCliRuntime` runs `docker run` over ssh) → the page polls `show` until ready, then calls `heartbeat` every minute (idle detector **and** hours meter) → submit.
- **Reaper.** `workspaces:reap` (every minute) stops idle / lapsed / out-of-hours / over-cap workspaces and orphan containers.
- **Authorship guard.** The editor image carries `aptrun-guard.js` which refuses pasting text not copied inside the workspace and reports metadata-only events (`project_workspace_integrity_events`); `AuthorshipReport` + git history are frozen on the submission for the reviewer as a label with reasons — advisory, never blocking. `WORKSPACE_PASTE_GUARD` = `enforce` / `report` / `off`.
- **Submit.** `ProjectSubmissionService` archives the live container's project (`find | tar`, skipping `.env`, `node_modules` …) to a private disk; Ops reviews (`pending_review → in_review → reviewed | needs_changes`).

Backend map

Controllers	`ProjectLabController` · `ProjectWorkspaceController` · `AdminProjectLabController`
Services	`ProjectLab/{ProjectLabAccess,ProjectLabPresenter,ProjectSubmissionService,SowPdfService}` · `Workspaces/*` (WorkspaceManager, DockerCliRuntime, WorkspaceHosts, SnapshotScript, AuthorshipReport …)
Config	`config/workspaces.php` (hosts, limits, session policy) · `config/authorship.php` (guard mode, thresholds, retention) · `.env` `WORKSPACES_ENABLED`, `WORKSPACE_RUNTIME`, `WORKSPACE_HOSTS`, `WORKSPACE_IMAGE`
Tables	`project_topics` · `project_milestones` · `project_enrollments` · `project_milestone_progress` · `project_workspaces` · `project_workspace_sessions` · `project_workspace_integrity_events` · `project_submissions`
Infra / docs	`infra/workspaces/` (Dockerfile, Caddyfile, host bootstrap + firewall, guard, extensions) · `docs/project-lab.md` (architecture, runbook, security)

Frontend map

Student	`/dashboard/projects` · `/view?slug=` · `/workspace?…` (`WorkspaceView`, `MilestonePanel`, `SubmitDialog`, `Splitter`, `useAuthorshipGuard`, `PurchasePanel`)
Ops	`/admin/projects` (+ `/edit` → `TopicEditor`, `/review` → `AuthorshipCard`)
Libs	`lib/projectLab.ts`, `projectLabGuard.ts`, `projectLabLayout.ts`

Rules and gotchas

- Every Project Lab limiter is **named and per user** so polling cannot exhaust the Start button's allowance.
- The workspace domain and the app domain must share a registrable domain in production (the editor runs in an iframe and needs first-party cookies).
- `WORKSPACE_RUNTIME=fake` simulates hosts for UI work; the iframe is then empty.

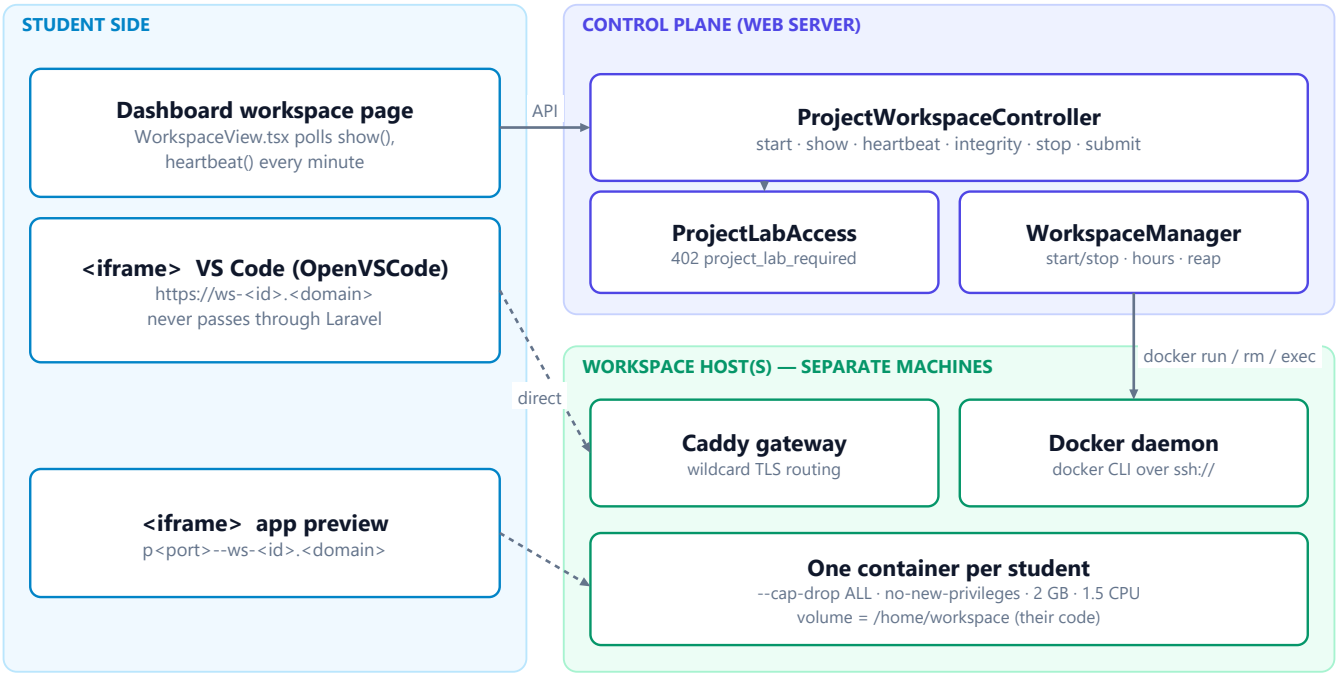


Figure 6 — Project Lab: Laravel is only the control plane; student code runs on separate Docker hosts.

5.12 Hack Arena

A separate full-screen world for hackathons: hosted by Ops or hiring partners, entered by students, judged by the host.

Roles	`user` (play) · `admin_internal` (`hackathons`) · `admin_company` (host) · `superadmin`; **TPOs cannot enter or host (403)**

How it works

- **Phases** (computed): `draft → upcoming → registration_open → starting_soon → live → judging → completed` (or `cancelled`). `status` (draft/published/cancelled) is stored; the phase is derived from that status and the hackathon's dates.
- **Registration** is race-safe: `HackathonRegistrationService` locks the hackathon row so two students cannot take the last seat. A solo entrant gets a team of one — there is no separate solo path.
- **Projects.** Any team member may edit the team's single submission (`draft` is invisible to judges; `submitted` is scored) until the deadline.
- **Judging.** Hosts mark each criterion 0–10; score = weighted average scaled to 0–100 (`HackathonResultsService`), ranks strictly ordered; students never see scores or disqualifications until the host **publishes results**. The public gallery opens only after the deadline.
- **Audience.** `hackathons.audience` = all students or specific colleges (`hackathon_colleges`); `Hackathon::scopeAvailableTo()` applies it.

Backend map

Controllers	`HackathonArenaController` (students) · `AdminHackathonController` / `CompanyHackathonController` (trait `ManagesHackathons`)
Services	`Hackathons/{HackathonRegistrationService,HackathonSubmissionService,HackathonResultsService,HackathonPresenter,HackathonRuleException}`
Tables	`hackathons` · `hackathon_colleges` · `hackathon_teams` · `hackathon_participants` · `hackathon_submissions` · `hackathon_announcements`
Limiters	`arena-action` 30/min · `arena-submit` 30/min · `arena-host` 120/min (all per user)
Docs	`docs/hack-arena.md` (rules, code map, demo data, local run)

Frontend map

Student	`/arena` (lobby) · `/arena/hackathon?slug=` · `/arena/my` — `components/arena/*`, styled by `app/arena/arena.css`
Host	`/admin/hackathons` and `/admin/company/hackathons` (+ `/edit`, `/manage`) — `components/hackathons/host/*`, `lib/hackathonHost.ts` (`hostApi(scope)`)

Rules and gotchas

- `HackathonRuleException` renders `{message, code}` JSON; the UI branches on `code`, so keep codes stable.
- Host routes bind by **id**, arena routes by **slug** — the two clients use different keys.

5.13 Plans, subscriptions & entitlements

What each account is allowed to do, how a college's plan covers its students, and how prices are decided. **No payment gateway is wired in yet.**

Roles	`user` (self-serve) · `admin_internal` / `superadmin` (institution + company plans) · every role reads `/me/subscription`

How it works

- **Plans** are rows in `plans` (`audience` individual / institution / company; `kind` base / add-on) with limit columns. Seeded by `PlanSeeder`: Coder (free), Expert, Grandmaster, the Project Lab add-on, Standard / Pro Campus / Academic Enterprise (institutions, with `max_students`) and Hiring Starter / Hiring Growth (companies).
- **Coverage precedence.** `User::effectiveSubscription()` → the student's **college plan first**, else their own. `effectiveEntitlements()` therefore governs a college student by their college's tier, not by anything they bought personally. Every new student automatically gets the free Coder plan (`User::created` hook).
- **Limits enforced in code:** `max_practice_problems_per_day` (SubmissionController), `max_mock_interviews_per_day` (InterviewController), `max_learning_centre_ai_attempts_per_day` (Learning Centre trait), `drive_access` (StudentDriveController), `project_lab_access` + hours/projects (ProjectLabAccess), `max_students` (College seat checks).
- **Pricing.** `PurchaseQuoteService` is the only place a price is decided; add-ons get `BILLING_ADDON_BUNDLE_DISCOUNT_PERCENT` (30 %) off when bought next to a paid base plan. `POST /me/subscription` re-quotes and returns **409** `price_changed` if the browser's `expected_total` is stale.
- **Expiry.** `subscriptions:expire` (daily) flips overdue rows; the countdown shown in the UI is computed live from dates.

Backend map

Controllers	`SubscriptionController` (`/me/subscription`, `/me/purchase-quote`) · `AdminController` (assign/adjust college plan) · `AdminCompanyController` (company plan)
Services	`SubscriptionService` (the one write path) · `PurchaseQuoteService`
Models / tables	`Plan` · `Subscription` (polymorphic subscriber: `User` or `College`; `is_trial`) · trait `HasSubscription` · `config/billing.php`
Seeded plans	Coder INR 0 · Expert INR 399/mo (3,999/yr) · Grandmaster INR 799/mo (7,999/yr) · Project Lab add-on INR 499/mo · Standard INR 149,000/yr (500 students) · Pro Campus INR 449,000/yr (2,000) · Academic Enterprise custom · Hiring Starter INR 99,000/yr · Hiring Growth INR 299,000/yr

Frontend map

Pages	`/pricing` (public + signed-in) · `/dashboard/billing` · `/settings` (Billing tab) · `PricingPlans`, `ProjectLabPricing`, `UsageLimitBanner`, `PurchasePanel`

Rules and gotchas

- **Production gap:** `SubscriptionService::purchase()` is documented in code as "the instant checkout stand-in — no money moves yet". A student calling `POST /me/subscription` is upgraded without paying. No Razorpay/Stripe-style integration exists in `backend/` or `frontend/`. Treat self-serve purchase as **not production-safe** until a gateway verifies payment before `purchase()` runs.
- Prices live in the `plans` table, not in code — changing a price means a data change in production (a migration or an `updateOrCreate` seeder run), not a deploy.

5.14 Marketing funnel, leads & customers

Two lead sources feed the Mellow marketing team; converted buyers move to Mellow Internal.

Roles	`admin_marketing` (`leads`, `lead_outreach`) · `admin_internal` (`customers`) · `superadmin`
Tables	users (lead columns) · lead_notes · contact_requests · contact_request_notes · newsletter_subscribers

How it works

- **Mellow Direct leads** = `users` with `role = user`, no college, not company-invited (`User::isMellowDirectLead()`). `User::created` assigns the **least-loaded** `admin_marketing` as `assigned_marketing_id`.
- **Lead status**: `new → contacted → engaged → converted | lost`. Marketing logs append-only notes (`lead_notes`) and can send outreach (`POST /marketing/leads/notify`, needs `lead_outreach`).
- **Conversion** happens automatically when the lead buys a plan: `SubscriptionService` → `LeadAssignmentService::convertLead()` sets `converted_at` and assigns an `admin_internal` owner (`assigned_internal_id`); the person then appears under Ops → Customers.
- **"Talk to Our Team"** (public form, rate-limited, no account created) → `contact_requests` (audience institution/company), load-balanced to marketing separately from leads, with confirmation and notification emails.
- **Newsletter** signup → `newsletter_subscribers` (public, tiny surface).

Backend map

Controllers	`MarketingLeadController` · `MarketingContactRequestController` · `InternalCustomerController` · `ContactRequestController` (public) · `NewsletterController` (public) · `SuperAdminController` (marketing performance)
Services	`LeadAssignmentService` · `MarketingLeadService` · `ContactRequestService`
Tables	`users` (lead columns) · `lead_notes` · `contact_requests` · `contact_request_notes` · `newsletter_subscribers`

Frontend map

Pages	`/marketing` (`LeadKanbanBoard`, `LeadPipelineBar`, `LeadDetailDrawer`) · `/marketing/inquiries` (`ContactRequestDetailDrawer`) · `/admin/customers` (`CustomerDetailDrawer`) · `/superadmin?tab=leads|marketing`
Public site	`TalkToTeamModal`, `/contact`, footer newsletter; marketing shell in `components/marketing-site/*`

Rules and gotchas

- Marketing and Ops cannot reach each other's routes; superadmin sees both.
- Everything in `users.lead_*` / `converted_at` / `assigned_*` is written with `forceFill` by the services — not mass-assignable.

5.15 Superadmin & platform operations

The trust tier: staff accounts, permissions, audit trail, infrastructure telemetry and feature flags.

Roles	`superadmin` only (`/superadmin/*`), plus every `/admin/*` route
Tables	activity_logs · feature_flags

How it works

- **Accounts.** `GET/POST /superadmin/users` (any role except a new superadmin), `…/{user}/detail`, `toggle-block`, `POST …/{user}/permissions` (replace a staff member's `permissions[]`).
- **Audit log.** `ActivityLog::record()` is called from ~52 places (onboarding, blocks, plan changes, hackathon moderation, …) and read at `GET /superadmin/audit-logs`.
- **Infrastructure.** `GET /superadmin/judge-nodes` shows live node health, queue depth and drain rate from `JudgeStatusService` (same data as `php artisan judge:status`).
- **Feature flags.** `feature_flags` rows can be toggled (`POST /superadmin/feature-flags/{flag}/toggle`) and are persisted, **but the model's own docblock says runtime gating is not implemented yet** — flipping a flag does not change behaviour by itself.

- **Overview.** `/superadmin/overview` returns a segmented platform overview (students, leads, colleges, staff).

Backend map

Controller	`SuperAdminController` (11 routes)
Services	`JudgeStatusService` · `LeadAssignmentService` · `StudentStatsService`
Tables	`activity_logs` · `feature_flags` · `users`

Frontend map

Page	`/superadmin?tab=overview|students|leads|marketing|colleges|staff|infrastructure|audit` → `components/admin/superadmin/*` (`OverviewPanel`, `StudentsPanel`, `LeadsPanel`, `MarketingPerformancePanel`, `MellowStaffPanel`, `InfrastructurePanel`, `AuditLogPanel`, `StudentDetailDrawer`); the Colleges tab reuses Ops' `PartnerCollegesPanel`

Rules and gotchas

- Minting a superadmin is deliberately impossible through the API — use `php artisan superadmin:sync`.

5.16 Reports, analytics & exports

Where the numbers and downloadable files come from — mostly the browser, fed by one-call data endpoints.

Roles	all dashboards

How it works

- **Readiness score** (`ReadinessScoreService`, 0–100): AI-scored interview performance 35 %, contest rating 20 %, practice depth & breadth 20 %, 7-day consistency 15 %, academic standing 10 %. Tiers: *Placement Ready*, *In Progress*, *Needs Training*; the weekly mail targets students under 60 %.
- **Data endpoints** return everything a page needs in one call: `GET /me/stats`, `GET /tpo/reports/data`, `GET /company/reports/data`, `GET /admin/overview`, `GET /superadmin/overview`. Every number is a real query — the code explicitly replaced earlier fabricated mock data.
- **Files generated in the browser** (exceljs + jspdf): `lib/generateTpoReports.ts`, `generateHiringReports.ts`, `generateCoordinatorReports.ts`, `generateStudentsExcel.ts`, `generateDriveApplicantsExcel.ts`, `generateReportPdf.ts` (+ `pdfTextSafe.ts`).
- **Files generated on the server**: weekly readiness Excel (phpspreadsheet, mailed), SOW PDF (dompdf), CSV import templates and the hackathon registrations export (downloaded as a blob through the authenticated client).

Backend map

Controllers	`StudentStatsController` · `TpoReportsController` · `CompanyReportsController` · `AdminController::overview` · `SuperAdminController::overview` · `*StudentReportController`
Services	`ReadinessScoreService` · `StudentStatsService` · `PlacementReportService` · `HiringReportService` · `StudentReportService` · `WeeklyReadinessReportService`

Frontend map

Pages	`/dashboard/reports`, `/admin/reports`, `/admin/analytics`, `/admin/company/reports`, `/coordinator/reports`; charts in `components/dashboard/student/*` and `components/dashboard/tpo/BarChart.tsx`

Rules and gotchas

- A download button that works for one role usually has a sibling generator for another (TPO / coordinator / company) — change them together.

PART VI

Database

Connection, conventions, the table families and how they relate, the status vocabularies, seeding, and the rules for changing the schema in production.

6.1 Connection and conventions

Topic	Fact	Where
Engine	MySQL (the code and its notes reference MySQL 8 behaviour; the server version was not inspected). Dev machine: `DB_CONNECTION=mysql`, host 127.0.0.1:3306, database `codegen_box` (the older product name). `.env.example` says `aptrun` — set your own per environment.	`.env`, `config/database.php`
Who connects	**Only the Laravel backend** opens a database connection (PDO via Eloquent, settings from `backend/.env` `DB_*`). The frontend never touches the database — it speaks HTTP to `/api`. Workers, the scheduler and artisan commands use the same connection settings.	`.env` → `config/database.php`
Charset	`utf8mb4` / `utf8mb4_unicode_ci`	`config/database.php`
Framework tables	`sessions`, `cache`, `cache_locks`, `jobs`, `job_batches`, `failed_jobs`, `password_reset_tokens`, `personal_access_tokens`, `migrations` — the **database** driver is the default for sessions, cache and queue (`SESSION_DRIVER`, `CACHE_STORE`, `QUEUE_CONNECTION`).	`config/{session,cache,queue}.php`
Production caching	Redis (predis) for the judge queue, result store and cache; the scheduler's `onOneServer()` needs a cache store with atomic locks.	`config/judge.php`, `infra/redis/`
Ids & keys	`BIGINT` auto-increment ids; slugs are separate unique columns on the 7 slug-bound models.	migrations
Mass assignment	Each model has a `$fillable` allow-list. Sensitive columns (`is_blocked`, `current_rating`, `phone_verified_at`, `resume_path`, `avatar_path`, lead/assignment columns, score columns) are **deliberately not fillable** and are written with `forceFill()` by one named service.	`app/Models/*`
JSON columns	Cast to arrays: `users.permissions`, `users.skills`, drive `eligible_branches`, many `meta` columns (`subscriptions.meta`).	model `casts()`
Money	Whole-rupee integers (`plans.monthly_price`, `annual_price`; `config/billing.php` currency `INR`).	`plans`
Audit	Append-only `activity_logs` (`actor_id`, action, target, meta) via `ActivityLog::record()`.	`ActivityLog`

6.2 How the tables fit together

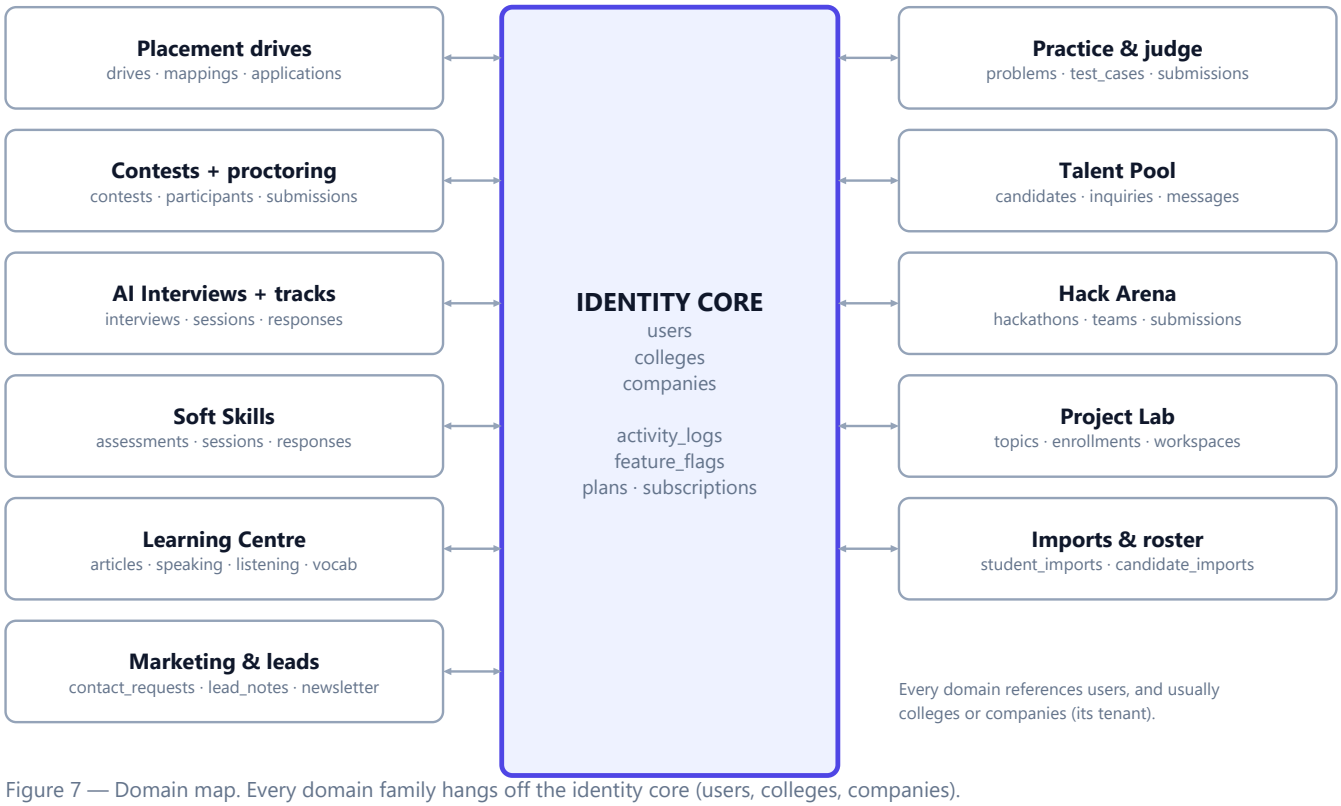


Figure 7 — Domain map. Every domain family hangs off the identity core (users, colleges, companies).

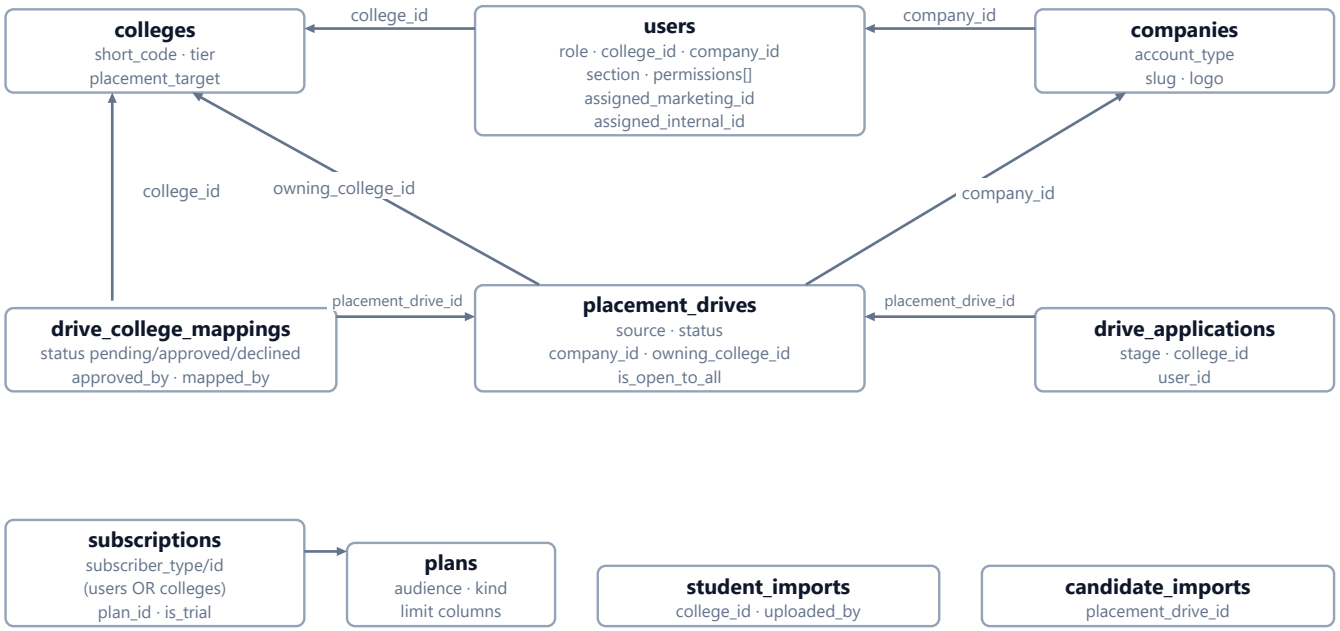


Figure 8 — Tenancy, placement and billing core. Arrows point from the table holding the foreign key to the table it references.

Notes on the core: `users` is intentionally wide (43 columns — identity, tenancy, academic profile, lead tracking, contest rating, recruiter profile) because every role shares it. `subscriptions` is **polymorphic** (`subscriber_type` + `subscriber_id` → a `User` or a `College`, via the `HasSubscription` trait), so it has no real foreign key to its owner. Every ownership column on contests/interviews/soft-skills/hackathons (`owning_college_id`, `owning_company_id`, `company_id`, `placement_drive_id`) is an ordinary FK.

6.3 Table inventory

85 tables in 13 families. “Model” is the Eloquent class (`App\Models`); “Refs” lists the tables it holds foreign keys to. The complete column list for every table is in **Appendix C**.

Framework & infrastructure (9)

Table	Cols	Model	Refs (foreign keys)
`cache`	3	—	—
`cache_locks`	3	—	—
`failed_jobs`	7	—	—
`job_batches`	10	—	—
`jobs`	7	—	—
`migrations`	3	—	—
`password_reset_tokens`	3	—	—
`personal_access_tokens`	10	—	—
`sessions`	6	—	—

Identity, tenancy & audit (8)

Table	Cols	Model	Refs (foreign keys)
`users`	43	User	colleges, companies, users
`colleges`	12	College	—
`companies`	13	Company	users
`activity_logs`	10	ActivityLog	users
`feature_flags`	9	FeatureFlag	users
`password_reset_otps`	9	PasswordResetOtp	users
`student_imports`	14	StudentImport	colleges, users
`candidate_imports`	14	CandidateImport	placement_drives, users

Placement drives (6)

Table	Cols	Model	Refs (foreign keys)
`placement_drives`	19	PlacementDrive	colleges, companies, users
`drive_college_mappings`	16	DriveCollegeMapping	colleges, placement_drives, users
`drive_applications`	11	DriveApplication	colleges, placement_drives, users
`company_prep_questions`	10	CompanyPrepQuestion	companies
`company_recommended_problems`	7	CompanyRecommendedProblem	companies
`company_recommended_interview_questions`	5	CompanyRecommendedInterviewQuestion	companies, interview_question_banks

Practice & judge (3)

Table	Cols	Model	Refs (foreign keys)
`problems`	19	Problem	—
`problem_test_cases`	9	ProblemTestCase	problems
`submissions`	11	Submission	problems, users

Contests & contest proctoring (7)

Table	Cols	Model	Refs (foreign keys)
`contests`	21	Contest	colleges, companies, placement_drives, users
`contest_problems`	7	ContestProblem	contests, problems
`contest_participants`	11	ContestParticipant	contests, users
`contest_submissions`	12	ContestSubmission	contest_problems, contests, users
`contest_colleges`	5	—	colleges, contests

Table	Cols	Model	Refs (foreign keys)
proctoring_sessions	11	ProctoringSession	contest_participants
proctoring_violations	10	ProctoringViolation	contest_problems, proctoring_sessions

AI Interviews & tracks (11)

Table	Cols	Model	Refs (foreign keys)
interviews	20	Interview	colleges, companies, interview_tracks, placement_drives, users
interview_colleges	5	—	colleges, interviews
interview_question_banks	11	InterviewQuestionBank	users
interview_questions	6	InterviewQuestion	interview_question_banks, interviews
interview_sessions	17	InterviewSession	interviews, users
interview_responses	14	InterviewResponse	interview_questions, interview_sessions, users
interview_role_templates	11	InterviewRoleTemplate	colleges, companies, users
interview_tracks	15	InterviewTrack	colleges, companies, interview_role_templates, placement_drives, users
interview_track_colleges	5	—	colleges, interview_tracks
interview_proctoring_sessions	10	InterviewProctoringSession	interview_sessions
interview_proctoring_violations	10	InterviewProctoringViolation	interview_proctoring_sessions, interview_questions

Soft Skills (7)

Table	Cols	Model	Refs (foreign keys)
soft_skill_questions	11	SoftSkillQuestion	users
soft_skill_assessments	14	SoftSkillAssessment	colleges, companies, users
soft_skill_assessment_questions	6	SoftSkillAssessmentQuestion	soft_skill_assessments, soft_skill_questions
soft_skill_sessions	12	SoftSkillSession	soft_skill_assessments, users
soft_skill_responses	8	SoftSkillResponse	soft_skill_assessment_questions, soft_skill_sessions
soft_skill_proctoring_sessions	10	SoftSkillProctoringSession	soft_skill_sessions
soft_skill_proctoring_violations	9	SoftSkillProctoringViolation	soft_skill_proctoring_sessions

Talent Pool (3)

Table	Cols	Model	Refs (foreign keys)
talent_pool_candidates	11	TalentPoolCandidate	companies, contests, users
talent_pool_inquiries	14	TalentPoolInquiry	companies, talent_pool_candidates, users
talent_pool_messages	5	TalentPoolMessage	talent_pool_inquiries, users

Learning Centre (11)

Table	Cols	Model	Refs (foreign keys)
articles	15	Article	article_topics, users
article_topics	10	ArticleTopic	users
article_reads	4	ArticleRead	articles, users
speaking_prompts	14	SpeakingPrompt	users
speaking_attempts	23	SpeakingAttempt	speaking_prompts, users
listening_lessons	16	ListeningLesson	users
listening_attempts	12	ListeningAttempt	listening_lessons, users
vocabulary_words	17	VocabularyWord	users
vocabulary_word_progress	13	VocabularyWordProgress	users, vocabulary_words
vocabulary_attempts	14	VocabularyAttempt	users
vocabulary_answers	12	VocabularyAnswer	users, vocabulary_attempts, vocabulary_words

Project Lab (8)

Table	Cols	Model	Refs (foreign keys)
project_topics	20	ProjectTopic	users
project_milestones	9	ProjectMilestone	project_topics
project_enrollments	9	ProjectEnrollment	project_topics, users
project_milestone_progress	6	ProjectMilestoneProgress	project_enrollments, project_milestones
project_workspaces	19	ProjectWorkspace	project_enrollments, users
project_workspace_sessions	10	ProjectWorkspaceSession	project_workspaces, users
project_workspace_integrity_events	11	ProjectWorkspaceIntegrityEvent	project_workspaces, users
project_submissions	23	ProjectSubmission	project_enrollments, users

Hack Arena (6)

Table	Cols	Model	Refs (foreign keys)
hackathons	39	Hackathon	companies, users
hackathon_colleges	5	—	colleges, hackathons
hackathon_teams	7	HackathonTeam	hackathons, users
hackathon_participants	8	HackathonParticipant	hackathon_teams, hackathons, users
hackathon_submissions	25	HackathonSubmission	hackathon_teams, hackathons, users
hackathon_announcements	7	HackathonAnnouncement	hackathons, users

Billing (2)

Table	Cols	Model	Refs (foreign keys)
plans	20	Plan	—
subscriptions	13	Subscription	plans

Marketing & leads (4)

Table	Cols	Model	Refs (foreign keys)
contact_requests	12	ContactRequest	users
contact_request_notes	6	ContactRequestNote	contact_requests, users
lead_notes	6	LeadNote	users
newsletter_subscribers	6	NewsletterSubscriber	—

6.4 Status and type vocabularies

These strings are the contract between PHP constants, DB columns and frontend `switch` statements. Change one, and grep all three.

Column	Allowed values	Defined / notes
`users.role`	user · admin_internal · admin_tpo · admin_marketing · superadmin · section_coordinator · admin_company	`User::ROLES`; MySQL ENUM widened by three migrations
`users.lead_status`	new → contacted → engaged → converted | lost	`User::LEAD_STATUSES`; only meaningful for Mellow Direct leads
`placement_drives.status` / `.source`	draft · published · completed · cancelled / catalog · tpo_created · company_direct	`PlacementDrive::STATUS_*`, `SOURCE_*`
`drive_college_mappings.status`	pending · approved · declined (+ `is_active`)	approval workflow; only approved + active counts
`drive_applications.stage`	registered · online_test · technical_interview · hr_round · offer_extended · offer_accepted · rejected · withdrawn	not a strict state machine
`contests.status` / `.contest_type` / `.audience_scope`	draft · published · cancelled / general · daily · company · tpo_mock · company_hiring · talent_pool / all · college · direct	`Contest::*`
`interviews.interview_type` (and tracks, soft-skills)	general · company · tpo_mock · company_hiring · talent_pool (track: no talent_pool; soft skills: general · tpo_mock · company)	same status triple draft/published/cancelled
`submissions.status`	accepted · wrong_answer · runtime_error · compile_error	`Submission::STATUS_*`; text verdicts like "Time Limit Exceeded" ride in the response body
`subscriptions.status`	active · expired · canceled (+ `is_trial`)	`Subscription::STATUS_*`
`plans.audience` / `.kind`	individual · institution · company / base · addon	`Plan::*`; one add-on code today: `project-lab`
`companies.account_type`	catalog_only · hiring_tenant	`Company::ACCOUNT_TYPE_*`
`talent_pool_candidates.visibility_status`	pending_consent · visible · hidden_by_candidate · hidden_by_mellow · hired	`TalentPoolCandidate::STATUS_*`
`talent_pool_inquiries.status`	interested · interview_scheduled · interview_completed · hired · declined_by_company · declined_by_candidate · withdrawn	one row per (candidate, company)
`project_enrollments.status`	active · submitted · reviewed · abandoned	one per (student, topic)
`project_submissions.status`	pending_review · in_review · reviewed · needs_changes	`attempt` increments per resubmission
`project_workspaces.status` / stop reason	starting · running · stopped · failed · destroyed / user · submitted · idle · max_session · lapsed · hours_exhausted · disk_full · host_error	`ProjectWorkspace::STATUS_*`, `STOP_*`
`hackathons.status` / phase	draft · published · cancelled → phases draft · upcoming · registration_open · starting_soon · live · judging · completed · cancelled	host_type platform|company; audience all|colleges; mode online|in_person|hybrid
`contact_requests.audience`	institution · company	`ContactRequest::AUDIENCE_*`

6.5 Seeding — what each seeder is, and what is safe in production

`php artisan db:seed` runs `DatabaseSeeder`, which (1) runs `PlanSeeder`, (2) **creates demo accounts for several roles with fixed passwords** — a superadmin, an Ops employee, a TPO, a student, a marketing employee, a company admin and two leads — and (3) calls the remaining content and demo seeders (28 `$this->call()` invocations in total). There is **no environment guard**. The passwords are in the file and in the repository history.

> **DANGER Never run a bare db:seed on a real database**
> It would create a superadmin and other privileged accounts with publicly known credentials. In production seed **individual** seeders (`php artisan db:seed --class=PlanSeeder`) and create the real superadmin with `php artisan superadmin:sync`. Some content seeders also look up the seeded Ops user for `created_by`, so read a seeder before running it alone.

Seeder	Kind	Notes
`PlanSeeder`	**Required**	Free Coder plan must exist before any student is created (the `User::created` hook attaches it). `updateOrCreate` by `code` — safe to re-run. Mirrors `frontend/src/data/pricing.ts` **by hand**.
`FeatureFlagSeeder`	Config rows	Starting rows of the superadmin flag registry

Seeder	Kind	Notes
`ProblemSeeder` + `DsaHundredProblemSeeder`, `…V2` – `…V10`	Catalog content	5 + 10 × 100 = ~1,005 DSA problems with test cases from `dsa_100_problems*.json`; `CompanyTagSeeder` adds company tags
`InterviewQuestionBankSeeder`	Catalog content	Starter bank across 5 categories ("realistic placeholder questions")
`SoftSkillQuestionSeeder`, `SoftSkillAssessmentSeeder`	Catalog content	75 hand-written questions + the default "Standard Placement Readiness Test" (20 aptitude, 15 reasoning, 15 English)
`SpeakingPromptSeeder`, `ListeningLessonSeeder`, `VocabularyWordSeeder`	Catalog content	Authored libraries (vocabulary: 8 decks × 20 words); idempotent
`ArticleTopicSeeder`, `ArticleSeeder`	Catalog content	Two launch topics/articles; need the seeded Ops user
`ProjectLabContentSeeder`	Catalog content	Loads `database/seeders/data/project-lab/<slug>.json` projects (a sample `hospital-management-system.json/.md` exists)
`PlacementSeeder`, `CampusRosterSeeder`, `CompanyHiringSeeder`, `InterviewSeeder`	**Demo data**	Placeholder companies/drives (Infosys …), a demo campus roster, the "Nimbus Labs" hiring tenant, demo interviews — do not load into production
`HackathonSeeder`	**Demo data**	Five demo hackathons in five different phases

6.6 Changing the schema safely

1. **Never edit a migration that has run anywhere.** Add a new, later-dated migration. The repo has 125; the newest are `2026_10_12_100000_create_hackathon_tables` and `2026_10_13_100000_add_audience_to_hackathons`.
2. **Prefer additive changes:** a new nullable column or a column with a default, a new table, a new index. Old code keeps working while new code rolls out; renames/drops need a two-step deploy.
3. **Backfill in the migration** when a new permission or column needs values for existing rows (see the `backfill_*_permission_for_full_access_staff` migrations).
4. **Test on MySQL, not only SQLite.** Real problems this project already hit: a `NOT NULL TIMESTAMP` column raises error 1067 (use `dateTime`); `rank` is a reserved word in MySQL 8 (quote it); an `orWhereHas` inside a `where(fn)` must be grouped or it escapes the tenant filter. The three `widen_users_role_enum_*` migrations rely on MySQL ENUM alteration.
5. **Run order in production:** deploy code that tolerates both schemas → `php artisan migrate --force` → `php artisan optimize` → `php artisan queue:restart`.
6. **Take a database backup first** — there is no `down()` safety net for data you transform. (Check each migration's `down()`; many drop what `up()` created but cannot restore data.)
7. Add or change an index deliberately: large tables are `submissions`, `contest_submissions`, `interview_responses`, `activity_logs`, `jobs`; the judge's per-day unique key on `submissions` is part of the daily-limit logic.

6.7 Recent migration history (themes)

The full list of 125 migrations (with dates) is in Appendix E. The shape of the history is useful context for where change is happening fastest:

Window	Theme
2026-09-11 → 09-13	Foundation: users + Sanctum tokens, colleges, roles/profile fields, companies, placement drives, mappings, audit log, student imports, problems
2026-09-15 → 09-19	Submissions, plans/subscriptions, contests (+ participants, submissions, ratings), drive applications, password OTP, leads/marketing role, judge nodes (later dropped as fabricated), granular permissions
2026-09-22 → 09-23	Section coordinators, forced password change, avatars, drive approval workflow, contest types/scope, proctoring, company tenant (role, columns, account type, candidate imports)
2026-09-24 → 09-25	AI interviews (bank, questions, sessions, responses, proctoring), articles, Talent Pool, phone verification, role templates + interview tracks
2026-09-26 → 09-30	Extended profile, AI scoring columns, plan entitlements, contact requests, newsletter, code in submissions, speaking, listening, vocabulary, soft skills
2026-10-01 → 10-09	Company tags on problems, trial subscriptions, soft-skills proctoring, speaking audio-first upgrade, listening formats/AI lessons, vocabulary SRS
2026-10-10 → 10-13	Project Lab (plans columns, tables, authorship evidence) and Hack Arena (tables, audience)

PART VII

Frontend reference

A static Next.js app: how it is built, how it routes, how it talks to the API, and where each kind of UI code lives.

7.1 The static-export model — and what it forces

`next.config.mjs` sets `output: "export"`, so `npm run build` produces plain files in `frontend/out/`. The comment in the config explains why: the app was targeted at shared hosting (BigRock) with no persistent Node process; it also deploys to Vercel unchanged. Consequences you will meet every day:

- **Every page is a `"use client"` component** that fetches its own data from the Laravel API at runtime. There is no server rendering, no API routes, no middleware, no cookies.
- **No `[param]` folders.** Dynamic ids travel in the query string: `/dashboard/practice/view?slug=two-sum`, `/dashboard/contests/view?slug=…`, `/admin/hackathons/manage?id=7`, `/arena/hackathon?slug=…`, `/admin/articles/edit?slug=…`. A page that reads `useSearchParams()` **must** sit inside `<Suspense>` or the export build fails (see `HostRoutes.tsx`).
- **Configuration is baked at build time.** `NEXT_PUBLIC_API_URL`, the Firebase keys and `NEXT_PUBLIC_SITE_URL` are compiled into the JS; changing the API host means *rebuilding and re-uploading*. If `NEXT_PUBLIC_API_URL` is unset the client falls back to `http://127.0.0.1:8000/api`.
- **Auth is client-side only.** A role guard is a `useEffect` redirect; the static HTML of a protected page is downloadable by anyone, which is fine because it contains no data — the API is the gate.
- **Every route is an HTML file.** A new page = a new folder with `page.tsx` under `src/app/`; it ships only after a rebuild.

Pages that take a query parameter	Parameter
`/dashboard/practice/view` · `/dashboard/contests/view` · `/dashboard/contests/problem` · `/dashboard/interviews/view` · `/dashboard/interviews/session` · `/dashboard/soft-skills/view|session` · `/dashboard/articles/view|topic` · `/dashboard/projects/view|workspace` · `/dashboard/learning-centre/*/session` · `/dashboard/drives` (detail)	`slug` or `id` (check the page)
`/admin/articles/edit` · `/admin/projects/edit|review` · `/admin/hackathons/edit|manage` · `/admin/company/hackathons/edit|manage` · `/admin/students/report`	`slug` / `id`
`/arena/hackathon` · `/marketing?status=…` · `/admin?view=…&tab=…` · `/superadmin?tab=…`	`slug`, `status`, `view`, `tab`

7.2 Source layout

Folder (`frontend/src/`)	Files	Contents
`app/`	123 files · 104 pages	App Router: one folder per URL, `page.tsx`. Also `layout.tsx` (fonts, `AuthProvider`, top loader), `robots.ts`, `manifest.ts`, social-card images, `globals.css` (design tokens).
`components/`	226	UI. By area: `admin/` (24), `arena/` (33), `dashboard/` (63: student, tpo, company, interviews, projectLab, softSkills, articles), `hackathons/host/`, `auth/`, `proctoring/`, `practice/`, `listening/`, `vocabulary/`, `pricing/`, `marketing-site/`, `layout/`, `navigation/`, `portal/` (premium kits), `ui/` (primitives), `sections/` + `hero/` (landing).
`lib/`	44	Non-visual logic: API client, auth, hooks, engines, report generators (see 7.6).
`types/`	23	TypeScript shapes of API payloads, **hand-maintained** (no code generation). Changing a response shape means editing `types/*.ts` too.
`data/`	9	Static content: marketing copy (`featurePages`, `audienceHubs`), `navigation.ts`, `pricing.ts`, `languages.ts`, plus legacy mock files (see 7.7).
`tests/`	31	`unit/` (Vitest, 17 files) and `e2e/` (Playwright: landing, footer pages, and `qa/` role walkthroughs 01–11).

7.3 Shell, navigation and the multi-view `/admin` page

- `DashboardShell` (`components/dashboard/`) wraps every signed-in page: sidebar + header + title/subtitle/action button. Props include `role` (which nav to show), `currentTpoView`, `title`, `subtitle`, `actionButton`.
- `DashboardSidebar.tsx` **(~1,290 lines) is the navigation map for all seven roles.** To add a menu item, add it to that role's section here **and** make sure the destination page guards the same roles. It also holds role badges ("Super Admin", "Mellow Staff", "College TPO", "Student Coder", "Hiring Partner" …).
- `/admin` **is one page serving three roles.** `src/app/admin/page.tsx` calls `useAuthGuard(["admin_internal","admin_tpo","superadmin","admin_company"])` and renders `OpsOverviewPanel`/tab panels (Ops), `TpoCommandCenter` (TPO) or `CompanyCommandCenter` (company) based on the **real role**. The `?view=` value in `homeRouteForRole` is cosmetic; nothing in the URL can switch an account into another role's view. Ops tabs use `?tab=overview|colleges|users|problems` and are filtered by permission.
- `/superadmin` is a tab shell (`?tab=`) in the same style; its Colleges tab renders the *same* `PartnerCollegesPanel` Ops uses, so features cannot drift.
- **Public site** uses `MarketingShell` + `Navbar`/`Footer`, driven by `data/navigation.ts` and `data/featurePages.tsx`; `/features/*`, `/colleges`, `/students`, `/recruiters`, `/pricing`, `/contact`, `/about`, `/privacy`, `/terms`.

Sidebar navigation per role

Role	Items (route)
`user`	Overview `/dashboard` · Practice Arena `/dashboard/practice` · Performance Report `/dashboard/reports` · Learning Centre `/dashboard/learning-centre` · Project Lab `/dashboard/projects` · Upcoming & Drives `/dashboard#placement-drives` · Contests · AI Interviews · Soft Skills · Talent Pool · Submissions `/dashboard#submissions` · Global Leaderboard · Profile Settings `/settings`
`admin_tpo`	TPO Command Center `/admin` · Campus Drives `/admin/drives` · Student Cohort `/admin/students` · Section Coordinators `/admin/coordinators` · Mock Contests `/admin/mock-contests` · Mock Interviews `/admin/mock-interviews` · Soft Skills `/admin/mock-soft-skills` · Readiness Analytics `/admin/analytics` · Placement Reports `/admin/reports` · Proctoring `/admin/proctoring`
`section_coordinator`	Section Roster `/coordinator` · Section Reports `/coordinator/reports` · Proctoring `/coordinator/proctoring`
`admin_company`	Hiring Command Center `/admin` · Partner Colleges · Job Openings · Candidates · Assessments · AI Interviews · Soft Skills · Hackathons · Talent Pool · Hiring Reports · Proctoring — all under `/admin/company/*`
`admin_internal`	Tabs of `/admin?view=mellow&tab=` (Overview, Partner Colleges & TPOs, Platform Users, Problem Bank) · Placement Drives `/admin/placements` · Contests · AI Interviews · Soft Skills · Project Lab `/admin/projects` · Hackathons · Articles · Talent Pool · My Customers `/admin/customers` — each item hidden unless the permission is held
`admin_marketing`	All Leads `/marketing` · New Leads `?status=new` · Converted `?status=converted` · Inquiries `/marketing/inquiries`
`superadmin`	`/superadmin?tab=` Overview & KPI · Partner Universities · Students · Leads · Marketing Performance · Mellow Staff · Audit & Security · Infrastructure & Flags

7.4 One console, three roles — the `basePath` pattern

Because the backend is mirrored per role (4.4), the frontend does **not** duplicate big screens; it renders one component and tells it which API prefix to use. The pages that do this are listed here with the prefix they pass — and therefore the controller family that will answer.

Page	Guard	API prefix(es) it passes
`/admin/company/hackathons`	admin_company	`/company/hackathons`
`/admin/company/hackathons/edit`	admin_company	`/company/hackathons`
`/admin/company/hackathons/manage`	admin_company	`/company/hackathons`
`/admin/company/interviews`	admin_company	`/company/interviews`
`/admin/company/proctoring`	admin_company	`/company/proctoring`
`/admin/company/soft-skills`	admin_company	`/company/soft-skills`, `/company/soft-skill-question-bank`
`/admin/contests`	admin_internal, superadmin	`/admin/contests`
`/admin/hackathons`	admin_internal, superadmin	`/admin/hackathons`
`/admin/hackathons/edit`	admin_internal, superadmin	`/admin/hackathons`
`/admin/hackathons/manage`	admin_internal, superadmin	`/admin/hackathons`
`/admin/interviews`	admin_internal, superadmin	`/admin/interviews`
`/admin/mock-contests`	admin_tpo	`/tpo/contests`

Page	Guard	API prefix(es) it passes
`/admin/mock-interviews`	admin_tpo	`/tpo/interviews`
`/admin/mock-soft-skills`	admin_tpo	`/tpo/soft-skills`, `/tpo/soft-skill-question-bank`
`/admin/proctoring`	admin_tpo	`/tpo/proctoring`
`/admin/soft-skills`	admin_internal, superadmin	`/admin/soft-skills`, `/admin/soft-skill-question-bank`
`/arena`	(any signed-in role)	`/arena/hackathons/{}`
`/arena/hackathon`	(any signed-in role)	`/arena/hackathons/{}`
`/arena/my`	user	`/arena/hackathons/{}`
`/coordinator/proctoring`	section_coordinator	`/coordinator/proctoring`

TIP How to find who consumes a component
Search for the component name under `src/app/` and `src/components/`, then read the props it receives: `basePath`, `bankBasePath`, `apiBasePath`, `scope="admin"|"company"`, `allowedRoles`. Those props — not the component — decide which controller runs. A fix inside the shared component affects every role that renders it.

7.5 Talking to the API

Concern	How it works
Calls	`api.get<T>(path)`, `api.post<T>(path, body)`, `api.put`, `api.delete`, `api.postFormData(path, formData)` (uploads: no JSON header so the browser sets the multipart boundary), `api.getFile(path)` (authenticated download → `Blob`).
Errors	Non-2xx → `ApiError(message, status, errors?, code?)`. Pages read `err.status` (401/403/402/409/422/429/503) and the optional machine `code` (`no_speech`, `project_lab_required`, `price_changed`, hackathon rule codes). 422 field errors are in `err.errors`.
De-duplication	Identical concurrent GETs share one promise (avoids double `/me` under React Strict Mode against a single-threaded dev server).
State	No Redux/Zustand/React-Query. Server data lives in component state loaded in `useEffect`; the only global store is `AuthContext`. Re-fetch after writes is explicit.
Long jobs	Judge: `executeJudged` posts, then polls `/judge/{token}` (400 ms → 2 s back-off, 120 s cap). Project Lab workspace: polls `show`, heartbeats every minute. Interviews: multipart answer per question.
Loading UX	`SessionLoader` while auth resolves; `nextjs-toploader` for route changes; per-page skeletons and toasts.

7.6 Domain libraries (`src/lib/`)

Group	Files
Core	`api.ts` (HTTP client) · `auth.ts` (types, token storage, `homeRouteForRole`, `userHasPermission`) · `AuthContext.tsx` · `useAuthGuard.ts` · `utils.ts` (`cn`, `greeting`) · `formatters.ts` · `datetime.ts` · `avatarColor.ts` · `useDebouncedValue.ts`
Data hooks	`useMyStats.ts` (`/me/stats`) · `useAdminOverviewCounts.ts` (sidebar badges) · `useLearningCentreOverview.ts` · `usePublicPlatformData.ts` (landing stats, no auth)
Judge & practice	`judge.ts` (`executeJudged`) · `judgeLanguages.ts` · `rating.ts` (contest tier colours/labels) · `activityGrid.ts` · `sectionBreakdown.ts`
Proctoring	`proctoring/useProctoring.ts` · `useInterviewProctoring.ts` · `recordingSink.ts` · `types.ts`
Learning Centre	`listening/` (`playerEngine`, `useListeningPlayer`, `voices`) · `vocabulary/` (`format`, `sessionState`, `speak`) · `voice/` (`VoiceEngine`, `browserVoiceEngine`)
Project Lab	`projectLab.ts` · `projectLabGuard.ts` · `projectLabLayout.ts` · `sowTemplate.ts`
Hack Arena	`hackathon.ts` (formatting, phases, student API) · `hackathonHost.ts` (`hostApi(scope)`, form ↔ payload, validation)
Reports	`generateTpoReports.ts` · `generateHiringReports.ts` · `generateCoordinatorReports.ts` · `generateStudentsExcel.ts` · `generateDriveApplicantsExcel.ts` · `generateReportPdf.ts` · `pdfTextSafe.ts`
Third party	`firebase.ts` (browser SDK init for phone OTP)

7.7 Static data, types and legacy files

- `data/pricing.ts` is the marketing copy of the plans and is **kept in sync by hand** with `PlanSeeder` / the `plans` table (the seeder says so). Changing a price means changing both.
- `data/mockDashboardData.ts` (imported only by `app/settings/page.tsx`) and `data/studentAnalytics.ts` (only by `ActivityHeatmap`) are remnants of the mock-data era — the backend comments repeatedly note the real endpoints replaced them. `data/languages.ts` has no non-test importer. Check the importer list before relying on, or deleting, any of them.
- `types/permissions.ts` is meant to mirror `User::PERMISSIONS_*` but **is out of sync today**: `INTERNAL_PERMISSIONS` lists 11 keys and omits `soft_skills` (the backend catalog has 12). `MellowStaffPanel` builds its checkboxes from that list, so the staff UI cannot grant — and a save from it may drop — the Soft Skills permission. Add `soft_skills` (and its label/description) there.
- Tests: `src/tests/unit/` covers auth session behaviour, formatters, judge languages, listening/vocabulary engines, hackathon helpers, Project Lab guard/layout; `src/tests/e2e/qa/` walks each role end to end.

7.8 Styling and design system

- **Tailwind 3.4** with colours defined as CSS variables in the `rgb(var(--x-rgb) / <alpha-value>)` form so opacity modifiers (`bg-surface/95`) work; dark mode is `class`-based (`ThemeToggle`). `tailwind.config.ts` must list `src/lib` and `src/data` in `content` because class strings are handed out from there (e.g. rating tier colours).
- **Fonts** are self-hosted (`app/fonts/*.woff2`: Inter and JetBrains Mono) so builds never need network access to Google.
- **Primitives** in `components/ui/` (`Button`, `Card`, `Modal`, `Badge`, `Toggle`, `SpotlightCard`, `SessionLoader`); **portal kits** in `components/portal/` (premium `Hp*` components used by TPO and hiring-partner pages); the Arena has its own stylesheet `app/arena/arena.css`.
- Animation: framer-motion and GSAP (landing); icons: lucide-react. Monaco is loaded for code editing; react-markdown + remark-gfm render articles and SOWs.

PART VIII

Infrastructure, configuration & deployment

What must be running in production, every environment variable by purpose (names only), how a release is packaged and shipped, and the verified problems to know about first.

8.1 Environment variables by purpose

Values are deliberately **not** reproduced here. Server-side variables live in `backend/.env` (template: `backend/.env.example`); the three frontend variables live in `frontend/.env.local` and are baked into the build.

Group	Variables	Notes
App core	`APP_NAME`, `APP_ENV`, `APP_KEY`, `APP_DEBUG`, `APP_URL`, `FRONTEND_URL`, `APP_LOCALE`, `BCRYPT_ROUNDS`, `LOG_CHANNEL`, `LOG_STACK`, `LOG_LEVEL`	Production: `APP_ENV=production`, `APP_DEBUG=false`, `LOG_LEVEL=warning`; `APP_URL` / `FRONTEND_URL` = real https URLs. **Changing `APP_KEY` makes encrypted queued jobs unreadable.**
Superadmin bootstrap	`SUPERADMIN_EMAIL`, `SUPERADMIN_PASSWORD`	Read only by `php artisan superadmin:sync`; rotate with `--update-password`.
Database	`DB_CONNECTION`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD` (`DB_URL`, `DB_SOCKET`)	MySQL. Dev DB name differs from `.env.example`.
Session / cache / queue	`SESSION_DRIVER`, `SESSION_LIFETIME`, `CACHE_STORE`, `QUEUE_CONNECTION`, `JUDGE_QUEUE_CONNECTION`	All `database` by default. `JUDGE_QUEUE_CONNECTION` defaults to `sync` only when `APP_ENV=local`.
Redis	`REDIS_CLIENT`, `REDIS_HOST`, `REDIS_PORT`, `REDIS_PASSWORD`, `REDIS_CACHE_HOST`, `REDIS_CACHE_PORT`	Two instances in `infra/redis` (queue: noeviction; cache: LRU).
Mail	`MAIL_MAILER`, `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_SCHEME`, `MAIL_FROM_ADDRESS`, `MAIL_FROM_NAME`	`log` mailer writes to `storage/logs`.
WhatsApp	`WHATSAPP_PROVIDER`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_ACCESS_TOKEN`	`log` provider by default (no real sends).
Gemini AI	`GEMINI_API_KEY`, `GEMINI_MODEL`, `GEMINI_SPEAKING_PRONUNCIATION_NOTES`	No key = generation endpoints report "not configured".
Firebase (server)	`FIREBASE_PROJECT_ID` (+ `FIREBASE_API_KEY`, `…_AUTH_DOMAIN`, `…_APP_ID` …)	Only the project id is used server-side to validate ID tokens.
Judge	`PISTON_NODES`, `PISTON_BASE_URL`, `PISTON_HTTP_TIMEOUT_SECONDS`, `PISTON_RUN_TIMEOUT_MS`, `PISTON_RUN_CPU_TIME_MS`, `PISTON_COMPILE_*`, `PISTON_BREAKER_*`, `JUDGE_*` (queues, caps, rate limits, `JUDGE_MAX_DEPTH_*`, TTLs, job retry)	`PISTON_NODES=http://host:2000|slots,…`. See `config/judge.php`, `config/piston.php`.
Proctoring / contests / billing	`PROCTORING_MAX_VIOLATIONS`, `PROCTORING_AUTO_SUBMIT_ON_LOCK`, `PROCTORING_SOFT_SKILLS` · `DAILY_CONTEST_*` · `BILLING_ADDON_BUNDLE_DISCOUNT_PERCENT`	Tunables with code defaults.
Project Lab	`WORKSPACES_ENABLED`, `WORKSPACE_RUNTIME`, `WORKSPACE_HOSTS` (JSON) `WORKSPACE_DOCKER_HOST`, `WORKSPACE_DOMAIN`, `WORKSPACE_SCHEME`, `WORKSPACE_IMAGE`, `WORKSPACE_NETWORK`, `WORKSPACE_MEMORY`, `WORKSPACE_CPUS`, `WORKSPACE_PIDS`, `WORKSPACE_IDLE_MINUTES`, `WORKSPACE_MAX_SESSION_MINUTES`, `WORKSPACE_SNAPSHOT_*`, `WORKSPACE_PASTE_GUARD`	Hosts, limits, session policy, authorship guard mode.
Unused-by-default	`AWS_*` · `MEMCACHED_HOST` · `BROADCAST_CONNECTION` · `VITE_APP_NAME` · `SANCTUM_STATEFUL_DOMAINS`	Laravel template leftovers; only matter if you switch to S3/SES/Memcached. Bearer-token auth ignores the stateful-domain list.
Frontend (build-time, `frontend/.env.local`)	`NEXT_PUBLIC_API_URL`, `NEXT_PUBLIC_SITE_URL`, `NEXT_PUBLIC_FIREBASE_*`	Compiled into the static files. Change = rebuild + re-upload.

8.2 What must be running in production

Process	How	If it is missing
Web (PHP)	Apache or PHP-FPM with document root `backend/public` (**never** the project folder); `storage/` and `bootstrap/cache/` writable; `php artisan storage:link` once	No API
Scheduler	cron every minute: `* * * * * cd /path/to/backend && php artisan schedule:run >> /dev/null 2>&1`	Contests never finalize, daily contest not created, subscriptions never expire, workspaces never reaped, weekly report not sent
Default queue worker	`php artisan queue:work` on the connection named by `QUEUE_CONNECTION`, under supervisor	No emails, no bulk imports, no AI interview scores, no WhatsApp
Judge workers	supervisor groups from `infra/judge/supervisor/judge-workers.conf`: `queue:work redis --queue=judge-contest,judge-submit,judge-run` (example 36 workers) plus an interactive group ordered `judge-run` first (16); `stopwaitsecs=90`; workers ≈ total Piston slots × 1.1	Run/Submit stay "queued"
Redis (2 instances)	`infra/redis/docker-compose.yml`: queue (`noeviction`, AOF) and cache (`allkeys-lru`); bound to 127.0.0.1 — keep private	Judge queue/cache unavailable; API answers 503
Piston nodes	`infra/judge/docker-compose.yml` + `install-runtimes.sh` once per node; slots = **physical** cores; port 2000 reachable only from workers	"Execution engine unavailable"
Workspace hosts	`infra/workspaces/` (Dockerfile, Caddyfile, `host-bootstrap.sh`, `host-firewall.sh`); never on the web server — they run student code	Project Lab workspaces fail to start
Static frontend	Contents of `frontend/out/` (or `frontend-out.zip`) at the web root with `index.html` on top	No UI

> **CAUTION After every backend deploy**
> Run `php artisan migrate --force && php artisan optimize && php artisan queue:restart`. Workers keep the old code in memory until they restart; `queue:restart` lets each finish its current job and exit so supervisor respawns it with the new code.

8.3 Packaging and shipping a release

`infra/deploy/make-package.ps1` is the supported way to produce deployable zips; it exists because zipping a working tree has repeatedly put the wrong things on a server (a dev `.env`, local caches, the scratch SQLite DB, test uploads, a frontend built against localhost).

1. On your PC: `./infra/deploy/make-package.ps1 -ApiUrl https://api.yourdomain.com/api` (add `-SkipFrontend` for backend only). Stop `next dev` first; both use `frontend/.next`. Run `composer install --no-dev --optimize-autoloader` in `backend/` beforehand.
2. The script copies only what the server needs, **verifies** the result (no `.env*`, no SQLite, no `storage/*` data, no bootstrap cache, `vendor` is `--no-dev`, none of the real secrets from `backend/.env` appear anywhere, no `localhost:8000` left in `out/`) and writes `dist/backend-deploy.zip`, `dist/frontend-out.zip` plus a SHA-256 for each.
3. Move the files by Google Drive / SFTP / cPanel — **not Gmail**, which rejects any archive containing `.js`, `.bat`, `.exe` …; this is expected, not malware. On the server `sha256sum` must equal the printed hash.
4. Backend: unzip so the web root is `backend/public`; create a **new** `.env` from `.env.example` (production values, new `APP_KEY` via `php artisan key:generate`); `php artisan storage:link && php artisan migrate --force && php artisan optimize`; permissions 775 on `storage/` and `bootstrap/cache/`.
5. Verify from outside that `/.env`, `/composer.json`, `/vendor/autoload.php`, `/storage/logs/laravel.log` and `/artisan` all return 403/404; if any returns content the web root is wrong — fix first.
6. Frontend: upload the contents of `frontend-out.zip` so `index.html` is at the top. The API URL is already baked in; to change it, rebuild with a new `-ApiUrl`.
7. Smoke test: `GET /up`, log in as each role you changed, run one practice submission, and `php artisan judge:status`.

If the server later misbehaves (unknown `*.php` files in the web root, unfamiliar cron jobs, CPU from a process you did not start, spam mail): change hosting/DB/SSH passwords, rotate every secret that was ever in an uploaded or emailed `.env` (`APP_KEY`, DB, Gemini, WhatsApp, AWS, mail, `SUPERADMIN_PASSWORD`), update PHP, and redeploy a fresh package rather than cleaning in place — this is the repo's own guidance.

8.4 Observability

- **Health:** `GET /up` (framework health route).
- **Judge:** `php artisan judge:status [--json]` (exits non-zero with no healthy node — usable as an uptime probe); superadmin → Infrastructure tab shows the same snapshot.
- **Queues:** `php artisan queue:failed` / `queue:retry`; failed jobs are in `failed_jobs`; mail jobs fail visibly there (and are rate-limited, not silently dropped).
- **Audit:** `activity_logs` (superadmin → Audit & Security) for sensitive admin actions. **Logs:** `storage/logs/laravel.log`; `php artisan pail` in dev.
- **Load testing:** `infra/loadtest/judge-load.mjs` + `judge:loadtest-seed`; measured results are in `docs/scaling-the-judge.md`.

8.5 Things to watch — verified observations

Each item below was checked directly against the repository (the evidence column says how). They are **not changed** by this guide — they are a to-do list for you, ordered by risk.

Risk	Observation	Evidence	Suggested action
High	Root `.env` is neither tracked nor ignored	`git status` lists `?? .env` at the repo root; `git check-ignore` matched only `backend/.env` and `frontend/.env.local`. It defines the same variable names as `backend/.env` — including `SUPERADMIN_PASSWORD`, `GEMINI_API_KEY`, `AWS_*`, `WHATSAPP_*`, `MAIL_*`, `FIREBASE_*` (values not inspected; treat as real secrets).	Delete it or add `/.env` to the root `.gitignore` before the next `git add .`. If it was ever committed or shared, rotate those secrets.
High	Self-serve plan purchase charges nothing	`SubscriptionService::purchase()` is documented as "the instant checkout stand-in — no money moves yet"; `POST /me/subscription` activates plans/add-ons directly. No payment-gateway code exists.	Do not expose paid self-serve in production until a gateway verifies payment before `purchase()`; otherwise hide the purchase UI.
High	`db:seed` creates privileged demo accounts	`DatabaseSeeder` makes a superadmin, Ops, TPO, student, marketing, company admin and two leads with fixed passwords; no environment guard.	Seed individual classes only (`--class=PlanSeeder`); create the real superadmin with `superadmin:sync`; never run bare `db:seed` against production.
High	Blocking a user does not end their session	`is_blocked` is checked only in `AuthController@login`; the toggles do not delete tokens; Sanctum `expiration` is `null`; no middleware re-checks the flag.	Delete the user's tokens when blocking (`$user->tokens()->delete()`) or add a check to the `api` middleware group.
Medium	CORS allow-list is hard-coded to localhost	`config/cors.php` `allowed_origins` = `localhost:3000`, `127.0.0.1:3000`; no env override.	Add the production frontend origin (ideally from an env var) before go-live.
Medium	`POST /register` has no throttle	Every other public write route has a named limiter; `/register` has none (route table shows only the `api` group). Each call creates a user, queues a welcome email and assigns a marketing owner.	Add `->middleware('throttle:…')` (e.g. per IP) to the route.
Medium	Default-queue worker is not in the repo's supervisor file	`infra/judge/supervisor/judge-workers.conf` starts only `judge-*` workers on **redis**. Emails, imports, AI scoring and WhatsApp jobs use the app's default connection/queue.	Run an additional `php artisan queue:work` for the default queue (matching `QUEUE_CONNECTION`) and include it in `queue:restart` after deploys.
Medium	Permission catalog drift (frontend)	`types/permissions.ts` has 11 internal keys; backend has 12 (`soft_skills` missing).	Add the key + label + description to `types/permissions.ts`.
Low	Feature flags are inert	`FeatureFlag` is read/written only by `SuperAdminController`; its docblock says runtime gating is not implemented.	Treat the Flags UI as a registry, not a kill switch.
Low	Two deployment stories	`backend/Dockerfile` is written for Render (PHP-Apache); `infra/deploy/README.md` describes cPanel/VPS with `public/` as web root; `next.config.mjs` mentions BigRock shared hosting.	Pick one target and delete or label the others to avoid confusion.
Low	Stale artefacts and placeholder README	`backend.zip` (repo root) and `frontend/out.zip` exist on disk (ignored by `*.zip`); root `README.md` is a three-line note.	Remove old zips (they may contain env copies); write real onboarding docs (this guide can seed them).

PART IX

Production change playbook

Where to go for a given change, step-by-step recipes for the common ones, a symptom-to-cause table, and the checklist to run before you ship.

9.1 “I need to change X” cheat sheet

I want to change…	Start here (backend)	…and here (frontend)
A column on the TPO student table	`TpoStudentController@index`, `StudentCohortService`; migration if new data	`src/app/admin/students/page.tsx`, `components/dashboard/tpo/*`, `types/cohort.ts` (+ coordinator copy: `CoordinatorStudentController`, `src/app/coordinator/page.tsx`)
Drive eligibility rules	`EligibilityService`; columns `min_cgpa`, `max_backlogs`, `eligible_branches` on `placement_drives`	`components/dashboard/EligibilityBadge.tsx`, `types/placement.ts`
Application pipeline stages	`DriveApplication::STAGE_*`, `DrivePipelineService`, `PlacementReportService`, `HiringReportService`	`components/dashboard/tpo/DrivePipelineBar.tsx`, company candidate pages, `types/placement.ts` / `hiring.ts`
Readiness score weights / tiers	`ReadinessScoreService`; at-risk threshold in `WeeklyReadinessReportService`	`components/dashboard/student/ReadinessBreakdown.tsx`, `ReadinessRing.tsx`, TPO cohort page
A plan's limits or price	`PlanSeeder` (`updateOrCreate` by `code`) → **run that seeder or a data migration**; enforcement points listed in 5.13	`src/data/pricing.ts` (hand-synced), `components/pricing/*`
Supported judge languages	`JudgeService::SUPPORTED_LANGUAGES`, `config/piston.php` map, `HarnessGenerator`, `infra/judge/install-runtimes.sh` (per node)	`lib/judgeLanguages.ts`, `data/languages.ts`, `components/practice/LanguagePicker.tsx`
Run/Submit limits, queue depth, timeouts	`config/judge.php`, `config/piston.php`; node ceilings in `infra/judge/docker-compose.yml`	—
Any rate limit	`AppServiceProvider::boot()` (named limiters); judge limits via `config/judge.php`	handle 429 in the calling page
Proctoring strictness	`config/proctoring.php`; violation types in `ProctoringViolation`	`lib/proctoring/*`, `components/proctoring/proctoringCopy.ts`
Contest scoring / penalty / rating	`ContestScoringService`, `ContestRatingService`, `ContestFinalizeService`	`lib/rating.ts`, `components/dashboard/student/RatingBadge.tsx`
Daily challenge time or size	`config/contests.php` (`DAILY_CONTEST_*`), schedule in `routes/console.php`	—
Interview scoring / thresholds	`GeminiInterviewScoringService`, `InterviewTrackAdvancementService`, role template `rounds_config`	`components/dashboard/interviews/RoundConfigEditor.tsx`
AI model or key	`.env` `GEMINI_MODEL`, `GEMINI_API_KEY` (`config/services.php`)	—
Email wording / look	`backend/resources/views/emails/*.blade.php`, mailable in `app/Mail/*`, job in `app/Jobs/*`	—
SOW PDF layout / watermark	`resources/views/project-lab/sow-pdf.blade.php`, `SowPdfService`	—
Where a role lands after login	—	`homeRouteForRole()` in `src/lib/auth.ts`
A sidebar item	—	`components/dashboard/DashboardSidebar.tsx` (that role's section)
Who can open a page	the `role:` / `permission:` group in `routes/api.php`	`useAuthGuard([...])` at the top of the page; `userHasPermission` for tabs
A staff permission	see recipe 9.2.2	`types/permissions.ts`, `MellowStaffPanel`
Student dashboard widgets	`StudentStatsController`, `StudentStatsService`	`src/app/dashboard/page.tsx`, `components/dashboard/student/*`, `lib/useMyStats.ts`
Public landing / feature pages	`PublicController` (stats, sample problems)	`components/sections/*`, `components/hero/*`, `data/featurePages.tsx`, `data/audienceHubs.tsx`
College seat caps	`College::studentLimit()`, `SubscriptionService::adjustSeatLimit`	`PartnerCollegesPanel`
Project Lab limits / images / idle time	`config/workspaces.php`, `PlanSeeder` `project-lab` row, `infra/workspaces/*`	`components/dashboard/projectLab/*`

I want to change…	Start here (backend)	…and here (frontend)
Hackathon rules	`app/Services/Hackathons/*`, trait `ManagesHackathons`	`lib/hackathon.ts`, `lib/hackathonHost.ts`, `components/arena/*`
The API address / allowed origins	`config/cors.php`	`NEXT_PUBLIC_API_URL` at **build** time
Password rules / token lifetime	`AuthController` validation; `config/sanctum.php` `expiration`	forms in `components/auth/*`, `app/settings/page.tsx`
Rotate the superadmin password	edit `.env` then `php artisan superadmin:sync --update-password`	—
Make a block take effect immediately	add `$user->tokens()->delete()` to the toggle methods (see 8.5)	—

9.2 Recipes

9.2.1 Add or change an API endpoint for an existing role

1. **Pick the guard.** Open `routes/api.php` and find the group for the role (`role:admin_tpo` → `prefix('tpo')`, etc.). Add the route inside it, **in the right declaration order** relative to wildcards.
2. **Write the controller method.** Validate with `$request->validate([...])`; derive scope from `$request->user()` (never from input); if you load a model by route binding, check it belongs to the caller; return `response()->json([...])`.
3. **Put reusable rules in a service**, and if sibling roles need the same behaviour make the service/trait the shared piece (4.4).
4. **Throttle** anything expensive or abusable with a named limiter in `AppServiceProvider`.
5. **Add a Feature test** under `backend/tests/Feature` (see `HackathonHostingTest`, `SoftSkillVisibilityTest` for the multi-role pattern).
6. **Frontend:** add the payload type to `src/types/*.ts`, call it with `api.get/post`, handle `ApiError` (`status`, `errors`, `code`), and update the page/component.

```
// routes/api.php  (inside the role:admin_tpo / prefix('tpo') group)
Route::get('/students/{student}/recent-submissions', [TpoStudentController::class, 'recentSubmissions']);

// TpoStudentController.php
public function recentSubmissions(Request $request, User $student)
{
    abort_unless($student->college_id === $request->user()->college_id, 404);   // tenant wall
    return response()->json(['submissions' => $student->submissions()->latest()->limit(20)->get()]);
}

// frontend
const res = await api.get<{ submissions: Submission[] }>(`/tpo/students/${id}/recent-submissions`);
```

9.2.2 Add a staff permission (new Ops section)

1. `User.php`: add `PERM_X = 'x'` and append it to `PERMISSIONS_INTERNAL`.
2. `routes/api.php`: wrap the new routes in `Route::middleware('permission:'.User::PERM_X)->group(...)` inside the `/admin` group.
3. **Migration:** add a `backfill_x_permission_for_full_access_staff` migration modelled on the existing five, or no existing employee will have the key.
4. Frontend: add the key, label and description to `src/types/permissions.ts` (and fix the missing `soft_skills`), add the sidebar item in `DashboardSidebar.tsx` guarded by `userHasPermission`, and create the page with `useAuthGuard(["admin_internal","superadmin"])`.
5. Test both a granted and an ungranted employee, and superadmin.

9.2.3 Add a page for a role

```
// frontend/src/app/admin/my-feature/page.tsx
"use client";
import { SessionLoader } from "@/components/ui/SessionLoader";
import { DashboardShell } from "@/components/dashboard/DashboardShell";
import { useAuthGuard } from "@/lib/useAuthGuard";
import { api } from "@/lib/api";

export default function MyFeaturePage() {
  const { status } = useAuthGuard(["admin_tpo"]);          // Layer 3 gate
  if (status !== "ready") return <SessionLoader />;
  return <DashboardShell role="admin_tpo" title="My feature" subtitle="…">{/* api.get("/tpo/...") */}</DashboardShell>;
}
```

- Add the sidebar entry for that role in `DashboardSidebar.tsx`. If the page needs an id, use a **query string** (`?id=`) and wrap `useSearchParams()` users in `<Suspense>`.
- The page is new HTML, so it ships only with the next **frontend build + upload**.

9.2.4 Add a database column safely

1. New migration (later date): nullable or with a default; no data loss; index if you will filter on it.
2. Model: add to `$fillable` **only if** a user may mass-assign it; otherwise set via `forceFill` in one service. Add a cast for JSON/dates/booleans.
3. Expose it in the controller payload(s) — remember mirrored controllers (4.4) — and in `src/types/*.ts`.
4. Deploy order: backend code that tolerates the old and new schema → `migrate --force` → frontend. Roll back by redeploying the previous code; leave the additive column in place.

9.2.5 Change something for all three authoring roles (Ops / TPO / Company)

Use the checklist in 4.4: change the shared service/trait first, then each `Admin*`, `Tpo*`, `Company*` controller, then verify each frontend consumer's `basePath`, then run the sibling tests. The route table (Appendix A) shows every sibling endpoint side by side; searching a path fragment such as `soft-skills/{softSkillAssessment}/questions` lists all three.

9.2.6 Add a scheduled task or a queued email

- **Scheduled:** create `app/Console/Commands/YourCommand.php` (`protected $signature`), register `Schedule::command('your:command')->daily()` in `routes/console.php`. It only runs where the cron entry exists.
- **Email:** create `app/Mail/YourMail.php` (+ Blade view in `resources/views/emails/`), a job `app/Jobs/SendYourMail.php` implementing `ShouldQueue` with the `RateLimited('transactional-emails')` middleware (copy an existing one), and `dispatch()` it from the service that owns the event. If the payload carries a secret, implement `ShouldBeEncrypted`.

9.2.7 Hotfix flow

1. Branch from `main`; reproduce with a failing test (backend `php artisan test --filter=…`, frontend `npm test`).
2. Fix, run `php artisan test` and `npm run type-check` (and the affected `qa-*` Playwright spec if it touches a role flow).
3. Package (`make-package.ps1`), compare the SHA-256 on the server, deploy backend then frontend, run `queue:restart`, smoke-test the roles you touched.
4. Rollback = redeploy the previous package. If a migration ran, only roll it back manually if it was destructive (prefer forward-fixing).

9.3 Symptom → likely cause → where to look

Symptom	Likely cause	Where to look
403 "You do not have permission to access this resource"	The user's role is not in the route group's `role:` list	Appendix A (guard column) for that URI; `routes/api.php`
403 "…Ask your superadmin to grant it"	Missing granular permission on an `admin_internal` / `admin_marketing` account	`/superadmin?tab=staff` → Manage Access; `users.permissions`
A tab/button shows but the call 403s	Layer 3 (frontend) out of sync with Layers 1–2	`useAuthGuard` list, `types/permissions.ts`, route group
401 on every request	Token revoked, deleted by a password reset, or the API URL points at the wrong environment	`personal_access_tokens`; `NEXT_PUBLIC_API_URL` in the build
Browser console: CORS error	Frontend origin not in the allow-list	`config/cors.php`
App talks to `localhost:8000` in production	Frontend was built without `-ApiUrl`	Rebuild via `make-package.ps1` (it refuses such a build)
402 `project_lab_required`	Student has no add-on / plan flag	`User::projectLabCoverage()`, `subscriptions`, `plans.project_lab_access`
409 `price_changed`	Plan price changed between quote and purchase	`PurchaseQuoteService`
422 on a form	Validation rule failed	Response body `errors` map; controller `validate()`
429	A named limiter tripped (per user or per IP)	4.8 table; `AppServiceProvider`; `config/judge.php`
503 + Retry-After on Run/Submit	Judge queue deeper than its limit	`php artisan judge:status`; add Piston nodes/workers or raise `JUDGE_MAX_DEPTH_*`
Run/Submit stuck on "queued"	No judge workers, or `JUDGE_QUEUE_CONNECTION` / `QUEUE_CONNECTION` mismatch	supervisor status; `queue:work` queue names; `.env`
"Execution engine unavailable"	No healthy Piston node (breaker open) or wrong `PISTON_NODES`	`judge:status`; node `docker ps`; port 2000 firewall
Emails never arrive	Default-queue worker down, `MAIL_MAILER=log`, SMTP rejected	`queue:failed`, `storage/logs`, `MAIL_*`
Contest never ends / ratings not updating	Scheduler cron not running	`crontab -l`; `php artisan contests:finalize` manually
A literal URL returns 404 (`/hackathons/colleges`)	Wildcard route declared before it	Declaration order in `routes/api.php` (4.2)
Student sees no drives	`drive_access = false` (Coder plan) or mapping not `approved` + `is_active`	`effectiveEntitlements()`, `drive_college_mappings`
Student cannot see a contest/interview	Type/audience rule	`Contest::isVisibleToCollege()` / `Interview::isVisibleToUser()` and 2.5
A TPO sees another college's data	**Missing tenant scope in a controller method**	Review the method against 2.6 immediately
Workspace stays "starting"	Host unreachable, Docker error, wrong `WORKSPACE_HOSTS`	`ProjectWorkspaceController@show` response; host `docker ps`; `config/workspaces.php`
Login returns `must_change_password`	First login with an admin-generated password	Use the OTP reset flow (2.7)

9.4 Pre-deploy checklist

Step	Check
Scope	Every new/changed route sits in the right `role:` / `permission:` group, and its controller scopes by the caller's `college_id` / `company_id` / `section` / `user_id`.
Mirrors	Admin / Tpo / Company siblings and shared frontend consoles updated together (4.4).
Schema	Migration is additive, tested on **MySQL**, backed up first; permission backfill included if you added a key.
Contracts	Response shape change is backward compatible for the still-deployed frontend; `src/types/*.ts` updated.
Config	New env vars added to the server `.env` (and `.env.example`); nothing secret in the package; `NEXT_PUBLIC_API_URL` correct.
Workers & cron	`queue:restart` planned; scheduler and default-queue worker confirmed running.
Tests	`php artisan test`, `npm test`, `npm run type-check` green; the `qa-*` spec for each touched role passes.
Smoke	`/up`, login per touched role, one judge run, `judge:status`, one email delivered.
Rollback	Previous packages (with SHA-256) kept; you know whether the migration is reversible.

9.5 Danger zones — where mistakes are expensive

Area	Why it is dangerous	Guard rail
`routes/api.php` groups	One misplaced route exposes an endpoint to the wrong role	Diff the route table (`route:list --json`) before and after
Tenant scoping in controllers	A missing `college_id` / `company_id` filter leaks another tenant's data	Review against 2.6; add a Feature test with two tenants
`SubscriptionService` / `PurchaseQuoteService` / `plans`	Entitlements and money; seed values mirrored in frontend copy	Change service, seeder and `pricing.ts` together; test coverage in `SubscriptionEntitlementTest`
Judge pipeline & `config/piston.php`	A wrong limit fails correct solutions or lets hostile code run longer; queue/worker mismatch stalls contests	Read `docs/scaling-the-judge.md` §3–§6 first; load-test with `infra/loadtest`
Contest finalize / rating	Irreversible score and rating changes	Idempotent via `finalized_at`; test on a copy
Proctoring lock logic	False locks cost students contest results (and auto-submit)	Keep reinstate working; change thresholds in config, not code
Bulk import / notify	Thousands of accounts/emails at once	Run against a small file first; watch `failed_jobs` and the 60/min mail limiter
Workspace hosts & guard	They execute student code; the host firewall and `--cap-drop` flags are the safety boundary	Do not loosen `DockerCliRuntime` flags without reading `docs/project-lab.md` §6
Auth middleware and `homeRouteForRole`	Lock-outs or privilege leaks	Test every role's login → landing → logout
Migrations on `users`	Wide hot table; MySQL ENUM role column	Off-peak, backup, additive

9.6 Command cheat sheet

```
# routing & config
php artisan route:list --path=api --json     # authoritative route table (what Appendix A was built from)
php artisan config:clear && php artisan optimize

# database
php artisan migrate:status                   # what is applied
php artisan migrate --force                  # production
php artisan db:seed --class=PlanSeeder       # NEVER a bare db:seed in production

# accounts
php artisan superadmin:sync [--update-password]

# queue / scheduler / judge
php artisan queue:restart                    # after every deploy
php artisan queue:failed ; php artisan queue:retry all
php artisan schedule:run                     # what cron runs every minute
php artisan judge:status [--json]
php artisan workspaces:reap ; php artisan contests:finalize ; php artisan subscriptions:expire

# tests
php artisan test --filter=SubscriptionEntitlementTest
npm test ; npm run type-check ; npm run test:e2e
```

9.7 A short path from 5% to 75%, and a self-check

Reading is not enough; the fastest way to own a codebase is to trace real requests. A suggested path (roughly three focused sessions):

Session	Do this	You will have learned
1 — Orient (2 h)	Read Parts I–III. Run the API and frontend locally. Run `php artisan route:list --path=api` and find five routes in Appendix A. Log in as a student, a TPO and Ops (demo accounts exist in your **dev** database) and watch the Network tab: note the `Authorization` header and the `/me` call.	The 7 roles, the guard stack, how the token flows
2 — Trace (3 h)	Pick **one** feature from Part V (suggest 5.2 drives). For one button, follow: page file (Appendix B) → `api.ts` call → route group (Appendix A) → controller → service → model → table (Appendix C). Write the chain down.	How to find any file in minutes
3 — Change (3 h)	Make a harmless change end to end on a branch: add a read-only field to one payload, show it on one page, add a Feature test, build a package with `make-package.ps1`, and dry-run the deploy on a staging server.	The mirror rule, the contract with the frontend, the release path

Self-check — if you can answer these without looking, you are at roughly 75%

1. A TPO opens `/admin/students`. Name the page file, the API prefix it calls, the controller, and the one value the controller must never take from the request.
2. Why does a college student on the free Coder plan still see drives? (Hint: which plan governs them, and where `drive_access` is read.)
3. You add a new "Ops" section for staff. List the five places that need changes so a new employee can open it.
4. A student presses Submit. Which queue does the job go on, what happens at 3 active jobs, and what does the browser poll?
5. Why can the same React console component appear under `/admin/soft-skills`, `/admin/mock-soft-skills` and `/admin/company/soft-skills`, and what changes between them?
6. A contest ended an hour ago and ratings have not moved. Name two causes and the command that proves each.
7. What happens to a user's open session when you block them today, and what would you change?
8. Which three things must be true for a `company` contest to be visible to a given college?
9. Where is a plan's price stored, and what else must be edited when it changes?
10. Why must the frontend be rebuilt to change the API URL, and what stops you shipping a build that points at `localhost`?
11. What does `must_change_password = true` do at login, and which endpoints finish the flow?
12. Why is a bare `php artisan db:seed` unsafe in production?

Answers: 1 → 2.6, 3.3, Appendix B · 2 → 5.13, 5.2 · 3 → 9.2.2 · 4 → Figure 5, 4.8 · 5 → 4.4, 7.4 · 6 → 4.7, 5.5 · 7 → 3.2, 8.5 · 8 → 2.5 · 9 → 5.13, 7.7 · 10 → 7.1, 8.3 · 11 → 2.7 · 12 → 6.5.

APPENDIX A

Complete API route reference

All 439 routes, generated from `php artisan route:list`. URIs omit the `/api` prefix. "Guard" shows only what differs from the area's default role(s), plus permissions and throttles. Role shorthand: user · tpo (admin_tpo) · coord (section_coordinator) · co (admin_company) · ops (admin_internal) · mkt (admin_marketing) · sa (superadmin).

Public — no token — 9 routes

Verb	URI	Handler	Guard
POST	/contact-requests	ContactRequest@store	throttle: contact-request
POST	/login	Auth@login	throttle: login
POST	/newsletter/subscribe	Newsletter@subscribe	throttle: newsletter-subscribe
POST	/password/forgot	PasswordReset@forgot	throttle: password-reset-request
POST	/password/otp/verify	PasswordReset@verifyOtp	throttle: password-reset-verify
POST	/password/reset	PasswordReset@reset	throttle: password-reset-verify
GET	/public/problems/sample	Public@sampleProblems	throttle: 60,1
GET	/public/stats	Public@stats	throttle: 60,1
POST	/register	Auth@register	—

Any signed-in user — /me/*, /logout — 16 routes

Verb	URI	Handler	Guard
POST	/logout	Auth@logout	—
GET	/me	Auth@me	—
DELETE	/me/avatar	Auth@deleteAvatar	—
POST	/me/avatar	Auth@updateAvatar	—
PUT	/me/password	Auth@updatePassword	—
POST	/me/phone/verify	PhoneVerification@verify	throttle: phone-verify
PUT	/me/profile	Auth@updateProfile	—
GET	/me/purchase-quote	Subscription@quote	—
DELETE	/me/resume	StudentProfile@deleteResume	—
GET	/me/resume	StudentProfile@downloadOwnResume	—
POST	/me/resume	StudentProfile@uploadResume	—
GET	/me/sessions	Auth@sessions	—
DELETE	/me/sessions/{tokenId}	Auth@revokeSession	—
PUT	/me/student-profile	StudentProfile@update	—
GET	/me/subscription	Subscription@mine	—
POST	/me/subscription	Subscription@subscribe	throttle: lab-purchase

Student routes at the API root (default role: user) — 82 routes

Verb	URI	Handler	Guard
GET	/articles/topics	Article@topics	—
GET	/articles/topics/{articleTopic}	Article@topicShow	—
GET	/articles/{article}	Article@show	—
POST	/articles/{article}/read	Article@markRead	—
GET	/contest-submissions/{contestSubmission}	MySubmissions@contestSubmission	—

Verb	URI	Handler	Guard
GET	/contests	Contest@index	—
GET	/contests/{contest}	Contest@show	—
GET	/contests/{contest}/leaderboard	Contest@leaderboard	—
GET	/contests/{contest}/problems/{contestProblem}	ContestSubmission@show	—
POST	/contests/{contest}/problems/{contestProblem}/proctoring/start	ContestProctoring@start	—
POST	/contests/{contest}/problems/{contestProblem}/proctoring/violations	ContestProctoring@reportViolation	throttle: proctoring-event
POST	/contests/{contest}/problems/{contestProblem}/run	ContestSubmission@run	throttle: judge-run
POST	/contests/{contest}/problems/{contestProblem}/submit	ContestSubmission@submit	throttle: judge-submit
DELETE	/contests/{contest}/register	Contest@unregister	—
POST	/contests/{contest}/register	Contest@register	—
GET	/drives	StudentDrive@index	—
GET	/drives/{placementDrive}	StudentDrive@show	—
GET	/interview-tracks	InterviewTrack@index	—
GET	/interview-tracks/{interviewTrack}	InterviewTrack@show	—
GET	/interviews	Interview@index	—
GET	/interviews/{interview}	Interview@show	—
POST	/interviews/{interview}/answer	Interview@answer	—
POST	/interviews/{interview}/complete	Interview@complete	—
POST	/interviews/{interview}/proctoring/start	InterviewProctoring@start	—
POST	/interviews/{interview}/proctoring/violations	InterviewProctoring@reportViolation	throttle: proctoring-event
GET	/interviews/{interview}/responses/{interviewResponse}/audio	Interview@responseAudio	—
GET	/interviews/{interview}/result	Interview@result	—
POST	/interviews/{interview}/start	Interview@start	—
GET	/judge/{token}	JudgeResult@show	throttle: judge-poll
GET	/leaderboard	Leaderboard@index	—
GET	/learning-centre/listening/lessons	ListeningLab@index	—
POST	/learning-centre/listening/lessons/generate	ListeningLab@generate	throttle: ai-generation
GET	/learning-centre/listening/lessons/{listeningLesson}	ListeningLab@show	—
POST	/learning-centre/listening/lessons/{listeningLesson}/attempts	ListeningLab@submit	—
GET	/learning-centre/overview	LearningCentre@overview	—
GET	/learning-centre/speaking/prompts	SpeakingPractice@index	—
POST	/learning-centre/speaking/prompts/generate	SpeakingPractice@generate	throttle: ai-generation
GET	/learning-centre/speaking/prompts/{speakingPrompt}	SpeakingPractice@show	—
POST	/learning-centre/speaking/prompts/{speakingPrompt}/attempts	SpeakingPractice@submit	throttle: ai-generation
GET	/learning-centre/vocabulary/attempts/{vocabularyAttempt}	Vocabulary@show	—
POST	/learning-centre/vocabulary/attempts/{vocabularyAttempt}/answer	Vocabulary@answer	throttle: 240,1
POST	/learning-centre/vocabulary/attempts/{vocabularyAttempt}/submit	Vocabulary@submit	—
POST	/learning-centre/vocabulary/generate	Vocabulary@generate	throttle: ai-generation
GET	/learning-centre/vocabulary/overview	VocabularyProgress@overview	—
POST	/learning-centre/vocabulary/sessions	Vocabulary@start	throttle: 60,1
GET	/learning-centre/vocabulary/words	VocabularyProgress@words	—
GET	/me/interviews	Interview@history	—

Verb	URI	Handler	Guard
GET	/me/rating-history	StudentStats@ratingHistory	—
GET	/me/soft-skills	SoftSkill@history	—
GET	/me/stats	StudentStats@mine	—
GET	/me/talent-pool	StudentTalentPool@mine	—
POST	/me/talent-pool/inquiries/{inquiry}/respond	StudentTalentPool@respondToInterview	—
POST	/me/talent-pool/opt-in	StudentTalentPool@optIn	—
POST	/me/talent-pool/opt-out	StudentTalentPool@optOut	—
GET	/problems	Problem@index	roles: user,ops,tpo,sa,co
GET	/problems/{problem}	Problem@show	—
POST	/problems/{problem}/run	Submission@run	throttle: judge-run
POST	/problems/{problem}/submit	Submission@submit	throttle: judge-submit
GET	/project-lab/enrollments/{enrollment}	ProjectLab@showEnrollment	—
POST	/project-lab/enrollments/{enrollment}/abandon	ProjectLab@abandon	—
POST	/project-lab/enrollments/{enrollment}/milestones/{milestone}/toggle	ProjectLab@toggleMilestone	throttle: lab-milestone
POST	/project-lab/enrollments/{enrollment}/submit	ProjectWorkspace@submit	throttle: lab-submit
GET	/project-lab/enrollments/{enrollment}/workspace	ProjectWorkspace@show	throttle: lab-workspace-poll
POST	/project-lab/enrollments/{enrollment}/workspace/heartbeat	ProjectWorkspace@heartbeat	throttle: lab-workspace-heartbeat
POST	/project-lab/enrollments/{enrollment}/workspace/integrity	ProjectWorkspace@integrity	throttle: lab-integrity
POST	/project-lab/enrollments/{enrollment}/workspace/start	ProjectWorkspace@start	throttle: lab-workspace-start
POST	/project-lab/enrollments/{enrollment}/workspace/stop	ProjectWorkspace@stop	throttle: lab-workspace-stop
GET	/project-lab/projects	ProjectLab@mine	—
GET	/project-lab/topics	ProjectLab@index	—
GET	/project-lab/topics/{projectTopic}	ProjectLab@show	—
GET	/project-lab/topics/{projectTopic}/sow.pdf	ProjectLab@downloadSow	throttle: lab-sow
POST	/project-lab/topics/{projectTopic}/start	ProjectLab@start	throttle: lab-topic-start
GET	/soft-skills	SoftSkill@index	—
GET	/soft-skills/sessions/{softSkillSession}	SoftSkill@viewSession	—
POST	/soft-skills/sessions/{softSkillSession}/answer	SoftSkill@answer	—
POST	/soft-skills/sessions/{softSkillSession}/proctoring/start	SoftSkillProctoring@start	—
POST	/soft-skills/sessions/{softSkillSession}/proctoring/violations	SoftSkillProctoring@reportViolation	throttle: proctoring-event
POST	/soft-skills/sessions/{softSkillSession}/submit	SoftSkill@submit	—
GET	/soft-skills/{softSkillAssessment}	SoftSkill@show	—
POST	/soft-skills/{softSkillAssessment}/start	SoftSkill@start	—
GET	/submissions	MySubmissions@index	—
GET	/submissions/{submission}	MySubmissions@show	—

Hack Arena — /arena/* (reads also open to Ops, company, superadmin) — 10 routes

Verb	URI	Handler	Guard
GET	/arena/hackathons	HackathonArena@index	roles: user,ops,co,sa
GET	/arena/hackathons/{hackathon}	HackathonArena@show	roles: user,ops,co,sa
GET	/arena/hackathons/{hackathon}/gallery	HackathonArena@gallery	roles: user,ops,co,sa
POST	/arena/hackathons/{hackathon}/register	HackathonArena@register	throttle: arena-action
GET	/arena/hackathons/{hackathon}/results	HackathonArena@results	roles: user,ops,co,sa

Verb	URI	Handler	Guard
PUT	/arena/hackathons/{hackathon}/submission	HackathonArena@saveSubmission	throttle: arena-submit
POST	/arena/hackathons/{hackathon}/team/members/{user}/remove	HackathonArena@removeMember	throttle: arena-action
POST	/arena/hackathons/{hackathon}/withdraw	HackathonArena@withdraw	throttle: arena-action
GET	/arena/home	HackathonArena@home	roles: user,ops,co,sa
GET	/arena/mine	HackathonArena@mine	—

College TPO — /tpo/* (role: admin_tpo) — 71 routes

Verb	URI	Handler	Guard
GET	/tpo/companies/search	TpoDrive@searchCompanies	—
GET	/tpo/contests	TpoContest@index	—
POST	/tpo/contests	TpoContest@store	—
POST	/tpo/contests/{contest}	TpoContest@update	—
POST	/tpo/contests/{contest}/finalize	TpoContest@finalize	—
GET	/tpo/contests/{contest}/participants	TpoContest@participants	—
GET	/tpo/contests/{contest}/participants/{student}/submissions	TpoContest@participantSubmissions	—
GET	/tpo/contests/{contest}/problems	TpoContest@problems	—
POST	/tpo/contests/{contest}/problems	TpoContest@storeProblem	—
DELETE	/tpo/contests/{contest}/problems/{contestProblem}	TpoContest@destroyProblem	—
GET	/tpo/coordinators	TpoCoordinator@index	—
POST	/tpo/coordinators	TpoCoordinator@store	—
PUT	/tpo/coordinators/{user}	TpoCoordinator@update	—
POST	/tpo/coordinators/{user}/toggle-block	TpoCoordinator@toggleBlock	—
POST	/tpo/drives	TpoDrive@store	—
GET	/tpo/drives/available	TpoDrive@available	—
GET	/tpo/drives/mapped	TpoDrive@mapped	—
GET	/tpo/drives/pending	TpoDrive@pending	—
GET	/tpo/drives/{placementDrive}/applications	TpoDriveApplication@index	—
POST	/tpo/drives/{placementDrive}/applications	TpoDriveApplication@store	—
POST	/tpo/drives/{placementDrive}/applications/bulk-register	TpoDriveApplication@bulkRegister	—
POST	/tpo/drives/{placementDrive}/applications/{application}/stage	TpoDriveApplication@updateStage	—
POST	/tpo/drives/{placementDrive}/map	TpoDrive@map	—
POST	/tpo/drives/{placementDrive}/mapping	TpoDrive@updateMapping	—
POST	/tpo/drives/{placementDrive}/respond	TpoDrive@respond	—
POST	/tpo/drives/{placementDrive}/unmap	TpoDrive@unmap	—
GET	/tpo/interview-role-templates	TpoInterviewRoleTemplate@index	—
POST	/tpo/interview-role-templates	TpoInterviewRoleTemplate@store	—
POST	/tpo/interview-role-templates/{interviewRoleTemplate}	TpoInterviewRoleTemplate@update	—
GET	/tpo/interview-tracks	TpoInterviewTrack@index	—
POST	/tpo/interview-tracks	TpoInterviewTrack@store	—
DELETE	/tpo/interview-tracks/{interviewTrack}	TpoInterviewTrack@destroy	—
POST	/tpo/interview-tracks/{interviewTrack}	TpoInterviewTrack@update	—
GET	/tpo/interviews	TpoInterview@index	—
POST	/tpo/interviews	TpoInterview@store	—
DELETE	/tpo/interviews/{interview}	TpoInterview@destroy	—
POST	/tpo/interviews/{interview}	TpoInterview@update	—

Verb	URI	Handler	Guard
GET	/tpo/interviews/{interview}/questions	TpoInterview@questions	—
POST	/tpo/interviews/{interview}/questions	TpoInterview@storeQuestion	—
DELETE	/tpo/interviews/{interview}/questions/{interviewQuestion}	TpoInterview@destroyQuestion	—
GET	/tpo/interviews/{interview}/responses/{interviewResponse}/audio	TpoInterview@responseAudio	—
POST	/tpo/interviews/{interview}/responses/{interviewResponse}/score	TpoInterview@scoreResponse	—
GET	/tpo/interviews/{interview}/sessions	TpoInterview@sessions	—
POST	/tpo/interviews/{interview}/sessions/{interviewSession}/finalize	TpoInterview@finalizeSession	—
POST	/tpo/interviews/{interview}/sessions/{interviewSession}/reinstate	TpoInterview@reinstateProctoring	—
GET	/tpo/interviews/{interview}/sessions/{interviewSession}/responses	TpoInterview@responses	—
POST	/tpo/placement-target	TpoReports@updateTarget	—
GET	/tpo/proctoring	TpoProctoring@index	—
GET	/tpo/proctoring/{proctoringSession}	TpoProctoring@show	—
POST	/tpo/proctoring/{proctoringSession}/reinstate	TpoProctoring@reinstate	—
GET	/tpo/reports/data	TpoReports@data	—
GET	/tpo/soft-skill-question-bank	AdminSoftSkillQuestionBank@index	—
POST	/tpo/soft-skill-question-bank/generate	AdminSoftSkillQuestionBank@generate	throttle: ai-generation
GET	/tpo/soft-skills	TpoSoftSkill@index	—
POST	/tpo/soft-skills	TpoSoftSkill@store	—
DELETE	/tpo/soft-skills/{softSkillAssessment}	TpoSoftSkill@destroy	—
POST	/tpo/soft-skills/{softSkillAssessment}	TpoSoftSkill@update	—
GET	/tpo/soft-skills/{softSkillAssessment}/questions	TpoSoftSkill@questions	—
POST	/tpo/soft-skills/{softSkillAssessment}/questions	TpoSoftSkill@storeQuestion	—
POST	/tpo/soft-skills/{softSkillAssessment}/questions/auto-fill	TpoSoftSkill@autoFillQuestions	—
DELETE	/tpo/soft-skills/{softSkillAssessment}/questions/{assessmentQuestion}	TpoSoftSkill@destroyQuestion	—
POST	/tpo/students	TpoStudent@store	—
POST	/tpo/students/bulk-notify	TpoStudent@bulkNotify	—
GET	/tpo/students/cohort	TpoStudent@index	—
POST	/tpo/students/import	StudentImport@storeMine	—
GET	/tpo/students/import-template	StudentImport@template	—
GET	/tpo/students/imports	StudentImport@indexMine	—
GET	/tpo/students/imports/{studentImport}	StudentImport@showMine	—
GET	/tpo/students/{student}/contest-submissions/{contestSubmission}	TpoStudentReport@contestSubmission	—
GET	/tpo/students/{student}/report	TpoStudentReport@show	—
GET	/tpo/students/{student}/submissions/{submission}	TpoStudentReport@submission	—

Section Coordinator — /coordinator/* — 9 routes

Verb	URI	Handler	Guard
GET	/coordinator/proctoring	CoordinatorProctoring@index	—
GET	/coordinator/proctoring/{proctoringSession}	CoordinatorProctoring@show	—
POST	/coordinator/proctoring/{proctoringSession}/reinstate	CoordinatorProctoring@reinstate	—
GET	/coordinator/students	CoordinatorStudent@index	—

Verb	URI	Handler	Guard
POST	/coordinator/students/bulk-notify	CoordinatorStudent@bulkNotify	—
GET	/coordinator/students/{student}/contest-submissions/{contestSubmission}	CoordinatorStudentReport@contestSubmission	—
GET	/coordinator/students/{student}/report	CoordinatorStudentReport@show	—
GET	/coordinator/students/{student}/submissions/{submission}	CoordinatorStudentReport@submission	—
POST	/coordinator/students/{user}/toggle-block	CoordinatorStudent@toggleBlock	—

Hiring partner — /company/* (role: admin_company) — 88 routes

Verb	URI	Handler	Guard
GET	/company/candidates/import-template	CompanyCandidate@template	—
GET	/company/colleges	CompanyCollege@index	—
GET	/company/contests	CompanyContest@index	—
POST	/company/contests	CompanyContest@store	—
POST	/company/contests/{contest}	CompanyContest@update	—
POST	/company/contests/{contest}/finalize	CompanyContest@finalize	—
POST	/company/contests/{contest}/invite	CompanyContest@inviteCandidates	—
GET	/company/contests/{contest}/invited	CompanyContest@invited	—
GET	/company/contests/{contest}/problems	CompanyContest@problems	—
POST	/company/contests/{contest}/problems	CompanyContest@storeProblem	—
DELETE	/company/contests/{contest}/problems/{contestProblem}	CompanyContest@destroyProblem	—
GET	/company/contests/{contest}/results	CompanyContest@results	—
POST	/company/contests/{contest}/results/import	CompanyContest@importResults	—
GET	/company/drives	CompanyDrive@index	—
POST	/company/drives	CompanyDrive@store	—
GET	/company/drives/{placementDrive}	CompanyDrive@show	—
POST	/company/drives/{placementDrive}	CompanyDrive@update	—
GET	/company/drives/{placementDrive}/candidates	CompanyCandidate@index	—
POST	/company/drives/{placementDrive}/candidates	CompanyCandidate@store	—
POST	/company/drives/{placementDrive}/candidates/import	CompanyCandidate@import	—
GET	/company/drives/{placementDrive}/candidates/imports	CompanyCandidate@indexImports	—
GET	/company/drives/{placementDrive}/candidates/imports/{candidateImport}	CompanyCandidate@showImport	—
POST	/company/drives/{placementDrive}/candidates/{application}/stage	CompanyCandidate@updateStage	—
GET	/company/drives/{placementDrive}/mappings	CompanyDrive@mappings	—
POST	/company/drives/{placementDrive}/propose	CompanyDrive@proposeToColleges	—
GET	/company/hackathons	CompanyHackathon@index	throttle: arena-host
POST	/company/hackathons	CompanyHackathon@store	throttle: arena-host
GET	/company/hackathons/colleges	CompanyHackathon@colleges	throttle: arena-host
DELETE	/company/hackathons/{hackathon}	CompanyHackathon@destroy	throttle: arena-host
GET	/company/hackathons/{hackathon}	CompanyHackathon@show	throttle: arena-host
PUT	/company/hackathons/{hackathon}	CompanyHackathon@update	throttle: arena-host
POST	/company/hackathons/{hackathon}/announcements	CompanyHackathon@storeAnnouncement	throttle: arena-host
DELETE	/company/hackathons/{hackathon}/announcements/{announcement}	CompanyHackathon@destroyAnnouncement	throttle: arena-host
POST	/company/hackathons/{hackathon}/cancel	CompanyHackathon@cancel	throttle: arena-host
DELETE	/company/hackathons/{hackathon}/cover	CompanyHackathon@deleteCover	throttle: arena-host
POST	/company/hackathons/{hackathon}/cover	CompanyHackathon@uploadCover	throttle: arena-host

Verb	URI	Handler	Guard
POST	/company/hackathons/{hackathon}/publish	CompanyHackathon@publish	throttle: arena-host
POST	/company/hackathons/{hackathon}/publish-results	CompanyHackathon@publishResults	throttle: arena-host
GET	/company/hackathons/{hackathon}/registrations	CompanyHackathon@registrations	throttle: arena-host
GET	/company/hackathons/{hackathon}/registrations/export	CompanyHackathon@exportRegistrations	throttle: arena-host
GET	/company/hackathons/{hackathon}/submissions	CompanyHackathon@submissions	throttle: arena-host
POST	/company/hackathons/{hackathon}/submissions/{submission}/disqualify	CompanyHackathon@disqualify	throttle: arena-host
POST	/company/hackathons/{hackathon}/submissions/{submission}/reinstate	CompanyHackathon@reinstate	throttle: arena-host
POST	/company/hackathons/{hackathon}/submissions/{submission}/score	CompanyHackathon@scoreSubmission	throttle: arena-host
POST	/company/hackathons/{hackathon}/unpublish	CompanyHackathon@unpublish	throttle: arena-host
GET	/company/interview-role-templates	CompanyInterviewRoleTemplate@index	—
POST	/company/interview-role-templates	CompanyInterviewRoleTemplate@store	—
POST	/company/interview-role-templates/{interviewRoleTemplate}	CompanyInterviewRoleTemplate@update	—
GET	/company/interview-tracks	CompanyInterviewTrack@index	—
POST	/company/interview-tracks	CompanyInterviewTrack@store	—
DELETE	/company/interview-tracks/{interviewTrack}	CompanyInterviewTrack@destroy	—
POST	/company/interview-tracks/{interviewTrack}	CompanyInterviewTrack@update	—
GET	/company/interviews	CompanyInterview@index	—
POST	/company/interviews	CompanyInterview@store	—
DELETE	/company/interviews/{interview}	CompanyInterview@destroy	—
POST	/company/interviews/{interview}	CompanyInterview@update	—
POST	/company/interviews/{interview}/invite	CompanyInterview@inviteCandidates	—
GET	/company/interviews/{interview}/invited	CompanyInterview@invited	—
GET	/company/interviews/{interview}/questions	CompanyInterview@questions	—
POST	/company/interviews/{interview}/questions	CompanyInterview@storeQuestion	—
DELETE	/company/interviews/{interview}/questions/{interviewQuestion}	CompanyInterview@destroyQuestion	—
GET	/company/interviews/{interview}/responses/{interviewResponse}/audio	CompanyInterview@responseAudio	—
POST	/company/interviews/{interview}/responses/{interviewResponse}/score	CompanyInterview@scoreResponse	—
POST	/company/interviews/{interview}/sessions/{interviewSession}/finalize	CompanyInterview@finalizeSession	—
POST	/company/interviews/{interview}/sessions/{interviewSession}/reinstate	CompanyInterview@reinstateProctoring	—
GET	/company/interviews/{interview}/sessions/{interviewSession}/responses	CompanyInterview@responses	—
GET	/company/proctoring	CompanyProctoring@index	—
GET	/company/proctoring/{proctoringSession}	CompanyProctoring@show	—
POST	/company/proctoring/{proctoringSession}/reinstate	CompanyProctoring@reinstate	—
GET	/company/reports/data	CompanyReports@data	—
GET	/company/soft-skill-question-bank	AdminSoftSkillQuestionBank@index	—
POST	/company/soft-skill-question-bank/generate	AdminSoftSkillQuestionBank@generate	throttle: ai-generation
GET	/company/soft-skills	CompanySoftSkill@index	—
POST	/company/soft-skills	CompanySoftSkill@store	—
DELETE	/company/soft-skills/{softSkillAssessment}	CompanySoftSkill@destroy	—
POST	/company/soft-skills/{softSkillAssessment}	CompanySoftSkill@update	—
GET	/company/soft-skills/{softSkillAssessment}/questions	CompanySoftSkill@questions	—

Verb	URI	Handler	Guard
POST	/company/soft-skills/{softSkillAssessment}/questions	CompanySoftSkill@storeQuestion	—
POST	/company/soft-skills/{softSkillAssessment}/questions/auto-fill	CompanySoftSkill@autoFillQuestions	—
DELETE	/company/soft-skills/{softSkillAssessment}/questions/{assessmentQuestion}	CompanySoftSkill@destroyQuestion	—
GET	/company/talent-pool	CompanyTalentPool@index	—
GET	/company/talent-pool/inquiries	CompanyTalentPool@myInquiries	—
GET	/company/talent-pool/{candidate}	CompanyTalentPool@show	—
POST	/company/talent-pool/{candidate}/hire	CompanyTalentPool@hire	—
POST	/company/talent-pool/{candidate}/interest	CompanyTalentPool@expressInterest	—
POST	/company/talent-pool/{candidate}/interview	CompanyTalentPool@scheduleInterview	—
POST	/company/talent-pool/{candidate}/notify	CompanyTalentPool@notify	—
GET	/company/talent-pool/{candidate}/resume	CompanyTalentPool@resume	—

Mellow Ops — /admin/* (roles: admin_internal, superadmin; most groups also need a permission) — 132 routes

Verb	URI	Handler	Guard
GET	/admin/article-topics	AdminArticleTopic@index	perm: articles
POST	/admin/article-topics	AdminArticleTopic@store	perm: articles
DELETE	/admin/article-topics/{articleTopic}	AdminArticleTopic@destroy	perm: articles
POST	/admin/article-topics/{articleTopic}	AdminArticleTopic@update	perm: articles
GET	/admin/articles	AdminArticle@index	perm: articles
POST	/admin/articles	AdminArticle@store	perm: articles
DELETE	/admin/articles/{article}	AdminArticle@destroy	perm: articles
GET	/admin/articles/{article}	AdminArticle@show	perm: articles
POST	/admin/articles/{article}	AdminArticle@update	perm: articles
GET	/admin/colleges	Admin@colleges	perm: colleges
POST	/admin/colleges	Admin@storeCollege	perm: colleges
GET	/admin/colleges/{college}/coordinators	Admin@collegeCoordinators	perm: colleges
POST	/admin/colleges/{college}/coordinators	Admin@storeCoordinatorForCollege	perm: colleges
PUT	/admin/colleges/{college}/seats	Admin@adjustCollegeSeats	perm: colleges
POST	/admin/colleges/{college}/students/import	StudentImport@storeForCollege	perm: colleges
GET	/admin/colleges/{college}/students/imports	StudentImport@indexForCollege	perm: colleges
GET	/admin/colleges/{college}/students/imports/{studentImport}	StudentImport@showForCollege	perm: colleges
POST	/admin/colleges/{college}/subscription	Admin@assignCollegeSubscription	perm: colleges
GET	/admin/colleges/{college}/tpos	Admin@collegeTpos	perm: colleges
GET	/admin/companies	AdminCompany@index	perm: placements
POST	/admin/companies	AdminCompany@store	perm: placements
POST	/admin/companies/admins/{user}/toggle-block	AdminCompany@toggleAdminBlock	perm: placements
POST	/admin/companies/{company}	AdminCompany@update	perm: placements
GET	/admin/companies/{company}/prep-questions	AdminCompany@prepQuestions	perm: placements
POST	/admin/companies/{company}/prep-questions	AdminCompany@storePrepQuestion	perm: placements
GET	/admin/companies/{company}/recommended-problems	AdminCompany@recommendedProblems	perm: placements
POST	/admin/companies/{company}/recommended-problems	AdminCompany@storeRecommendedProblem	perm: placements
POST	/admin/companies/{company}/subscription	AdminCompany@assignSubscription	perm: placements
GET	/admin/contests	AdminContest@index	perm: contests
POST	/admin/contests	AdminContest@store	perm: contests

Verb	URI	Handler	Guard
POST	/admin/contests/{contest}	AdminContest@update	perm: contests
POST	/admin/contests/{contest}/colleges	AdminContest@updateColleges	perm: contests
POST	/admin/contests/{contest}/finalize	AdminContest@finalize	perm: contests
GET	/admin/contests/{contest}/participants	AdminContest@participants	perm: contests
GET	/admin/contests/{contest}/participants/{student}/submissions	AdminContest@participantSubmissions	perm: contests
GET	/admin/contests/{contest}/problems	AdminContest@problems	perm: contests
POST	/admin/contests/{contest}/problems	AdminContest@storeProblem	perm: contests
DELETE	/admin/contests/{contest}/problems/{contestProblem}	AdminContest@destroyProblem	perm: contests
PUT	/admin/coordinators/{user}	Admin@updateCoordinatorSection	perm: colleges
POST	/admin/coordinators/{user}/toggle-block	Admin@toggleCoordinatorBlock	perm: colleges
GET	/admin/customers	InternalCustomer@index	perm: customers
GET	/admin/customers/{customer}	InternalCustomer@show	perm: customers
POST	/admin/customers/{customer}/notes	InternalCustomer@storeNote	perm: customers
GET	/admin/hackathons	AdminHackathon@index	perm: hackathons; throttle: arena-host
POST	/admin/hackathons	AdminHackathon@store	perm: hackathons; throttle: arena-host
GET	/admin/hackathons/colleges	AdminHackathon@colleges	perm: hackathons; throttle: arena-host
DELETE	/admin/hackathons/{hackathon}	AdminHackathon@destroy	perm: hackathons; throttle: arena-host
GET	/admin/hackathons/{hackathon}	AdminHackathon@show	perm: hackathons; throttle: arena-host
PUT	/admin/hackathons/{hackathon}	AdminHackathon@update	perm: hackathons; throttle: arena-host
POST	/admin/hackathons/{hackathon}/announcements	AdminHackathon@storeAnnouncement	perm: hackathons; throttle: arena-host
DELETE	/admin/hackathons/{hackathon}/announcements/{announcement}	AdminHackathon@destroyAnnouncement	perm: hackathons; throttle: arena-host
POST	/admin/hackathons/{hackathon}/cancel	AdminHackathon@cancel	perm: hackathons; throttle: arena-host
DELETE	/admin/hackathons/{hackathon}/cover	AdminHackathon@deleteCover	perm: hackathons; throttle: arena-host
POST	/admin/hackathons/{hackathon}/cover	AdminHackathon@uploadCover	perm: hackathons; throttle: arena-host
POST	/admin/hackathons/{hackathon}/publish	AdminHackathon@publish	perm: hackathons; throttle: arena-host
POST	/admin/hackathons/{hackathon}/publish-results	AdminHackathon@publishResults	perm: hackathons; throttle: arena-host
GET	/admin/hackathons/{hackathon}/registrations	AdminHackathon@registrations	perm: hackathons; throttle: arena-host
GET	/admin/hackathons/{hackathon}/registrations/export	AdminHackathon@exportRegistrations	perm: hackathons; throttle: arena-host
GET	/admin/hackathons/{hackathon}/submissions	AdminHackathon@submissions	perm: hackathons; throttle: arena-host
POST	/admin/hackathons/{hackathon}/submissions/{submission}/disqualify	AdminHackathon@disqualify	perm: hackathons; throttle: arena-host
POST	/admin/hackathons/{hackathon}/submissions/{submission}/reinstate	AdminHackathon@reinstate	perm: hackathons; throttle: arena-host
POST	/admin/hackathons/{hackathon}/submissions/{submission}/score	AdminHackathon@scoreSubmission	perm: hackathons; throttle: arena-host
POST	/admin/hackathons/{hackathon}/unpublish	AdminHackathon@unpublish	perm: hackathons; throttle: arena-host
GET	/admin/interview-question-bank	AdminInterviewQuestionBank@index	perm: interviews
POST	/admin/interview-question-bank	AdminInterviewQuestionBank@store	perm: interviews

Verb	URI	Handler	Guard
POST	/admin/interview-question-bank/{interviewQuestionBank}	AdminInterviewQuestionBank@update	perm: interviews
GET	/admin/interview-role-templates	AdminInterviewRoleTemplate@index	perm: interviews
POST	/admin/interview-role-templates	AdminInterviewRoleTemplate@store	perm: interviews
POST	/admin/interview-role-templates/{interviewRoleTemplate}	AdminInterviewRoleTemplate@update	perm: interviews
GET	/admin/interview-tracks	AdminInterviewTrack@index	perm: interviews
POST	/admin/interview-tracks	AdminInterviewTrack@store	perm: interviews
DELETE	/admin/interview-tracks/{interviewTrack}	AdminInterviewTrack@destroy	perm: interviews
POST	/admin/interview-tracks/{interviewTrack}	AdminInterviewTrack@update	perm: interviews
POST	/admin/interview-tracks/{interviewTrack}/colleges	AdminInterviewTrack@updateColleges	perm: interviews
GET	/admin/interviews	AdminInterview@index	perm: interviews
POST	/admin/interviews	AdminInterview@store	perm: interviews
DELETE	/admin/interviews/{interview}	AdminInterview@destroy	perm: interviews
POST	/admin/interviews/{interview}	AdminInterview@update	perm: interviews
POST	/admin/interviews/{interview}/colleges	AdminInterview@updateColleges	perm: interviews
GET	/admin/interviews/{interview}/questions	AdminInterview@questions	perm: interviews
POST	/admin/interviews/{interview}/questions	AdminInterview@storeQuestion	perm: interviews
DELETE	/admin/interviews/{interview}/questions/{interviewQuestion}	AdminInterview@destroyQuestion	perm: interviews
GET	/admin/interviews/{interview}/responses/{interviewResponse}/audio	AdminInterview@responseAudio	perm: interviews
POST	/admin/interviews/{interview}/responses/{interviewResponse}/score	AdminInterview@scoreResponse	perm: interviews
GET	/admin/interviews/{interview}/sessions	AdminInterview@sessions	perm: interviews
POST	/admin/interviews/{interview}/sessions/{interviewSession}/finalize	AdminInterview@finalizeSession	perm: interviews
POST	/admin/interviews/{interview}/sessions/{interviewSession}/reinstate	AdminInterview@reinstateProctoring	perm: interviews
GET	/admin/interviews/{interview}/sessions/{interviewSession}/responses	AdminInterview@responses	perm: interviews
GET	/admin/overview	Admin@overview	—
GET	/admin/partner-colleges	AdminPlacementDrive@partnerColleges	perm: placements
GET	/admin/placement-drives	AdminPlacementDrive@index	perm: placements
POST	/admin/placement-drives	AdminPlacementDrive@store	perm: placements
POST	/admin/placement-drives/{placementDrive}	AdminPlacementDrive@update	perm: placements
POST	/admin/placement-drives/{placementDrive}/map-colleges	AdminPlacementDrive@mapColleges	perm: placements
GET	/admin/placement-drives/{placementDrive}/mappings	AdminPlacementDrive@mappings	perm: placements
DELETE	/admin/prep-questions/{prepQuestion}	AdminCompany@destroyPrepQuestion	perm: placements
GET	/admin/problems	AdminProblem@index	perm: problem_bank
POST	/admin/problems	AdminProblem@store	perm: problem_bank
PUT	/admin/problems/{problem}	AdminProblem@update	perm: problem_bank
GET	/admin/project-lab/submissions	AdminProjectLab@submissions	perm: project_lab
GET	/admin/project-lab/submissions/{submission}	AdminProjectLab@showSubmission	perm: project_lab
POST	/admin/project-lab/submissions/{submission}/review	AdminProjectLab@review	perm: project_lab
GET	/admin/project-lab/submissions/{submission}/snapshot	AdminProjectLab@downloadSnapshot	perm: project_lab
GET	/admin/project-lab/topics	AdminProjectLab@topics	perm: project_lab
POST	/admin/project-lab/topics	AdminProjectLab@storeTopic	perm: project_lab
DELETE	/admin/project-lab/topics/{projectTopic}	AdminProjectLab@destroyTopic	perm: project_lab
GET	/admin/project-lab/topics/{projectTopic}	AdminProjectLab@showTopic	perm: project_lab

Verb	URI	Handler	Guard
PUT	/admin/project-lab/topics/{projectTopic}	AdminProjectLab@updateTopic	perm: project_lab
DELETE	/admin/recommended-problems/{recommendedProblem}	AdminCompany@destroyRecommendedProblem	perm: placements
GET	/admin/soft-skill-question-bank	AdminSoftSkillQuestionBank@index	perm: soft_skills
POST	/admin/soft-skill-question-bank	AdminSoftSkillQuestionBank@store	perm: soft_skills
POST	/admin/soft-skill-question-bank/generate	AdminSoftSkillQuestionBank@generate	perm: soft_skills; throttle: ai-generation
POST	/admin/soft-skill-question-bank/{softSkillQuestion}	AdminSoftSkillQuestionBank@update	perm: soft_skills
GET	/admin/soft-skills	AdminSoftSkill@index	perm: soft_skills
POST	/admin/soft-skills	AdminSoftSkill@store	perm: soft_skills
DELETE	/admin/soft-skills/{softSkillAssessment}	AdminSoftSkill@destroy	perm: soft_skills
POST	/admin/soft-skills/{softSkillAssessment}	AdminSoftSkill@update	perm: soft_skills
GET	/admin/soft-skills/{softSkillAssessment}/questions	AdminSoftSkill@questions	perm: soft_skills
POST	/admin/soft-skills/{softSkillAssessment}/questions	AdminSoftSkill@storeQuestion	perm: soft_skills
POST	/admin/soft-skills/{softSkillAssessment}/questions/auto-fill	AdminSoftSkill@autoFillQuestions	perm: soft_skills
DELETE	/admin/soft-skills/{softSkillAssessment}/questions/{assessmentQuestion}	AdminSoftSkill@destroyQuestion	perm: soft_skills
GET	/admin/students/import-template	StudentImport@template	perm: colleges
GET	/admin/students/{student}/contest-submissions/{contestSubmission}	AdminStudentReport@contestSubmission	perm: platform_users
GET	/admin/students/{student}/report	AdminStudentReport@show	perm: platform_users
GET	/admin/students/{student}/submissions/{submission}	AdminStudentReport@submission	perm: platform_users
GET	/admin/talent-pool/candidates	AdminTalentPool@index	perm: talent_pool
POST	/admin/talent-pool/candidates/{candidate}/visibility	AdminTalentPool@updateVisibility	perm: talent_pool
GET	/admin/talent-pool/colleges	AdminTalentPool@colleges	perm: talent_pool
POST	/admin/tpos/{user}/toggle-block	Admin@toggleTpoBlock	perm: colleges
GET	/admin/users	Admin@users	perm: platform_users
POST	/admin/users	Admin@storeUser	perm: platform_users
POST	/admin/users/{user}/toggle-block	Admin@toggleUserBlock	perm: platform_users

Marketing — /marketing/* (roles: admin_marketing, superadmin) — 9 routes

Verb	URI	Handler	Guard
GET	/marketing/contact-requests	MarketingContactRequest@index	perm: leads
GET	/marketing/contact-requests/{contactRequest}	MarketingContactRequest@show	perm: leads
POST	/marketing/contact-requests/{contactRequest}/notes	MarketingContactRequest@storeNote	perm: leads
POST	/marketing/contact-requests/{contactRequest}/status	MarketingContactRequest@updateStatus	perm: leads
GET	/marketing/leads	MarketingLead@index	perm: leads
POST	/marketing/leads/notify	MarketingLead@notify	perm: lead_outreach
GET	/marketing/leads/{lead}	MarketingLead@show	perm: leads
POST	/marketing/leads/{lead}/notes	MarketingLead@storeNote	perm: leads
POST	/marketing/leads/{lead}/status	MarketingLead@updateStatus	perm: leads

Superadmin — /superadmin/* — 11 routes

Verb	URI	Handler	Guard
GET	/superadmin/audit-logs	SuperAdmin@auditLogs	—
GET	/superadmin/feature-flags	SuperAdmin@featureFlags	—
POST	/superadmin/feature-flags/{flag}/toggle	SuperAdmin@toggleFeatureFlag	—
GET	/superadmin/judge-nodes	SuperAdmin@judgeNodes	—
GET	/superadmin/marketing-performance	SuperAdmin@marketingPerformance	—
GET	/superadmin/overview	SuperAdmin@overview	—
GET	/superadmin/users	SuperAdmin@users	—
POST	/superadmin/users	SuperAdmin@storeUser	—
GET	/superadmin/users/{user}/detail	SuperAdmin@showUser	—
POST	/superadmin/users/{user}/permissions	SuperAdmin@updatePermissions	—
POST	/superadmin/users/{user}/toggle-block	SuperAdmin@toggleBlock	—

Shared staff reads — /interview-question-bank/* — 2 routes

Verb	URI	Handler	Guard
GET	/interview-question-bank/browse	InterviewQuestionBank@index	—
POST	/interview-question-bank/generate	InterviewQuestionGeneration@generate	throttle: ai-generation

APPENDIX B

Frontend page → API map

Every one of the 104 pages: its guard, source file and the API calls found in the page and the components/hooks it imports. `→ Controller@method` is resolved against the route table. Calls made through a shared console with a variable base path are shown with the page's resolved base, or as `(A | B | C)@method` when several controller families could answer; `{x}` is a path parameter.

Public pages and sign-in — 22 pages

Page	Guard	API calls
/ src/app/page.tsx	public	POST /contact-requests → ContactRequest@store GET /public/problems/sample → Public@sampleProblems GET /public/stats → Public@stats
/about src/app/about/page.tsx	public	–
/colleges src/app/colleges/page.tsx	public	POST /contact-requests → ContactRequest@store
/contact src/app/contact/page.tsx	public	POST /contact-requests → ContactRequest@store
/features/ai-interviews src/app/features/ai-interviews/page.tsx	public	POST /contact-requests → ContactRequest@store
/features/campus-drives src/app/features/campus-drives/page.tsx	public	POST /contact-requests → ContactRequest@store
/features/campus-hiring src/app/features/campus-hiring/page.tsx	public	POST /contact-requests → ContactRequest@store
/features/company-prep src/app/features/company-prep/page.tsx	public	POST /contact-requests → ContactRequest@store
/features/learning-centre src/app/features/learning-centre/page.tsx	public	POST /contact-requests → ContactRequest@store
/features/placement-analytics src/app/features/placement-analytics/page.tsx	public	POST /contact-requests → ContactRequest@store
/features/practice-arena src/app/features/practice-arena/page.tsx	public	POST /contact-requests → ContactRequest@store
/features/proctored-assessments src/app/features/proctored-assessments/page.tsx	public	POST /contact-requests → ContactRequest@store
/features/student-onboarding src/app/features/student-onboarding/page.tsx	public	POST /contact-requests → ContactRequest@store
/features/talent-pool src/app/features/talent-pool/page.tsx	public	POST /contact-requests → ContactRequest@store
/login src/app/login/page.tsx	public	POST /login → Auth@login POST /password/forgot → PasswordReset@forgot POST /password/otp/verify → PasswordReset@verifyOtp POST /password/reset → PasswordReset@reset GET /public/problems/sample → Public@sampleProblems GET /public/stats → Public@stats
/mellow-internal src/app/mellow-internal/page.tsx	public	POST /login → Auth@login
/pricing src/app/pricing/page.tsx	public	GET /me/purchase-quote → Subscription@quote GET /me/subscription → Subscription@mine POST /me/subscription → Subscription@subscribe
/privacy src/app/privacy/page.tsx	public	–
/recruiters src/app/recruiters/page.tsx	public	POST /contact-requests → ContactRequest@store

Page	Guard	API calls
/signup src/app/signup/page.tsx	public	POST /contact-requests → ContactRequest@store GET /public/problems/sample → Public@sampleProblems GET /public/stats → Public@stats POST /register → Auth@register
/students src/app/students/page.tsx	public	POST /contact-requests → ContactRequest@store
/terms src/app/terms/page.tsx	public	–

Student (/dashboard, /settings) — 33 pages

Page	Guard	API calls
/dashboard src/app/dashboard/page.tsx	user	GET /arena/hackathons → HackathonArena@index GET /arena/home → HackathonArena@home GET /arena/mine → HackathonArena@mine GET /contests → Contest@index GET /drives → StudentDrive@index GET /leaderboard → Leaderboard@index GET /me/rating-history → StudentStats@ratingHistory GET /me/stats → StudentStats@mine GET {x}/gallery → (HackathonArena)@gallery POST {x}/register → (Auth | Contest | HackathonArena)@register GET {x}/results → (CompanyContest | HackathonArena)@results PUT {x}/submission → (HackathonArena)@saveSubmission POST {x}/team/members/{x}/remove → (HackathonArena)@removeMember POST {x}/withdraw → (HackathonArena)@withdraw
/dashboard/articles src/app/dashboard/articles/page.tsx	user	GET /articles/topics → Article@topics
/dashboard/articles/topic src/app/dashboard/articles/topic/page.tsx	user	GET /articles/topics/{x} → Article@topicShow
/dashboard/articles/view src/app/dashboard/articles/view/page.tsx	user	GET /articles/topics/{x} → Article@topicShow GET /articles/{x} → Article@show POST /articles/{x}/read → Article@markRead
/dashboard/billing src/app/dashboard/billing/page.tsx	user	GET /me/purchase-quote → Subscription@quote GET /me/subscription → Subscription@mine POST /me/subscription → Subscription@subscribe
/dashboard/contests src/app/dashboard/contests/page.tsx	user	GET /contests → Contest@index
/dashboard/contests/problem src/app/dashboard/contests/problem/page.tsx	user	GET /judge/{x} → JudgeResult@show POST {x}/proctoring/start → (ContestProctoring | InterviewProctoring | SoftSkillProctoring)@start POST {x}/proctoring/violations → (ContestProctoring | InterviewProctoring | SoftSkillProctoring)@reportViolation
/dashboard/contests/view src/app/dashboard/contests/view/page.tsx	user	GET /contests/{x} → Contest@show GET /contests/{x}/leaderboard → Contest@leaderboard DELETE /contests/{x}/register → Contest@unregister POST /contests/{x}/register → Contest@register
/dashboard/drives src/app/dashboard/drives/page.tsx	user	GET /drives/{x} → StudentDrive@show
/dashboard/interview-tracks src/app/dashboard/interview-tracks/page.tsx	user	GET /interview-tracks/{x} → InterviewTrack@show
/dashboard/interviews src/app/dashboard/interviews/page.tsx	user	GET /interview-tracks → InterviewTrack@index GET /interviews → Interview@index GET /me/subscription → Subscription@mine
/dashboard/interviews/session src/app/dashboard/interviews/session/page.tsx	user	POST {x}/answer → (Interview | SoftSkill | Vocabulary)@answer POST {x}/proctoring/start → (ContestProctoring | InterviewProctoring | SoftSkillProctoring)@start POST {x}/proctoring/violations → (ContestProctoring | InterviewProctoring | SoftSkillProctoring)@reportViolation GET {x}/result → (Interview)@result POST {x}/start → (dynamic base)
/dashboard/interviews/view src/app/dashboard/interviews/view/page.tsx	user	GET /interviews/{x} → Interview@show GET /interviews/{x}/responses/{x}/audio → Interview@responseAudio GET /interviews/{x}/result → Interview@result

Page	Guard	API calls
/dashboard/leaderboard src/app/dashboard/leaderboard/page.tsx	(any signed-in role)	GET /leaderboard → Leaderboard@index
/dashboard/learning-centre src/app/dashboard/learning-centre/page.tsx	user	GET /learning-centre/overview → LearningCentre@overview
/dashboard/learning-centre/listening src/app/dashboard/learning-centre/listening/page.tsx	user	GET /learning-centre/listening/lessons → ListeningLab@index POST /learning-centre/listening/lessons/generate → ListeningLab@generate
/dashboard/learning-centre/listening/session src/app/dashboard/learning-centre/listening/session/page.tsx	user	GET /learning-centre/listening/lessons/{x} → ListeningLab@show POST /learning-centre/listening/lessons/{x}/attempts → ListeningLab@submit
/dashboard/learning-centre/speaking src/app/dashboard/learning-centre/speaking/page.tsx	user	GET /learning-centre/speaking/prompts → SpeakingPractice@index POST /learning-centre/speaking/prompts/generate → SpeakingPractice@generate
/dashboard/learning-centre/speaking/session src/app/dashboard/learning-centre/speaking/session/page.tsx	user	GET /learning-centre/speaking/prompts/{x} → SpeakingPractice@show POST /learning-centre/speaking/prompts/{x}/attempts → SpeakingPractice@submit
/dashboard/learning-centre/vocabulary src/app/dashboard/learning-centre/vocabulary/page.tsx	user	POST /learning-centre/vocabulary/generate → Vocabulary@generate GET /learning-centre/vocabulary/overview → VocabularyProgress@overview
/dashboard/learning-centre/vocabulary/session src/app/dashboard/learning-centre/vocabulary/session/page.tsx	user	GET /learning-centre/vocabulary/attempts/{x} → Vocabulary@show POST /learning-centre/vocabulary/attempts/{x}/answer → Vocabulary@answer POST /learning-centre/vocabulary/sessions → Vocabulary@start
/dashboard/learning-centre/vocabulary/words src/app/dashboard/learning-centre/vocabulary/words/page.tsx	user	GET /learning-centre/vocabulary/words → VocabularyProgress@words
/dashboard/practice src/app/dashboard/practice/page.tsx	user	GET /me/stats → StudentStats@mine GET /me/subscription → Subscription@mine GET /problems → Problem@index
/dashboard/practice/view src/app/dashboard/practice/view/page.tsx	user	GET /judge/{x} → JudgeResult@show GET /problems/{x} → Problem@show
/dashboard/projects src/app/dashboard/projects/page.tsx	user	GET /me/purchase-quote → Subscription@quote GET /me/subscription → Subscription@mine POST /me/subscription → Subscription@subscribe GET /project-lab/topics → ProjectLab@index
/dashboard/projects/view src/app/dashboard/projects/view/page.tsx	user	GET /me/purchase-quote → Subscription@quote GET /me/subscription → Subscription@mine POST /me/subscription → Subscription@subscribe GET /project-lab/topics/{x} → ProjectLab@show GET /project-lab/topics/{x}/sow.pdf → ProjectLab@downloadSow POST /project-lab/topics/{x}/start → ProjectLab@start
/dashboard/projects/workspace src/app/dashboard/projects/workspace/page.tsx	user	POST /project-lab/enrollments/{x}/submit → ProjectWorkspace@submit POST /project-lab/enrollments/{x}/workspace/integrity → ProjectWorkspace@integrity POST {x}/milestones/{x}/toggle → (ProjectLab)@toggleMilestone GET {x}/workspace → (ProjectWorkspace)@show POST {x}/workspace/heartbeat → (ProjectWorkspace)@heartbeat POST {x}/workspace/start → (ProjectWorkspace)@start POST {x}/workspace/stop → (ProjectWorkspace)@stop
/dashboard/reports src/app/dashboard/reports/page.tsx	user	GET /drives → StudentDrive@index GET /me/interviews → Interview@history GET /me/rating-history → StudentStats@ratingHistory GET /me/soft-skills → SoftSkill@history GET /me/stats → StudentStats@mine GET /submissions → MySubmissions@index
/dashboard/soft-skills src/app/dashboard/soft-skills/page.tsx	user	GET /soft-skills → SoftSkill@index
/dashboard/soft-skills/session src/app/dashboard/soft-skills/session/page.tsx	user	POST /soft-skills/sessions/{x}/answer → SoftSkill@answer POST /soft-skills/sessions/{x}/submit → SoftSkill@submit GET /soft-skills/{x} → SoftSkill@show POST /soft-skills/{x}/start → SoftSkill@start POST {x}/proctoring/start → (ContestProctoring | InterviewProctoring | SoftSkillProctoring)@start POST {x}/proctoring/violations → (ContestProctoring | InterviewProctoring | SoftSkillProctoring)@reportViolation

Page	Guard	API calls
/dashboard/soft-skills/view src/app/dashboard/soft-skills/view/page.tsx	user	GET /soft-skills/sessions/{x} → SoftSkill@viewSession
/dashboard/talent-pool src/app/dashboard/talent-pool/page.tsx	user	GET /me/talent-pool → StudentTalentPool@mine POST /me/talent-pool/inquiries/{x}/respond → StudentTalentPool@respondToInterview POST /me/talent-pool/opt-in → StudentTalentPool@optIn POST /me/talent-pool/opt-out → StudentTalentPool@optOut
/settings src/app/settings/page.tsx	(any signed-in role)	DELETE /me/avatar → Auth@deleteAvatar POST /me/avatar → Auth@updateAvatar PUT /me/password → Auth@updatePassword POST /me/phone/verify → PhoneVerification@verify PUT /me/profile → Auth@updateProfile DELETE /me/resume → StudentProfile@deleteResume GET /me/resume → StudentProfile@downloadOwnResume POST /me/resume → StudentProfile@uploadResume GET /me/sessions → Auth@sessions DELETE /me/sessions/{x} → Auth@revokeSession PUT /me/student-profile → StudentProfile@update GET /me/subscription → Subscription@mine

Hack Arena (/arena) — 3 pages

Page	Guard	API calls
/arena src/app/arena/page.tsx	(any signed-in role)	GET /arena/hackathons → HackathonArena@index GET /arena/home → HackathonArena@home GET /arena/mine → HackathonArena@mine GET /arena/hackathons/{x}/gallery → HackathonArena@gallery POST /arena/hackathons/{x}/register → HackathonArena@register GET /arena/hackathons/{x}/results → HackathonArena@results PUT /arena/hackathons/{x}/submission → HackathonArena@saveSubmission POST /arena/hackathons/{x}/team/members/{x}/remove → HackathonArena@removeMember POST /arena/hackathons/{x}/withdraw → HackathonArena@withdraw
/arena/hackathon src/app/arena/hackathon/page.tsx	(any signed-in role)	GET /arena/hackathons → HackathonArena@index GET /arena/home → HackathonArena@home GET /arena/mine → HackathonArena@mine GET /arena/hackathons/{x}/gallery → HackathonArena@gallery POST /arena/hackathons/{x}/register → HackathonArena@register GET /arena/hackathons/{x}/results → HackathonArena@results PUT /arena/hackathons/{x}/submission → HackathonArena@saveSubmission POST /arena/hackathons/{x}/team/members/{x}/remove → HackathonArena@removeMember POST /arena/hackathons/{x}/withdraw → HackathonArena@withdraw
/arena/my src/app/arena/my/page.tsx	user	GET /arena/hackathons → HackathonArena@index GET /arena/home → HackathonArena@home GET /arena/mine → HackathonArena@mine GET /arena/hackathons/{x}/gallery → HackathonArena@gallery POST /arena/hackathons/{x}/register → HackathonArena@register GET /arena/hackathons/{x}/results → HackathonArena@results PUT /arena/hackathons/{x}/submission → HackathonArena@saveSubmission POST /arena/hackathons/{x}/team/members/{x}/remove → HackathonArena@removeMember POST /arena/hackathons/{x}/withdraw → HackathonArena@withdraw

Mellow Ops and TPO pages (/admin/*, except /admin/company/*) — 28 pages

Page	Guard	API calls
/admin src/app/admin/page.tsx	ops, tpo, sa, co	GET /admin/colleges → Admin@colleges POST /admin/colleges → Admin@storeCollege GET /admin/colleges/{x}/coordinators → Admin@collegeCoordinators POST /admin/colleges/{x}/coordinators → Admin@storeCoordinatorForCollege PUT /admin/colleges/{x}/seats → Admin@adjustCollegeSeats POST /admin/colleges/{x}/subscription → Admin@assignCollegeSubscription PUT /admin/coordinators/{x} → Admin@updateCoordinatorSection POST /admin/coordinators/{x}/toggle-block → Admin@toggleCoordinatorBlock GET /admin/problems → AdminProblem@index POST /admin/tpos/{x}/toggle-block → Admin@toggleTpoBlock GET /admin/users → Admin@users POST /admin/users → Admin@storeUser POST /admin/users/{x}/toggle-block → Admin@toggleUserBlock GET /company/drives → CompanyDrive@index GET /company/reports/data → CompanyReports@data GET /me/subscription → Subscription@mine GET /tpo/coordinators → TpoCoordinator@index GET /tpo/drives/mapped → TpoDrive@mapped POST /tpo/placement-target → TpoReports@updateTarget GET /tpo/reports/data → TpoReports@data POST {x}/import → (CompanyCandidate | CompanyContest | StudentImport)@storeForCollege GET {x}/imports → (CompanyCandidate | StudentImport)@indexForCollege
/admin/analytics src/app/admin/analytics/page.tsx	tpo	GET /tpo/reports/data → TpoReports@data
/admin/articles src/app/admin/articles/page.tsx	ops, sa	GET /admin/article-topics → AdminArticleTopic@index POST /admin/article-topics → AdminArticleTopic@store DELETE /admin/article-topics/{x} → AdminArticleTopic@destroy POST /admin/article-topics/{x} → AdminArticleTopic@update GET /admin/articles → AdminArticle@index DELETE /admin/articles/{x} → AdminArticle@destroy POST /admin/articles/{x} → AdminArticle@update
/admin/articles/edit src/app/admin/articles/edit/page.tsx	ops, sa	GET /admin/article-topics → AdminArticleTopic@index POST /admin/articles → AdminArticle@store GET /admin/articles/{x} → AdminArticle@show POST /admin/articles/{x} → AdminArticle@update
/admin/articles/new src/app/admin/articles/new/page.tsx	ops, sa	GET /admin/article-topics → AdminArticleTopic@index POST /admin/articles → AdminArticle@store POST /admin/articles/{x} → AdminArticle@update
/admin/contests src/app/admin/contests/page.tsx	ops, sa	GET /admin/companies → AdminCompany@index GET /admin/companies/{x}/recommended-problems → AdminCompany@recommendedProblems GET /admin/contests → AdminContest@index POST /admin/contests → AdminContest@store POST /admin/contests/{x} → AdminContest@update POST /admin/contests/{x}/colleges → AdminContest@updateColleges POST /admin/contests/{x}/finalize → AdminContest@finalize GET /admin/contests/{x}/problems → AdminContest@problems POST /admin/contests/{x}/problems → AdminContest@storeProblem DELETE /admin/contests/{x}/problems/{x} → AdminContest@destroyProblem GET /admin/partner-colleges → AdminPlacementDrive@partnerColleges GET /admin/placement-drives → AdminPlacementDrive@index POST /admin/placement-drives/{x}/map-colleges → AdminPlacementDrive@mapColleges GET /admin/placement-drives/{x}/mappings → AdminPlacementDrive@mappings GET /problems → Problem@index GET {x}/students/{x}/contest-submissions/{x} → (AdminStudentReport | CoordinatorStudentReport | TpoStudentReport)@contestSubmission GET /admin/contests/{x}/participants → AdminContest@participants GET /admin/contests/{x}/participants/{x}/submissions → AdminContest@participantSubmissions
/admin/coordinators src/app/admin/coordinators/page.tsx	tpo	GET /tpo/coordinators → TpoCoordinator@index POST /tpo/coordinators → TpoCoordinator@store PUT /tpo/coordinators/{x} → TpoCoordinator@update POST /tpo/coordinators/{x}/toggle-block → TpoCoordinator@toggleBlock
/admin/customers src/app/admin/customers/page.tsx	ops, sa	GET /admin/customers → InternalCustomer@index GET /admin/customers/{x} → InternalCustomer@show POST /admin/customers/{x}/notes → InternalCustomer@storeNote

Page	Guard	API calls
/admin/drives src/app/admin/drives/page.tsx	tpo	GET /tpo/companies/search → TpoDrive@searchCompanies POST /tpo/drives → TpoDrive@store GET /tpo/drives/available → TpoDrive@available GET /tpo/drives/mapped → TpoDrive@mapped GET /tpo/drives/pending → TpoDrive@pending GET /tpo/drives/{x}/applications → TpoDriveApplication@index POST /tpo/drives/{x}/applications → TpoDriveApplication@store POST /tpo/drives/{x}/applications/bulk-register → TpoDriveApplication@bulkRegister POST /tpo/drives/{x}/applications/{x}/stage → TpoDriveApplication@updateStage POST /tpo/drives/{x}/map → TpoDrive@map POST /tpo/drives/{x}/respond → TpoDrive@respond POST /tpo/drives/{x}/unmap → TpoDrive@unmap GET /tpo/students/cohort → TpoStudent@index
/admin/hackathons src/app/admin/hackathons/page.tsx	ops, sa	GET /arena/hackathons → HackathonArena@index GET /arena/home → HackathonArena@home GET /arena/mine → HackathonArena@mine POST /admin/hackathons/{x}/announcements → AdminHackathon@storeAnnouncement DELETE /admin/hackathons/{x}/announcements/{x} → AdminHackathon@destroyAnnouncement POST /admin/hackathons/{x}/cancel → AdminHackathon@cancel GET /admin/hackathons/colleges → AdminHackathon@colleges DELETE /admin/hackathons/cover → AdminHackathon@destroy POST /admin/hackathons/{x}/cover → AdminHackathon@uploadCover GET /admin/hackathons/gallery → AdminHackathon@show POST /admin/hackathons/{x}/publish → AdminHackathon@publish POST /admin/hackathons/{x}/publish-results → AdminHackathon@publishResults POST {x}/register → (Auth | Contest | HackathonArena)@register GET /admin/hackathons/registrations → AdminHackathon@show GET /admin/hackathons/{x}/registrations/export → AdminHackathon@exportRegistrations GET /admin/hackathons/results → AdminHackathon@show PUT /admin/hackathons/submission → AdminHackathon@update GET /admin/hackathons/submissions → AdminHackathon@show POST /admin/hackathons/{x}/submissions/{x}/disqualify → AdminHackathon@disqualify POST /admin/hackathons/{x}/submissions/{x}/reinstate → AdminHackathon@reinstate POST /admin/hackathons/{x}/submissions/{x}/score → AdminHackathon@scoreSubmission POST {x}/team/members/{x}/remove → (HackathonArena)@removeMember POST /admin/hackathons/{x}/unpublish → AdminHackathon@unpublish POST {x}/withdraw → (HackathonArena)@withdraw
/admin/hackathons/edit src/app/admin/hackathons/edit/page.tsx	ops, sa	GET /arena/hackathons → HackathonArena@index GET /arena/home → HackathonArena@home GET /arena/mine → HackathonArena@mine POST /admin/hackathons/{x}/announcements → AdminHackathon@storeAnnouncement DELETE /admin/hackathons/{x}/announcements/{x} → AdminHackathon@destroyAnnouncement POST /admin/hackathons/{x}/cancel → AdminHackathon@cancel GET /admin/hackathons/colleges → AdminHackathon@colleges DELETE /admin/hackathons/cover → AdminHackathon@destroy POST /admin/hackathons/{x}/cover → AdminHackathon@uploadCover GET /admin/hackathons/gallery → AdminHackathon@show POST /admin/hackathons/{x}/publish → AdminHackathon@publish POST /admin/hackathons/{x}/publish-results → AdminHackathon@publishResults POST {x}/register → (Auth | Contest | HackathonArena)@register GET /admin/hackathons/registrations → AdminHackathon@show GET /admin/hackathons/{x}/registrations/export → AdminHackathon@exportRegistrations GET /admin/hackathons/results → AdminHackathon@show PUT /admin/hackathons/submission → AdminHackathon@update GET /admin/hackathons/submissions → AdminHackathon@show POST /admin/hackathons/{x}/submissions/{x}/disqualify → AdminHackathon@disqualify POST /admin/hackathons/{x}/submissions/{x}/reinstate → AdminHackathon@reinstate POST /admin/hackathons/{x}/submissions/{x}/score → AdminHackathon@scoreSubmission POST {x}/team/members/{x}/remove → (HackathonArena)@removeMember POST /admin/hackathons/{x}/unpublish → AdminHackathon@unpublish POST {x}/withdraw → (HackathonArena)@withdraw

Page	Guard	API calls
/admin/hackathons/manage src/app/admin/hackathons/manage/page.tsx	ops, sa	GET /arena/hackathons → HackathonArena@index GET /arena/home → HackathonArena@home GET /arena/mine → HackathonArena@mine POST /admin/hackathons/{x}/announcements → AdminHackathon@storeAnnouncement DELETE /admin/hackathons/{x}/announcements/{x} → AdminHackathon@destroyAnnouncement POST /admin/hackathons/{x}/cancel → AdminHackathon@cancel GET /admin/hackathons/colleges → AdminHackathon@colleges DELETE /admin/hackathons/cover → AdminHackathon@destroy POST /admin/hackathons/{x}/cover → AdminHackathon@uploadCover GET /admin/hackathons/gallery → AdminHackathon@show POST /admin/hackathons/{x}/publish → AdminHackathon@publish POST /admin/hackathons/{x}/publish-results → AdminHackathon@publishResults POST {x}/register → (Auth | Contest | HackathonArena)@register GET /admin/hackathons/registrations → AdminHackathon@show GET /admin/hackathons/{x}/registrations/export → AdminHackathon@exportRegistrations GET /admin/hackathons/results → AdminHackathon@show PUT /admin/hackathons/submission → AdminHackathon@update GET /admin/hackathons/submissions → AdminHackathon@show POST /admin/hackathons/{x}/submissions/{x}/disqualify → AdminHackathon@disqualify POST /admin/hackathons/{x}/submissions/{x}/reinstate → AdminHackathon@reinstate POST /admin/hackathons/{x}/submissions/{x}/score → AdminHackathon@scoreSubmission POST {x}/team/members/{x}/remove → (HackathonArena)@removeMember POST /admin/hackathons/{x}/unpublish → AdminHackathon@unpublish POST {x}/withdraw → (HackathonArena)@withdraw
/admin/interviews src/app/admin/interviews/page.tsx	ops, sa	GET /admin/companies → AdminCompany@index GET /admin/partner-colleges → AdminPlacementDrive@partnerColleges GET /admin/placement-drives → AdminPlacementDrive@index POST /admin/placement-drives/{x}/map-colleges → AdminPlacementDrive@mapColleges GET /admin/placement-drives/{x}/mappings → AdminPlacementDrive@mappings GET /interview-question-bank/browse → InterviewQuestionBank@index POST /interview-question-bank/generate → InterviewQuestionGeneration@generate DELETE /admin/interviews/{x} → AdminInterview@destroy POST /admin/interviews/{x} → AdminInterview@update POST /admin/interviews/{x}/colleges → AdminInterview@updateColleges GET /admin/interviews/{x}/questions → AdminInterview@questions POST /admin/interviews/{x}/questions → AdminInterview@storeQuestion DELETE /admin/interviews/{x}/questions/{x} → AdminInterview@destroyQuestion
/admin/interviews/question-bank src/app/admin/interviews/question-bank/page.tsx	ops, sa	GET /admin/interview-question-bank → AdminInterviewQuestionBank@index POST /admin/interview-question-bank → AdminInterviewQuestionBank@store POST /admin/interview-question-bank/{x} → AdminInterviewQuestionBank@update
/admin/interviews/tracks src/app/admin/interviews/tracks/page.tsx	ops, sa	GET /admin/companies → AdminCompany@index GET /admin/partner-colleges → AdminPlacementDrive@partnerColleges GET /admin/placement-drives → AdminPlacementDrive@index POST /admin/placement-drives/{x}/map-colleges → AdminPlacementDrive@mapColleges GET /admin/placement-drives/{x}/mappings → AdminPlacementDrive@mappings GET /company/drives → CompanyDrive@index POST /interview-question-bank/generate → InterviewQuestionGeneration@generate DELETE {x}/{x} → (dynamic base) POST {x}/{x} → (dynamic base) POST {x}/{x}/colleges → (Admin | AdminContest | AdminInterview…)@storeCollege GET {x}/{x}/questions → (AdminInterview | AdminSoftSkill | CompanyInterview…)@questions POST {x}/{x}/questions → (AdminInterview | AdminSoftSkill | CompanyInterview…)@storeQuestion GET {x}/{x}/responses/{x}/audio → (AdminInterview | CompanyInterview | Interview…)@responseAudio POST {x}/{x}/responses/{x}/score → (AdminInterview | CompanyInterview | TpoInterview)@scoreResponse POST {x}/{x}/sessions/{x}/finalize → (AdminInterview | CompanyInterview | TpoInterview)@finalizeSession POST {x}/{x}/sessions/{x}/reinstate → (AdminInterview | CompanyInterview | TpoInterview)@reinstateProctoring GET {x}/{x}/sessions/{x}/responses → (AdminInterview | CompanyInterview | TpoInterview)@responses GET {x}/{x}/{x} → (dynamic base)
/admin/mock-contests src/app/admin/mock-contests/page.tsx	tpo	GET /problems → Problem@index GET /tpo/contests → TpoContest@index POST /tpo/contests → TpoContest@store POST /tpo/contests/{x} → TpoContest@update POST /tpo/contests/{x}/finalize → TpoContest@finalize GET /tpo/contests/{x}/problems → TpoContest@problems POST /tpo/contests/{x}/problems → TpoContest@storeProblem DELETE /tpo/contests/{x}/problems/{x} → TpoContest@destroyProblem GET {x}/students/{x}/contest-submissions/{x} → (AdminStudentReport | CoordinatorStudentReport | TpoStudentReport)@contestSubmission GET /tpo/contests/{x}/participants → TpoContest@participants GET /tpo/contests/{x}/participants/{x}/submissions → TpoContest@participantSubmissions

Page	Guard	API calls
/admin/mock-interviews src/app/admin/mock-interviews/page.tsx	tpo	GET /interview-question-bank/browse → InterviewQuestionBank@index POST /interview-question-bank/generate → InterviewQuestionGeneration@generate DELETE /tpo/interviews/{x} → TpoInterview@destroy POST /tpo/interviews/{x} → TpoInterview@update GET /tpo/interviews/{x}/questions → TpoInterview@questions POST /tpo/interviews/{x}/questions → TpoInterview@storeQuestion DELETE /tpo/interviews/{x}/questions/{x} → TpoInterview@destroyQuestion GET /tpo/interviews/{x}/responses/{x}/audio → TpoInterview@responseAudio POST /tpo/interviews/{x}/responses/{x}/score → TpoInterview@scoreResponse POST /tpo/interviews/{x}/sessions/{x}/finalize → TpoInterview@finalizeSession POST /tpo/interviews/{x}/sessions/{x}/reinstate → TpoInterview@reinstateProctoring GET /tpo/interviews/{x}/sessions/{x}/responses → TpoInterview@responses GET {x}/{x}/{x} → (dynamic base)
/admin/mock-soft-skills src/app/admin/mock-soft-skills/page.tsx	tpo	POST /tpo/soft-skills/generate → TpoSoftSkill@update DELETE /tpo/soft-skills/{x} → TpoSoftSkill@destroy POST /tpo/soft-skills/{x} → TpoSoftSkill@update GET /tpo/soft-skills/{x}/questions → TpoSoftSkill@questions POST /tpo/soft-skills/{x}/questions/auto-fill → TpoSoftSkill@autoFillQuestions DELETE /tpo/soft-skills/{x}/questions/{x} → TpoSoftSkill@destroyQuestion
/admin/placements src/app/admin/placements/page.tsx	ops, sa	GET /admin/companies → AdminCompany@index POST /admin/companies → AdminCompany@store POST /admin/companies/admins/{x}/toggle-block → AdminCompany@toggleAdminBlock POST /admin/companies/{x} → AdminCompany@update GET /admin/companies/{x}/prep-questions → AdminCompany@prepQuestions POST /admin/companies/{x}/prep-questions → AdminCompany@storePrepQuestion GET /admin/companies/{x}/recommended-problems → AdminCompany@recommendedProblems POST /admin/companies/{x}/recommended-problems → AdminCompany@storeRecommendedProblem GET /admin/partner-colleges → AdminPlacementDrive@partnerColleges GET /admin/placement-drives → AdminPlacementDrive@index POST /admin/placement-drives → AdminPlacementDrive@store POST /admin/placement-drives/{x} → AdminPlacementDrive@update POST /admin/placement-drives/{x}/map-colleges → AdminPlacementDrive@mapColleges DELETE /admin/prep-questions/{x} → AdminCompany@destroyPrepQuestion DELETE /admin/recommended-problems/{x} → AdminCompany@destroyRecommendedProblem
/admin/proctoring src/app/admin/proctoring/page.tsx	tpo	GET /tpo/proctoring → TpoProctoring@index GET /tpo/proctoring/{x} → TpoProctoring@show POST /tpo/proctoring/{x}/reinstate → TpoProctoring@reinstate
/admin/projects src/app/admin/projects/page.tsx	ops, sa	GET /admin/project-lab/submissions → AdminProjectLab@submissions GET /admin/project-lab/topics → AdminProjectLab@topics
/admin/projects/edit src/app/admin/projects/edit/page.tsx	ops, sa	POST /admin/project-lab/topics → AdminProjectLab@storeTopic GET /admin/project-lab/topics/{x} → AdminProjectLab@showTopic PUT /admin/project-lab/topics/{x} → AdminProjectLab@updateTopic
/admin/projects/review src/app/admin/projects/review/page.tsx	ops, sa	GET /admin/project-lab/submissions/{x} → AdminProjectLab@showSubmission POST /admin/project-lab/submissions/{x}/review → AdminProjectLab@review GET /admin/project-lab/submissions/{x}/snapshot → AdminProjectLab@downloadSnapshot
/admin/reports src/app/admin/reports/page.tsx	tpo	GET /tpo/coordinators → TpoCoordinator@index GET /tpo/reports/data → TpoReports@data
/admin/soft-skills src/app/admin/soft-skills/page.tsx	ops, sa	POST /admin/soft-skills/generate → AdminSoftSkill@update DELETE /admin/soft-skills/{x} → AdminSoftSkill@destroy POST /admin/soft-skills/{x} → AdminSoftSkill@update GET /admin/soft-skills/{x}/questions → AdminSoftSkill@questions POST /admin/soft-skills/{x}/questions/auto-fill → AdminSoftSkill@autoFillQuestions DELETE /admin/soft-skills/{x}/questions/{x} → AdminSoftSkill@destroyQuestion
/admin/students src/app/admin/students/page.tsx	tpo	GET /tpo/coordinators → TpoCoordinator@index POST /tpo/students → TpoStudent@store GET /tpo/students/cohort → TpoStudent@index POST {x}/import → (CompanyCandidate | CompanyContest | StudentImport)@storeForCollege GET {x}/imports → (CompanyCandidate | StudentImport)@indexForCollege
/admin/students/report src/app/admin/students/report/page.tsx	tpo, ops, sa, coord	GET {x}/contests/{x}/participants/{x}/submissions → (AdminContest | TpoContest)@participantSubmissions GET {x}/students/{x}/contest-submissions/{x} → (AdminStudentReport | CoordinatorStudentReport | TpoStudentReport)@contestSubmission GET {x}/students/{x}/report → (AdminStudentReport | CoordinatorStudentReport | TpoStudentReport)@show GET {x}/{x}/responses/{x}/audio → (AdminInterview | CompanyInterview | Interview…)@responseAudio POST {x}/{x}/responses/{x}/score → (AdminInterview | CompanyInterview | TpoInterview)@scoreResponse POST {x}/{x}/sessions/{x}/finalize → (AdminInterview | CompanyInterview | TpoInterview)@finalizeSession POST {x}/{x}/sessions/{x}/reinstate → (AdminInterview | CompanyInterview | TpoInterview)@reinstateProctoring GET {x}/{x}/sessions/{x}/responses → (AdminInterview | CompanyInterview | TpoInterview)@responses GET {x}/{x}/{x} → (dynamic base)

Page	Guard	API calls
/admin/talent-pool src/app/admin/talent-pool/page.tsx	ops, sa	GET /admin/contests → AdminContest@index POST /admin/contests → AdminContest@store POST /admin/contests/{x} → AdminContest@update POST /admin/contests/{x}/finalize → AdminContest@finalize GET /admin/contests/{x}/problems → AdminContest@problems POST /admin/contests/{x}/problems → AdminContest@storeProblem DELETE /admin/contests/{x}/problems/{x} → AdminContest@destroyProblem GET /admin/interviews → AdminInterview@index POST /admin/interviews → AdminInterview@store POST /admin/interviews/{x} → AdminInterview@update GET /admin/interviews/{x}/questions → AdminInterview@questions POST /admin/interviews/{x}/questions → AdminInterview@storeQuestion DELETE /admin/interviews/{x}/questions/{x} → AdminInterview@destroyQuestion GET /admin/talent-pool/candidates → AdminTalentPool@index POST /admin/talent-pool/candidates/{x}/visibility → AdminTalentPool@updateVisibility GET /admin/talent-pool/colleges → AdminTalentPool@colleges GET /interview-question-bank/browse → InterviewQuestionBank@index GET /problems → Problem@index

Hiring partner (/admin/company/*) — 12 pages

Page	Guard	API calls
/admin/company/assessments src/app/admin/company/assessments/page.tsx	co	GET /company/contests → CompanyContest@index POST /company/contests → CompanyContest@store POST /company/contests/{x} → CompanyContest@update POST /company/contests/{x}/finalize → CompanyContest@finalize POST /company/contests/{x}/invite → CompanyContest@inviteCandidates GET /company/contests/{x}/invited → CompanyContest@invited GET /company/contests/{x}/problems → CompanyContest@problems POST /company/contests/{x}/problems → CompanyContest@storeProblem DELETE /company/contests/{x}/problems/{x} → CompanyContest@destroyProblem GET /company/contests/{x}/results → CompanyContest@results POST /company/contests/{x}/results/import → CompanyContest@importResults GET /company/drives → CompanyDrive@index GET /company/drives/{x}/candidates → CompanyCandidate@index GET /problems → Problem@index
/admin/company/candidates src/app/admin/company/candidates/page.tsx	co	GET /company/candidates/import-template → CompanyCandidate@template GET /company/drives → CompanyDrive@index GET /company/drives/{x}/candidates → CompanyCandidate@index POST /company/drives/{x}/candidates → CompanyCandidate@store POST /company/drives/{x}/candidates/import → CompanyCandidate@import GET /company/drives/{x}/candidates/imports → CompanyCandidate@indexImports POST /company/drives/{x}/candidates/{x}/stage → CompanyCandidate@updateStage
/admin/company/colleges src/app/admin/company/colleges/page.tsx	co	GET /company/colleges → CompanyCollege@index GET /company/drives → CompanyDrive@index POST /company/drives/{x}/propose → CompanyDrive@proposeToColleges
/admin/company/drives src/app/admin/company/drives/page.tsx	co	GET /company/colleges → CompanyCollege@index GET /company/drives → CompanyDrive@index POST /company/drives → CompanyDrive@store POST /company/drives/{x} → CompanyDrive@update POST /company/drives/{x}/propose → CompanyDrive@proposeToColleges
/admin/company/hackathons src/app/admin/company/hackathons/page.tsx	co	GET /arena/hackathons → HackathonArena@index GET /arena/home → HackathonArena@home GET /arena/mine → HackathonArena@mine POST /company/hackathons/{x}/announcements → CompanyHackathon@storeAnnouncement DELETE /company/hackathons/{x}/announcements/{x} → CompanyHackathon@destroyAnnouncement POST /company/hackathons/{x}/cancel → CompanyHackathon@cancel GET /company/hackathons/colleges → CompanyHackathon@colleges DELETE /company/hackathons/cover → CompanyHackathon@destroy POST /company/hackathons/{x}/cover → CompanyHackathon@uploadCover GET /company/hackathons/gallery → CompanyHackathon@show POST /company/hackathons/{x}/publish → CompanyHackathon@publish POST /company/hackathons/{x}/publish-results → CompanyHackathon@publishResults POST {x}/register → (Auth | Contest | HackathonArena)@register GET /company/hackathons/registrations → CompanyHackathon@show GET /company/hackathons/{x}/registrations/export → CompanyHackathon@exportRegistrations GET /company/hackathons/results → CompanyHackathon@show PUT /company/hackathons/submission → CompanyHackathon@update GET /company/hackathons/submissions → CompanyHackathon@show POST /company/hackathons/{x}/submissions/{x}/disqualify → CompanyHackathon@disqualify POST /company/hackathons/{x}/submissions/{x}/reinstate → CompanyHackathon@reinstate POST /company/hackathons/{x}/submissions/{x}/score → CompanyHackathon@scoreSubmission POST {x}/team/members/{x}/remove → (HackathonArena)@removeMember POST /company/hackathons/{x}/unpublish → CompanyHackathon@unpublish POST {x}/withdraw → (HackathonArena)@withdraw

Page	Guard	API calls
/admin/company/hackathons/edit src/app/admin/company/hackathons/edit/page.tsx	co	GET /arena/hackathons → HackathonArena@index GET /arena/home → HackathonArena@home GET /arena/mine → HackathonArena@mine POST /company/hackathons/{x}/announcements → CompanyHackathon@storeAnnouncement DELETE /company/hackathons/{x}/announcements/{x} → CompanyHackathon@destroyAnnouncement POST /company/hackathons/{x}/cancel → CompanyHackathon@cancel GET /company/hackathons/colleges → CompanyHackathon@colleges DELETE /company/hackathons/cover → CompanyHackathon@destroy POST /company/hackathons/{x}/cover → CompanyHackathon@uploadCover GET /company/hackathons/gallery → CompanyHackathon@show POST /company/hackathons/{x}/publish → CompanyHackathon@publish POST /company/hackathons/{x}/publish-results → CompanyHackathon@publishResults POST {x}/register → (Auth | Contest | HackathonArena)@register GET /company/hackathons/registrations → CompanyHackathon@show GET /company/hackathons/{x}/registrations/export → CompanyHackathon@exportRegistrations GET /company/hackathons/results → CompanyHackathon@show PUT /company/hackathons/submission → CompanyHackathon@update GET /company/hackathons/submissions → CompanyHackathon@show POST /company/hackathons/{x}/submissions/{x}/disqualify → CompanyHackathon@disqualify POST /company/hackathons/{x}/submissions/{x}/reinstate → CompanyHackathon@reinstate POST /company/hackathons/{x}/submissions/{x}/score → CompanyHackathon@scoreSubmission POST {x}/team/members/{x}/remove → (HackathonArena)@removeMember POST /company/hackathons/{x}/unpublish → CompanyHackathon@unpublish POST {x}/withdraw → (HackathonArena)@withdraw
/admin/company/hackathons/manage src/app/admin/company/hackathons/manage/page.tsx	co	GET /arena/hackathons → HackathonArena@index GET /arena/home → HackathonArena@home GET /arena/mine → HackathonArena@mine POST /company/hackathons/{x}/announcements → CompanyHackathon@storeAnnouncement DELETE /company/hackathons/{x}/announcements/{x} → CompanyHackathon@destroyAnnouncement POST /company/hackathons/{x}/cancel → CompanyHackathon@cancel GET /company/hackathons/colleges → CompanyHackathon@colleges DELETE /company/hackathons/cover → CompanyHackathon@destroy POST /company/hackathons/{x}/cover → CompanyHackathon@uploadCover GET /company/hackathons/gallery → CompanyHackathon@show POST /company/hackathons/{x}/publish → CompanyHackathon@publish POST /company/hackathons/{x}/publish-results → CompanyHackathon@publishResults POST {x}/register → (Auth | Contest | HackathonArena)@register GET /company/hackathons/registrations → CompanyHackathon@show GET /company/hackathons/{x}/registrations/export → CompanyHackathon@exportRegistrations GET /company/hackathons/results → CompanyHackathon@show PUT /company/hackathons/submission → CompanyHackathon@update GET /company/hackathons/submissions → CompanyHackathon@show POST /company/hackathons/{x}/submissions/{x}/disqualify → CompanyHackathon@disqualify POST /company/hackathons/{x}/submissions/{x}/reinstate → CompanyHackathon@reinstate POST /company/hackathons/{x}/submissions/{x}/score → CompanyHackathon@scoreSubmission POST {x}/team/members/{x}/remove → (HackathonArena)@removeMember POST /company/hackathons/{x}/unpublish → CompanyHackathon@unpublish POST {x}/withdraw → (HackathonArena)@withdraw
/admin/company/interviews src/app/admin/company/interviews/page.tsx	co	GET /company/drives → CompanyDrive@index GET /company/drives/{x}/candidates → CompanyCandidate@index POST /company/interviews/{x}/invite → CompanyInterview@inviteCandidates GET /company/interviews/{x}/invited → CompanyInterview@invited GET /interview-question-bank/browse → InterviewQuestionBank@index POST /interview-question-bank/generate → InterviewQuestionGeneration@generate DELETE /company/interviews/{x} → CompanyInterview@destroy POST /company/interviews/{x} → CompanyInterview@update GET /company/interviews/{x}/questions → CompanyInterview@questions POST /company/interviews/{x}/questions → CompanyInterview@storeQuestion DELETE /company/interviews/{x}/questions/{x} → CompanyInterview@destroyQuestion GET /company/interviews/{x}/responses/{x}/audio → CompanyInterview@responseAudio POST /company/interviews/{x}/responses/{x}/score → CompanyInterview@scoreResponse POST /company/interviews/{x}/sessions/{x}/finalize → CompanyInterview@finalizeSession POST /company/interviews/{x}/sessions/{x}/reinstate → CompanyInterview@reinstateProctoring GET /company/interviews/{x}/sessions/{x}/responses → CompanyInterview@responses GET {x}/{x}/{x} → (dynamic base)
/admin/company/proctoring src/app/admin/company/proctoring/page.tsx	co	GET /company/proctoring → CompanyProctoring@index GET /company/proctoring/{x} → CompanyProctoring@show POST /company/proctoring/{x}/reinstate → CompanyProctoring@reinstate
/admin/company/reports src/app/admin/company/reports/page.tsx	co	GET /company/reports/data → CompanyReports@data
/admin/company/soft-skills src/app/admin/company/soft-skills/page.tsx	co	POST /company/soft-skills/generate → CompanySoftSkill@update DELETE /company/soft-skills/{x} → CompanySoftSkill@destroy POST /company/soft-skills/{x} → CompanySoftSkill@update GET /company/soft-skills/{x}/questions → CompanySoftSkill@questions POST /company/soft-skills/{x}/questions/auto-fill → CompanySoftSkill@autoFillQuestions DELETE /company/soft-skills/{x}/questions/{x} → CompanySoftSkill@destroyQuestion

Page	Guard	API calls
/admin/company/talent-pool src/app/admin/company/talent-pool/page.tsx	co	GET /company/colleges → CompanyCollege@index GET /company/talent-pool → CompanyTalentPool@index GET /company/talent-pool/inquiries → CompanyTalentPool@myInquiries GET /company/talent-pool/{x} → CompanyTalentPool@show POST /company/talent-pool/{x}/hire → CompanyTalentPool@hire POST /company/talent-pool/{x}/interest → CompanyTalentPool@expressInterest POST /company/talent-pool/{x}/interview → CompanyTalentPool@scheduleInterview POST /company/talent-pool/{x}/notify → CompanyTalentPool@notify GET /company/talent-pool/{x}/resume → CompanyTalentPool@resume

Section Coordinator — 3 pages

Page	Guard	API calls
/coordinator src/app/coordinator/page.tsx	coord	GET /coordinator/students → CoordinatorStudent@index POST /coordinator/students/{x}/toggle-block → CoordinatorStudent@toggleBlock
/coordinator/proctoring src/app/coordinator/proctoring/page.tsx	coord	GET /coordinator/proctoring → CoordinatorProctoring@index GET /coordinator/proctoring/{x} → CoordinatorProctoring@show POST /coordinator/proctoring/{x}/reinstate → CoordinatorProctoring@reinstate
/coordinator/reports src/app/coordinator/reports/page.tsx	coord	GET /coordinator/students → CoordinatorStudent@index

Marketing — 2 pages

Page	Guard	API calls
/marketing src/app/marketing/page.tsx	mkt, sa	GET /marketing/leads → MarketingLead@index POST /marketing/leads/notify → MarketingLead@notify GET /marketing/leads/{x} → MarketingLead@show POST /marketing/leads/{x}/notes → MarketingLead@storeNote POST /marketing/leads/{x}/status → MarketingLead@updateStatus
/marketing/inquiries src/app/marketing/inquiries/page.tsx	mkt, sa	GET /marketing/contact-requests → MarketingContactRequest@index GET /marketing/contact-requests/{x} → MarketingContactRequest@show POST /marketing/contact-requests/{x}/notes → MarketingContactRequest@storeNote POST /marketing/contact-requests/{x}/status → MarketingContactRequest@updateStatus

Superadmin — 1 pages

Page	Guard	API calls
/superadmin src/app/superadmin/page.tsx	sa	GET /admin/colleges → Admin@colleges POST /admin/colleges → Admin@storeCollege GET /admin/colleges/{x}/coordinators → Admin@collegeCoordinators POST /admin/colleges/{x}/coordinators → Admin@storeCoordinatorForCollege PUT /admin/colleges/{x}/seats → Admin@adjustCollegeSeats POST /admin/colleges/{x}/subscription → Admin@assignCollegeSubscription PUT /admin/coordinators/{x} → Admin@updateCoordinatorSection POST /admin/coordinators/{x}/toggle-block → Admin@toggleCoordinatorBlock POST /admin/tpos/{x}/toggle-block → Admin@toggleTpoBlock GET /marketing/leads → MarketingLead@index GET /marketing/leads/{x} → MarketingLead@show POST /marketing/leads/{x}/notes → MarketingLead@storeNote POST /marketing/leads/{x}/status → MarketingLead@updateStatus GET /superadmin/audit-logs → SuperAdmin@auditLogs GET /superadmin/feature-flags → SuperAdmin@featureFlags POST /superadmin/feature-flags/{x}/toggle → SuperAdmin@toggleFeatureFlag GET /superadmin/judge-nodes → SuperAdmin@judgeNodes GET /superadmin/marketing-performance → SuperAdmin@marketingPerformance GET /superadmin/overview → SuperAdmin@overview GET /superadmin/users → SuperAdmin@users POST /superadmin/users → SuperAdmin@storeUser GET /superadmin/users/{x}/detail → SuperAdmin@showUser POST /superadmin/users/{x}/permissions → SuperAdmin@updatePermissions POST /superadmin/users/{x}/toggle-block → SuperAdmin@toggleBlock POST {x}/import → (CompanyCandidate | CompanyContest | StudentImport)@storeForCollege GET {x}/imports → (CompanyCandidate | StudentImport)@indexForCollege

APPENDIX C

Database column reference

All 85 tables, taken from a real run of the 125 migrations. `col type` per column; `→ table` marks a foreign key; PK primary key; ! NOT NULL. Types are normalised by SQLite (int, str, bool, dt, text, dec, date): the **exact MySQL type, length, ENUM values and decimal precision are in the migration**.

Framework & infrastructure

cache 3 columns · model —

key str PK · value text! · expiration int!
idx (expiration)

cache_locks 3 columns · model —

key str PK · owner str! · expiration int!
idx (expiration)

failed_jobs 7 columns · model —

id int PK · uuid str! · connection text! · queue text! · payload text! · exception text! · failed_at dt!
UNIQUE (uuid)

job_batches 10 columns · model —

id str PK · name str! · total_jobs int! · pending_jobs int! · failed_jobs int! · failed_job_ids text! · options text · cancelled_at int · created_at int! · finished_at int
UNIQUE (id)

jobs 7 columns · model —

id int PK · queue str! · payload text! · attempts int! · reserved_at int · available_at int! · created_at int!
idx (queue)

migrations 3 columns · model —

id int PK · migration str! · batch int!

password_reset_tokens 3 columns · model —

email str PK · token str! · created_at dt
UNIQUE (email)

personal_access_tokens 10 columns · model —

id int PK · tokenable_type str! · tokenable_id int! · name text! · token str! · abilities text · last_used_at dt · expires_at dt · created_at dt · updated_at dt
idx (expires_at)

sessions 6 columns · model —

id str PK · user_id int · ip_address str · user_agent text · payload text! · last_activity int!
idx (last_activity)

Identity, tenancy & audit

users 43 columns · model User

id int PK · name str! · email str! · email_verified_at dt · password str! · remember_token str · created_at dt · updated_at dt · role str! · handle str · college_id int→colleges · is_blocked bool! · blocked_at dt · blocked_reason str · blocked_by int→users · roll_number str · branch str · cgpa dec · backlogs int · phone str · credentials_email_sent_at dt · parent_phone str · current_rating int! · rated_contests_count int! · lead_status str! · assigned_marketing_id int→users · converted_at dt · assigned_internal_id int→users · permissions text · section str · must_change_password bool! · avatar_path str · company_id int→companies · invited_by_company_id int→companies · phone_verified_at dt · bio text · linkedin_url str · github_url str · skills text · resume_path str · resume_original_name str · resume_uploaded_at dt · profile_completion_percent int!
idx (role)

colleges 12 columns · model College

id int PK · name str! · short_code str! · city str · state str · tier str! · placement_rate dec! · is_active bool! · created_at dt · updated_at dt · placement_target_percent dec · placement_target_deadline date
UNIQUE (short_code)

companies 13 columns · model Company

id int PK · name str! · slug str! · logo str · website_url str · industry str · overview text · hiring_process text · is_active bool! · created_at dt · updated_at dt · created_by int→users · account_type str!
idx (account_type)

activity_logs 10 columns · model ActivityLog

id int PK · actor_id int→users · actor_name str! · actor_role str! · action str! · target_type str · target_label str · meta text · created_at dt · updated_at dt
idx (created_at)

feature_flags 9 columns · model FeatureFlag

id int PK · key str! · name str! · description text · category str! · enabled bool! · updated_by int→users · created_at dt · updated_at dt
UNIQUE (key)

password_reset_otps 9 columns · model PasswordResetOtp

id int PK · user_id int!→users · code_hash str! · reset_token_hash str · attempts int! · expires_at dt! · consumed_at dt · created_at dt · updated_at dt
idx (user_id,consumed_at)

student_imports 14 columns · model StudentImport

id int PK · college_id int!→colleges · uploaded_by int→users · original_filename str! · status str! · total_rows int! · processed_rows int! · successful_rows int! · failed_rows int! · errors text · email_batch_id str · completed_at dt · created_at dt · updated_at dt

candidate_imports 14 columns · model CandidateImport

id int PK · placement_drive_id int!→placement_drives · uploaded_by int→users · original_filename str! · status str! · total_rows int! · processed_rows int! · successful_rows int! · failed_rows int! · errors text · email_batch_id str · completed_at dt · created_at dt · updated_at dt

Placement drives

placement_drives 19 columns · model PlacementDrive

id int PK · company_id int!→companies · title str! · role_title str! · ctc_range str · drive_date dt! · duration_minutes int · min_cgpa dec · max_backlogs int · eligible_branches text · status str! · created_at dt · updated_at dt · source str! · owning_college_id int→colleges · created_by int→users · is_open_to_all bool! · interview_date dt · terms_and_conditions text
idx (status,drive_date)

drive_college_mappings 16 columns · model DriveCollegeMapping

id int PK · placement_drive_id int!→placement_drives · college_id int!→colleges · min_cgpa_override dec · max_backlogs_override int · eligible_branches_override text · mapped_by int→users · mapped_at dt · unmapped_at dt · is_active bool! · created_at dt · updated_at dt · approved_by int→users · approved_at dt · declined_at dt · status str!
UNIQUE (placement_drive_id,college_id)

drive_applications 11 columns · model DriveApplication

id int PK · placement_drive_id int!→placement_drives · user_id int!→users · college_id int!→colleges · stage str! · stage_updated_at dt! · stage_updated_by int→users · ctc_offered dec · notes text · created_at dt · updated_at dt
idx (placement_drive_id,stage)

company_prep_questions 10 columns · model CompanyPrepQuestion

id int PK · company_id int!→companies · asked_year int! · category str! · round_name str · question text! · answer_notes text · display_order int! · created_at dt · updated_at dt
idx (company_id,asked_year)

company_recommended_problems 7 columns · model CompanyRecommendedProblem

id int PK · company_id int!→companies · problem_slug str! · topic_tag str · priority int! · created_at dt · updated_at dt
UNIQUE (company_id,problem_slug)

company_recommended_interview_questions 5 columns · model CompanyRecommendedInterviewQuestion

id int PK · company_id int!→companies · interview_question_bank_id int!→interview_question_banks · created_at dt · updated_at dt
UNIQUE (company_id,interview_question_bank_id)

Practice & judge

problems 19 columns · model Problem

id int PK · slug str! · title str! · difficulty str! · tags text · description text! · constraints text · hints text · acceptance_rate dec · total_submissions int! · display_order int! · created_at dt · updated_at dt · function_name str · params text · return_type str · comparison_mode str! · comparison_epsilon dec · companies text
UNIQUE (slug)

problem_test_cases 9 columns · model ProblemTestCase

id int PK · problem_id int!→problems · inputs text! · expected_output text! · explanation text · is_sample bool! · display_order int! · created_at dt · updated_at dt
idx (problem_id,is_sample)

submissions 11 columns · model Submission

id int PK · user_id int!→users · problem_id int!→problems · language str! · status str! · submitted_on date! · created_at dt · updated_at dt · runtime_ms int · memory_kb int · code text
idx (user_id,submitted_on)

Contests & contest proctoring

contests 21 columns · model Contest

id int PK · title str! · slug str! · description text · start_at dt! · end_at dt! · is_rated bool! · status str! · finalized_at dt · created_by int→users · created_at dt · updated_at dt · contest_type str! · daily_key str · is_auto_generated bool! · company_id int→companies · placement_drive_id int→placement_drives · owning_college_id int→colleges · owning_company_id int→companies · audience_scope str · qualifying_score_percent dec
idx (owning_company_id)

contest_problems 7 columns · model ContestProblem

id int PK · contest_id int!→contests · problem_id int!→problems · points int! · display_order int! · created_at dt · updated_at dt
UNIQUE (contest_id,problem_id)

contest_participants 11 columns · model ContestParticipant

id int PK · contest_id int!→contests · user_id int!→users · registered_at dt! · score int! · penalty_minutes int! · rank int · rating_before int · rating_after int · created_at dt · updated_at dt
idx (contest_id,score,penalty_minutes)

contest_submissions 12 columns · model ContestSubmission

id int PK · contest_id int!→contests · contest_problem_id int!→contest_problems · user_id int!→users · language str! · status str! · submitted_at dt! · points_awarded int · created_at dt · updated_at dt · judge_token str · code text
UNIQUE (judge_token)

contest_colleges 5 columns · model —

id int PK · contest_id int!→contests · college_id int!→colleges · created_at dt · updated_at dt
UNIQUE (contest_id,college_id)

proctoring_sessions 11 columns · model ProctoringSession

id int PK · contest_participant_id int!→contest_participants · status str! · consented_at dt · violation_count int! · locked_at dt · completed_at dt · reported_at dt · device_info text · created_at dt · updated_at dt
idx (status)

proctoring_violations 10 columns · model ProctoringViolation

id int PK · proctoring_session_id int!→proctoring_sessions · contest_problem_id int→contest_problems · type str! · counted_toward_lock bool! · ip_address str · meta text · occurred_at dt! · created_at dt · updated_at dt
idx (proctoring_session_id,occurred_at)

AI Interviews & tracks

interviews 20 columns · model Interview

id int PK · title str! · slug str! · description text · status str! · interview_type str! · company_id int→companies · placement_drive_id int→placement_drives · owning_college_id int→colleges · owning_company_id int→companies · created_by int→users · created_at dt · updated_at dt · is_mock bool! · audience_scope str · interview_track_id int→interview_tracks · round_number int · round_name str · category_weights text · qualifying_score_percent dec
idx (interview_track_id,round_number)

interview_colleges 5 columns · model —

id int PK · interview_id int!→interviews · college_id int!→colleges · created_at dt · updated_at dt
UNIQUE (interview_id,college_id)

interview_question_banks 11 columns · model InterviewQuestionBank

id int PK · question_text text! · category str! · difficulty str! · expected_duration_seconds int! · tags text · notes_for_reviewer text · is_active bool! · created_by int→users · created_at dt · updated_at dt
idx (difficulty)

interview_questions 6 columns · model InterviewQuestion

id int PK · interview_id int!→interviews · interview_question_bank_id int!→interview_question_banks · display_order int! · created_at dt · updated_at dt
UNIQUE (interview_id,interview_question_bank_id)

interview_sessions 17 columns · model InterviewSession

id int PK · interview_id int!→interviews · user_id int!→users · status str! · invited_at dt · invited_by int→users · started_at dt · completed_at dt · current_question_order int! · created_at dt · updated_at dt · composite_score_percent dec · reviewed_at dt · reviewed_by int→users · advanced bool! · ai_scored_at dt · scoring_failed_at dt
UNIQUE (interview_id,user_id)

interview_responses 14 columns · model InterviewResponse

id int PK · interview_session_id int!→interview_sessions · interview_question_id int!→interview_questions · transcript_text text · audio_path str · audio_duration_seconds int · answered_at dt · created_at dt · updated_at dt · score int · review_notes text · scored_by int→users · scored_at dt · ai_scored bool!
UNIQUE (interview_session_id,interview_question_id)

interview_role_templates 11 columns · model InterviewRoleTemplate

id int PK · name str! · description text · tech_stack_tags text · rounds_config text! · owning_college_id int→colleges · owning_company_id int→companies · is_active bool! · created_by int→users · created_at dt · updated_at dt
idx (owning_company_id)

interview_tracks 15 columns · model InterviewTrack

id int PK · title str! · slug str! · description text · status str! · track_type str! · role_title str · interview_role_template_id int→interview_role_templates · company_id int→companies · placement_drive_id int→placement_drives · owning_college_id int→colleges · owning_company_id int→companies · created_by int→users · created_at dt · updated_at dt
UNIQUE (slug)

interview_track_colleges 5 columns · model —

id int PK · interview_track_id int!→interview_tracks · college_id int!→colleges · created_at dt · updated_at dt
UNIQUE (interview_track_id,college_id)

interview_proctoring_sessions 10 columns · model InterviewProctoringSession

id int PK · interview_session_id int!→interview_sessions · status str! · consented_at dt · violation_count int! · locked_at dt · completed_at dt · device_info text · created_at dt · updated_at dt
idx (status)

interview_proctoring_violations 10 columns · model InterviewProctoringViolation

id int PK · interview_proctoring_session_id int!→interview_proctoring_sessions · interview_question_id int→interview_questions · type str! · counted_toward_lock bool! · ip_address str · meta text · occurred_at dt! · created_at dt · updated_at dt
idx (interview_proctoring_session_id,occurred_at)

Soft Skills

soft_skill_questions 11 columns · model SoftSkillQuestion

id int PK · category str! · difficulty str! · question_text text! · options text! · correct_index int! · explanation text · is_active bool! · created_by int→users · created_at dt · updated_at dt
idx (difficulty)

soft_skill_assessments 14 columns · model SoftSkillAssessment

id int PK · title str! · slug str! · description text · status str! · assessment_type str! · owning_college_id int→colleges · owning_company_id int→companies · duration_minutes int! · pass_percentage int! · max_attempts int · created_by int→users · created_at dt · updated_at dt
UNIQUE (slug)

soft_skill_assessment_questions 6 columns · model SoftSkillAssessmentQuestion

id int PK · soft_skill_assessment_id int!→soft_skill_assessments · soft_skill_question_id int!→soft_skill_questions · display_order int! · created_at dt · updated_at dt
UNIQUE (soft_skill_assessment_id,soft_skill_question_id)

soft_skill_sessions 12 columns · model SoftSkillSession

id int PK · soft_skill_assessment_id int!→soft_skill_assessments · user_id int!→users · status str! · attempt_number int! · started_at dt! · completed_at dt · score_percent dec · passed bool! · category_breakdown text · created_at dt · updated_at dt
idx (user_id,status)

soft_skill_responses 8 columns · model SoftSkillResponse

id int PK · soft_skill_session_id int!→soft_skill_sessions · soft_skill_assessment_question_id int!→soft_skill_assessment_questions · selected_index int · is_correct bool · answered_at dt · created_at dt · updated_at dt
UNIQUE (soft_skill_session_id,soft_skill_assessment_question_id)

soft_skill_proctoring_sessions 10 columns · model SoftSkillProctoringSession

id int PK · soft_skill_session_id int!→soft_skill_sessions · status str! · consented_at dt · violation_count int! · locked_at dt · completed_at dt · device_info text · created_at dt · updated_at dt
idx (status)

soft_skill_proctoring_violations 9 columns · model SoftSkillProctoringViolation

id int PK · soft_skill_proctoring_session_id int!→soft_skill_proctoring_sessions · type str! · counted_toward_lock bool! · ip_address str · meta text · occurred_at dt! · created_at dt · updated_at dt
idx (soft_skill_proctoring_session_id,occurred_at)

Talent Pool

talent_pool_candidates 11 columns · model TalentPoolCandidate

id int PK · user_id int!→users · source_contest_id int→contests · score_percent dec! · qualified_at dt! · visibility_status str! · consent_given_at dt · hired_by_company_id int→companies · hired_at dt · created_at dt · updated_at dt
UNIQUE (user_id)

talent_pool_inquiries 14 columns · model TalentPoolInquiry

id int PK · talent_pool_candidate_id int!→talent_pool_candidates · company_id int!→companies · initiated_by int!→users · status str! · interview_scheduled_at dt · interview_mode str · interview_location str · interview_notes text · responded_at dt · ctc_offered dec · hired_at dt · created_at dt · updated_at dt
idx (status)

talent_pool_messages 5 columns · model TalentPoolMessage

id int PK · talent_pool_inquiry_id int!→talent_pool_inquiries · sender_id int!→users · message text! · created_at dt!

Learning Centre

articles 15 columns · model Article

id int PK · article_topic_id int!→article_topics · title str! · slug str! · excerpt text! · content text! · icon str · status str! · display_order int! · reading_time_minutes int! · created_by int→users · published_at dt · view_count int! · created_at dt · updated_at dt
UNIQUE (slug)

article_topics 10 columns · model ArticleTopic

id int PK · name str! · slug str! · description text · icon str · color str · display_order int! · created_by int→users · created_at dt · updated_at dt
UNIQUE (slug)

article_reads 4 columns · model ArticleRead

id int PK · article_id int!→articles · user_id int!→users · read_at dt!
UNIQUE (article_id,user_id)

speaking_prompts 14 columns · model SpeakingPrompt

id int PK · title str! · passage_text text! · category str! · difficulty str! · target_wpm_min int! · target_wpm_max int! · is_active bool! · display_order int! · created_at dt · updated_at dt · user_id int→users · source str! · interest str
idx (user_id,created_at)

speaking_attempts 23 columns · model SpeakingAttempt

id int PK · user_id int!→users · speaking_prompt_id int!→speaking_prompts · attempt_number int! · transcript_text text · audio_path str · duration_seconds int · pacing_wpm int · overall_score int · clarity_score int · fluency_score int · accuracy_score int · feedback text · improvement_tips text · passed bool! · scored_at dt · scoring_failed_at dt · created_at dt · updated_at dt · heard_transcript text · word_feedback text · filler_count int · scored_with_audio bool!
idx (user_id,created_at)

listening_lessons 16 columns · model ListeningLesson

id int PK · title str! · passage_text text! · category str! · difficulty str! · questions text! · is_active bool! · display_order int! · created_at dt · updated_at dt · user_id int→users · format str! · source str! · interest str · speakers text · script text
idx (user_id,created_at)

listening_attempts 12 columns · model ListeningAttempt

id int PK · user_id int!→users · listening_lesson_id int!→listening_lessons · attempt_number int! · answers text! · score int! · passed bool! · created_at dt · updated_at dt · mode str! · plays_used int · skill_breakdown text
idx (user_id,created_at)

vocabulary_words 17 columns · model VocabularyWord

id int PK · user_id int→users · source str! · deck str · word str! · part_of_speech str! · level str! · meaning str! · example str! · synonyms text · distractors text · pair_word str · note str · sort_order int! · is_active bool! · created_at dt · updated_at dt
UNIQUE (user_id,source,word)

vocabulary_word_progress 13 columns · model VocabularyWordProgress

id int PK · user_id int!→users · vocabulary_word_id int!→vocabulary_words · box int! · streak int! · seen_count int! · correct_count int! · lapse_count int! · last_seen_at dt · due_on date · mastered_at dt · created_at dt · updated_at dt
idx (user_id,due_on)

vocabulary_attempts 14 columns · model VocabularyAttempt

id int PK · user_id int!→users · topic str! · difficulty str! · questions text! · answers text · score int · passed bool! · submitted_at dt · created_at dt · updated_at dt · kind str! · deck str · meta text
idx (user_id,kind)

vocabulary_answers 12 columns · model VocabularyAnswer

id int PK · user_id int!→users · vocabulary_attempt_id int!→vocabulary_attempts · question_index int! · vocabulary_word_id int→vocabulary_words · type str! · response str · is_correct bool! · box_before int · box_after int · response_ms int · created_at dt
idx (user_id,created_at)

Project Lab

project_topics 20 columns · model ProjectTopic

id int PK · slug str! · title str! · category str! · summary text! · difficulty str! · estimated_hours int! · tech_stack text · outcomes text · sow_markdown text · sow_version int! · status str! · is_free_sample bool! · icon str · display_order int! · created_by int→users · published_at dt · created_at dt · updated_at dt · authorship_mode str!
UNIQUE (slug)

project_milestones 9 columns · model ProjectMilestone

id int PK · project_topic_id int!→project_topics · position int! · title str! · description text · acceptance_criteria text · estimated_hours int! · created_at dt · updated_at dt
idx (project_topic_id,position)

project_enrollments 9 columns · model ProjectEnrollment

id int PK · user_id int!→users · project_topic_id int!→project_topics · sow_version int! · status str! · started_at dt! · submitted_at dt · created_at dt · updated_at dt
UNIQUE (user_id,project_topic_id)

project_milestone_progress 6 columns · model ProjectMilestoneProgress

id int PK · project_enrollment_id int!→project_enrollments · project_milestone_id int!→project_milestones · completed_at dt! · created_at dt · updated_at dt
UNIQUE (project_enrollment_id,project_milestone_id)

project_workspaces 19 columns · model ProjectWorkspace

id int PK · public_id str! · project_enrollment_id int!→project_enrollments · user_id int!→users · host str! · container_name str! · volume_name str! · status str! · connection_token text · last_heartbeat_at dt · last_started_at dt · last_stopped_at dt · stop_reason str · error text · disk_mb int · created_at dt · updated_at dt · guard_last_seen_at dt · guard_version str
UNIQUE (project_enrollment_id)

project_workspace_sessions 10 columns · model ProjectWorkspaceSession

id int PK · project_workspace_id int!→project_workspaces · user_id int!→users · started_at dt! · ended_at dt · seconds int! · stop_reason str · created_at dt · updated_at dt · guard_active_seconds int!
idx (user_id,started_at)

project_workspace_integrity_events 11 columns · model ProjectWorkspaceIntegrityEvent

id int PK · project_workspace_id int!→project_workspaces · user_id int!→users · client_event_id str! · kind str! · size int · target str · guard_version str · meta text · occurred_at dt! · created_at dt!
idx (occurred_at)

project_submissions 23 columns · model ProjectSubmission

id int PK · project_enrollment_id int!→project_enrollments · user_id int!→users · attempt int! · snapshot_path str · snapshot_size_bytes int! · snapshot_sha256 str · repo_url str · live_url str · notes text · milestones_done int! · milestones_total int! · warnings text · status str! · score int · rubric text · feedback text · reviewed_by int→users · reviewed_at dt · submitted_at dt! · created_at dt · updated_at dt · integrity text
UNIQUE (project_enrollment_id,attempt)

Hack Arena

hackathons 39 columns · model Hackathon

id int PK · slug str! · title str! · tagline str! · description text! · cover_image str · accent str! · themes text · mode str! · location str · host_type str! · company_id int→companies · created_by int→users · status str! · registration_opens_at dt! · registration_closes_at dt! · starts_at dt! · ends_at dt! · results_expected_at dt · results_published_at dt · published_at dt · cancelled_at dt · cancelled_reason str · min_team_size int! · max_team_size int! · max_participants int · prize_pool_amount int! · currency str! · prizes text · tracks text · criteria text · rules text · eligibility text · faqs text · resources text · is_featured bool! · created_at dt · updated_at dt · audience str!
idx (status,audience)

hackathon_colleges 5 columns · model —

id int PK · hackathon_id int!→hackathons · college_id int!→colleges · created_at dt · updated_at dt
idx (college_id)

hackathon_teams 7 columns · model HackathonTeam

id int PK · hackathon_id int!→hackathons · name str! · invite_code str! · leader_id int!→users · created_at dt · updated_at dt
UNIQUE (hackathon_id,name)

hackathon_participants 8 columns · model HackathonParticipant

id int PK · hackathon_id int!→hackathons · user_id int!→users · hackathon_team_id int!→hackathon_teams · role str! · registered_at dt! · created_at dt · updated_at dt
idx (user_id)

hackathon_submissions 25 columns · model HackathonSubmission

id int PK · hackathon_id int!→hackathons · hackathon_team_id int!→hackathon_teams · submitted_by int→users · title str! · tagline str · description text · track str · repo_url str · demo_url str · video_url str · tech_stack text · status str! · submitted_at dt · scores text · final_score dec · rank int · award_label str · feedback text · scored_by int→users · scored_at dt · is_disqualified bool! · disqualified_reason str · created_at dt · updated_at dt
idx (hackathon_id,rank)

hackathon_announcements 7 columns · model HackathonAnnouncement

id int PK · hackathon_id int!→hackathons · author_id int→users · title str! · body text! · created_at dt · updated_at dt
idx (hackathon_id,created_at)

Billing

plans 20 columns · model Plan

id int PK · code str! · name str! · audience str! · monthly_price int · annual_price int · duration_days int · is_active bool! · sort_order int! · created_at dt · updated_at dt · max_practice_problems_per_day int · max_mock_interviews_per_day int · drive_access bool! · max_students int · max_learning_centre_ai_attempts_per_day int · kind str! · project_lab_access bool! · max_workspace_hours_per_month int · max_active_projects int
UNIQUE (code)

subscriptions 13 columns · model Subscription

id int PK · subscriber_type str! · subscriber_id int! · plan_id int!→plans · status str! · started_at dt! · current_period_end dt · auto_renew bool! · canceled_at dt · meta text · created_at dt · updated_at dt · is_trial bool!
idx (subscriber_type,subscriber_id,status)

Marketing & leads

contact_requests 12 columns · model ContactRequest

id int PK · name str! · email str! · phone str · audience str! · organization_name str! · message text · status str! · converted_at dt · assigned_marketing_id int→users · created_at dt · updated_at dt
idx (status)

contact_request_notes 6 columns · model ContactRequestNote

id int PK · contact_request_id int!→contact_requests · author_id int→users · note text! · created_at dt · updated_at dt
idx (contact_request_id)

lead_notes 6 columns · model LeadNote

id int PK · user_id int!→users · author_id int→users · note text! · created_at dt · updated_at dt
idx (user_id)

newsletter_subscribers 6 columns · model NewsletterSubscriber

id int PK · email str! · subscribed_at dt! · unsubscribed_at dt · created_at dt · updated_at dt
UNIQUE (email)

APPENDIX D

Models and relationships

All 73 Eloquent models in `app/Models/`: the table they map to and their declared relations (`belongsTo`, `hasMany`, `hasOne`, `belongsToMany`, `morph*`).

Model	Table	Relations
ActivityLog	activity_logs	belongsTo(User)
Article	articles	belongsTo(ArticleTopic), belongsTo(User), hasMany(ArticleRead)
ArticleRead	article_reads	belongsTo(Article), belongsTo(User)
ArticleTopic	article_topics	hasMany(Article), belongsTo(User)
CandidateImport	candidate_imports	belongsTo(PlacementDrive), belongsTo(User)
College	colleges	hasMany(User), hasMany(User), hasMany(DriveApplication)
Company	companies	hasMany(User), hasMany(User), hasMany(PlacementDrive), hasMany(CompanyPrepQuestion), hasMany(CompanyRecommendedProblem), belongsTo(User), hasMany(TalentPoolInquiry)
CompanyPrepQuestion	company_prep_questions	belongsTo(Company)
CompanyRecommendedInterviewQuestion	company_recommended_interview_questions	belongsTo(Company), belongsTo(InterviewQuestionBank)
CompanyRecommendedProblem	company_recommended_problems	belongsTo(Company)
ContactRequest	contact_requests	hasMany(ContactRequestNote), belongsTo(User)
ContactRequestNote	contact_request_notes	belongsTo(ContactRequest), belongsTo(User)
Contest	contests	hasMany(ContestProblem), hasMany(ContestParticipant), hasMany(ContestSubmission), belongsTo(User), belongsTo(Company), belongsTo(PlacementDrive), belongsTo(College), belongsTo(Company), belongsToMany(College)
ContestParticipant	contest_participants	belongsTo(Contest), belongsTo(User), hasOne(ProctoringSession)
ContestProblem	contest_problems	belongsTo(Contest), belongsTo(Problem), hasMany(ContestSubmission)
ContestSubmission	contest_submissions	belongsTo(Contest), belongsTo(ContestProblem), belongsTo(User)
DriveApplication	drive_applications	belongsTo(PlacementDrive), belongsTo(User), belongsTo(College), belongsTo(User)
DriveCollegeMapping	drive_college_mappings	belongsTo(PlacementDrive), belongsTo(College), belongsTo(User), belongsTo(User)
FeatureFlag	feature_flags	belongsTo(User)
Hackathon	hackathons	belongsTo(Company), belongsTo(User), belongsToMany(College), hasMany(HackathonTeam), hasMany(HackathonParticipant), hasMany(HackathonSubmission), hasMany(HackathonAnnouncement)
HackathonAnnouncement	hackathon_announcements	belongsTo(Hackathon), belongsTo(User)
HackathonParticipant	hackathon_participants	belongsTo(Hackathon), belongsTo(User), belongsTo(HackathonTeam)
HackathonSubmission	hackathon_submissions	belongsTo(Hackathon), belongsTo(HackathonTeam), belongsTo(User), belongsTo(User)
HackathonTeam	hackathon_teams	belongsTo(Hackathon), belongsTo(User), hasMany(HackathonParticipant), hasOne(HackathonSubmission)
HasSubscription	–	morphMany(Subscription)
Interview	interviews	hasMany(InterviewQuestion), hasMany(InterviewSession), belongsTo(InterviewTrack), belongsTo(User), belongsTo(Company), belongsTo(PlacementDrive), belongsTo(College), belongsTo(Company), belongsToMany(College)
InterviewProctoringSession	interview_proctoring_sessions	belongsTo(InterviewSession), hasMany(InterviewProctoringViolation)
InterviewProctoringViolation	interview_proctoring_violations	belongsTo(InterviewProctoringSession), belongsTo(InterviewQuestion)
InterviewQuestion	interview_questions	belongsTo(Interview), belongsTo(InterviewQuestionBank), hasMany(InterviewResponse)
InterviewQuestionBank	interview_question_banks	belongsTo(User), hasMany(InterviewQuestion), belongsToMany(Company)
InterviewResponse	interview_responses	belongsTo(InterviewSession), belongsTo(InterviewQuestion), belongsTo(User)

Model	Table	Relations
InterviewRoleTemplate	interview_role_templates	belongsTo(College), belongsTo(Company), belongsTo(User), hasMany(InterviewTrack)
InterviewSession	interview_sessions	belongsTo(Interview), belongsTo(User), belongsTo(User), belongsTo(User), hasMany(InterviewResponse), hasOne(InterviewProctoringSession)
InterviewTrack	interview_tracks	hasMany(Interview), belongsTo(InterviewRoleTemplate), belongsTo(User), belongsTo(Company), belongsTo(PlacementDrive), belongsTo(College), belongsTo(Company), belongsToMany(College)
LeadNote	lead_notes	belongsTo(User), belongsTo(User)
ListeningAttempt	listening_attempts	belongsTo(User), belongsTo(ListeningLesson)
ListeningLesson	listening_lessons	belongsTo(User), hasMany(ListeningAttempt)
NewsletterSubscriber	newsletter_subscribers	—
PasswordResetOtp	password_reset_otps	belongsTo(User)
PlacementDrive	placement_drives	belongsTo(Company), hasMany(DriveCollegeMapping), hasMany(DriveApplication), hasMany(Interview), belongsTo(College), belongsTo(User)
Plan	plans	hasMany(Subscription)
Problem	problems	hasMany(ProblemTestCase), hasMany(Submission)
ProblemTestCase	problem_test_cases	belongsTo(Problem)
ProctoringSession	proctoring_sessions	belongsTo(ContestParticipant), hasMany(ProctoringViolation)
ProctoringViolation	proctoring_violations	belongsTo(ProctoringSession), belongsTo(ContestProblem)
ProjectEnrollment	project_enrollments	belongsTo(User), belongsTo(ProjectTopic), hasMany(ProjectMilestoneProgress), hasOne(ProjectWorkspace), hasMany(ProjectSubmission)
ProjectMilestone	project_milestones	belongsTo(ProjectTopic)
ProjectMilestoneProgress	project_milestone_progress	belongsTo(ProjectEnrollment), belongsTo(ProjectMilestone)
ProjectSubmission	project_submissions	belongsTo(ProjectEnrollment), belongsTo(User), belongsTo(User)
ProjectTopic	project_topics	hasMany(ProjectMilestone), hasMany(ProjectEnrollment), belongsTo(User)
ProjectWorkspace	project_workspaces	belongsTo(ProjectEnrollment), belongsTo(User), hasMany(ProjectWorkspaceSession), hasMany(ProjectWorkspaceIntegrityEvent)
ProjectWorkspaceIntegrityEvent	project_workspace_integrity_events	belongsTo(ProjectWorkspace)
ProjectWorkspaceSession	project_workspace_sessions	belongsTo(ProjectWorkspace)
SoftSkillAssessment	soft_skill_assessments	hasMany(SoftSkillAssessmentQuestion), hasMany(SoftSkillSession), belongsTo(College), belongsTo(Company), belongsTo(User)
SoftSkillAssessmentQuestion	soft_skill_assessment_questions	belongsTo(SoftSkillAssessment), belongsTo(SoftSkillQuestion)
SoftSkillProctoringSession	soft_skill_proctoring_sessions	belongsTo(SoftSkillSession), hasMany(SoftSkillProctoringViolation)
SoftSkillProctoringViolation	soft_skill_proctoring_violations	belongsTo(SoftSkillProctoringSession)
SoftSkillQuestion	soft_skill_questions	belongsTo(User), hasMany(SoftSkillAssessmentQuestion)
SoftSkillResponse	soft_skill_responses	belongsTo(SoftSkillSession), belongsTo(SoftSkillAssessmentQuestion)
SoftSkillSession	soft_skill_sessions	belongsTo(SoftSkillAssessment), belongsTo(User), hasMany(SoftSkillResponse), hasOne(SoftSkillProctoringSession)
SpeakingAttempt	speaking_attempts	belongsTo(User), belongsTo(SpeakingPrompt)
SpeakingPrompt	speaking_prompts	belongsTo(User), hasMany(SpeakingAttempt)
StudentImport	student_imports	belongsTo(College), belongsTo(User)
Submission	submissions	belongsTo(User), belongsTo(Problem)
Subscription	subscriptions	morphTo(?), belongsTo(Plan)
TalentPoolCandidate	talent_pool_candidates	belongsTo(User), belongsTo(Contest), belongsTo(Company), hasMany(TalentPoolInquiry)
TalentPoolInquiry	talent_pool_inquiries	belongsTo(TalentPoolCandidate), belongsTo(Company), belongsTo(User), hasMany(TalentPoolMessage)
TalentPoolMessage	talent_pool_messages	belongsTo(TalentPoolInquiry), belongsTo(User)

Model	Table	Relations
User	users	belongsTo(College), belongsTo(Company), belongsTo(Company), hasMany(Submission), hasMany(ContestParticipant), hasMany(ArticleRead), hasOne(TalentPoolCandidate), hasMany(LeadNote), belongsTo(User), hasMany(User), hasMany(ContactRequest), belongsTo(User), hasMany(User), belongsTo(User)
VocabularyAnswer	vocabulary_answers	belongsTo(VocabularyAttempt), belongsTo(VocabularyWord)
VocabularyAttempt	vocabulary_attempts	belongsTo(User), hasMany(VocabularyAnswer)
VocabularyWord	vocabulary_words	belongsTo(User), hasMany(VocabularyWordProgress)
VocabularyWordProgress	vocabulary_word_progress	belongsTo(User), belongsTo(VocabularyWord)

APPENDIX E

Migration timeline

All 125 migrations in run order (file date prefix and name). Never edit one that has been applied; add a new one.

Date	Migration	Date	Migration
0001-01-01	create_users_table	2026-09-24	create_interview_sessions_table
0001-01-01	create_cache_table	2026-09-24	create_interview_responses_table
0001-01-01	create_jobs_table	2026-09-24	create_company_recommended_interview_questions_table
2026-09-11	create_personal_access_tokens_table	2026-09-24	create_interview_proctoring_sessions_table
2026-09-11	create_colleges_table	2026-09-24	create_interview_proctoring_violations_table
2026-09-11	add_role_and_profile_fields_to_users_table	2026-09-24	backfill_interviews_permission_for_full_access_staff
2026-09-12	create_companies_table	2026-09-24	create_article_topics_table
2026-09-12	create_placement_drives_table	2026-09-24	create_articles_table
2026-09-12	create_drive_college_mappings_table	2026-09-24	create_article_reads_table
2026-09-12	create_company_prep_questions_table	2026-09-24	backfill_articles_permission_for_full_access_staff
2026-09-12	create_company_recommended_problems_table	2026-09-24	add_interview_date_to_placement_drives_table
2026-09-12	add_provenance_to_companies_and_placement_drives	2026-09-24	add_is_mock_to_interviews_table
2026-09-12	create_activity_logs_table	2026-09-24	add_talent_pool_fields_to_contests_table
2026-09-13	add_academic_fields_to_users_table	2026-09-24	add_audience_scope_to_interviews_table
2026-09-13	create_student_imports_table	2026-09-24	create_talent_pool_candidates_table
2026-09-13	add_missing_performance_indexes	2026-09-24	create_talent_pool_inquiries_table
2026-09-13	create_problems_table	2026-09-24	create_talent_pool_messages_table
2026-09-14	add_test_harness_to_problems_table	2026-09-24	backfill_talent_pool_permission_for_full_access_staff
2026-09-15	add_parent_phone_to_users_table	2026-09-24	add_phone_verified_at_to_users_table
2026-09-15	create_submissions_table	2026-09-25	create_interview_role_templates_table
2026-09-16	create_plans_table	2026-09-25	create_interview_tracks_table
2026-09-16	create_subscriptions_table	2026-09-25	create_interview_track_colleges_table
2026-09-16	create_contests_table	2026-09-25	add_track_fields_to_interviews_table
2026-09-16	create_contest_problems_table	2026-09-25	add_scoring_fields_to_interview_responses_table
2026-09-16	create_contest_participants_table	2026-09-25	add_review_fields_to_interview_sessions_table
2026-09-16	create_contest_submissions_table	2026-09-26	add_extended_profile_fields_to_users_table
2026-09-16	add_rating_fields_to_users_table	2026-09-27	add_ai_scoring_fields_to_interview_responses_table
2026-09-16	create_drive_applications_table	2026-09-27	add_ai_scoring_fields_to_interview_sessions_table
2026-09-16	add_placement_target_to_colleges_table	2026-09-27	add_entitlements_to_plans_table
2026-09-16	add_runtime_stats_to_submissions_table	2026-09-28	add_max_students_to_plans_table
2026-09-16	create_password_reset_otps_table	2026-09-29	create_contact_requests_table
2026-09-16	add_lead_tracking_to_users_and_create_lead_notes	2026-09-29	create_contact_request_notes_table
2026-09-16	add_admin_marketing_to_users_role_enum	2026-09-29	add_custom_tier_to_colleges_table
2026-09-17	create_judge_nodes_table	2026-09-29	create_newsletter_subscribers_table
2026-09-17	create_feature_flags_table	2026-09-29	add_code_to_submission_tables
2026-09-17	add_assigned_marketing_id_to_users_table	2026-09-30	add_terms_and_conditions_to_placement_drives_table
2026-09-17	restructure_problems_for_generated_harness	2026-09-30	create_speaking_prompts_table
2026-09-17	create_problem_test_cases_table	2026-09-30	create_speaking_attempts_table
2026-09-18	add_conversion_tracking_to_users_table	2026-09-30	create_listening_lessons_table
2026-09-19	add_permissions_to_users_table	2026-09-30	create_listening_attempts_table
2026-09-22	add_judge_token_to_contest_submissions_table	2026-09-30	create_vocabulary_attempts_table
2026-09-22	drop_fabricated_judge_nodes_table	2026-09-30	add_learning_centre_limit_to_plans_table

Date	Migration	Date	Migration
2026-09-22	add_section_to_users_table	2026-09-30	create_soft_skill_questions_table
2026-09-22	add_must_change_password_to_users_table	2026-09-30	create_soft_skill_assessments_table
2026-09-22	widen_users_role_enum_for_section_coordinator	2026-09-30	create_soft_skill_assessment_questions_table
2026-09-22	add_avatar_path_to_users_table	2026-09-30	create_soft_skill_sessions_table
2026-09-22	add_approval_workflow_to_drive_college_mappings_table	2026-09-30	create_soft_skill_responses_table
2026-09-22	add_type_and_scope_to_contests_table	2026-09-30	backfill_soft_skills_permission_for_full_access_staff
2026-09-22	create_contest_colleges_table	2026-10-01	add_companies_to_problems_table
2026-09-22	create_proctoring_sessions_table	2026-10-01	add_is_trial_to_subscriptions_table
2026-09-22	create_proctoring_violations_table	2026-10-07	upgrade_speaking_practice_audio_first_and_ai_passages
2026-09-23	widen_users_role_enum_for_admin_company	2026-10-07	create_soft_skill_proctoring_sessions_table
2026-09-23	add_company_fields_to_users_table	2026-10-07	create_soft_skill_proctoring_violations_table
2026-09-23	add_account_type_to_companies_table	2026-10-08	upgrade_listening_lab_formats_skills_and_ai_lessons
2026-09-23	add_owning_company_id_to_contests_table	2026-10-09	upgrade_vocabulary_sprint_word_bank_and_spaced_repetition
2026-09-23	widen_placement_drives_source_enum_for_company_direct	2026-10-10	add_project_lab_columns_to_plans_table
2026-09-23	create_candidate_imports_table	2026-10-10	create_project_lab_tables
2026-09-23	make_college_id_nullable_on_drive_applications_table	2026-10-10	backfill_project_lab_permission_for_full_access_staff
2026-09-23	add_is_open_to_all_to_placement_drives_table	2026-10-11	add_authorship_evidence_to_project_lab
2026-09-24	create_interviews_table	2026-10-12	create_hackathon_tables
2026-09-24	create_interview_colleges_table	2026-10-12	backfill_hackathons_permission_for_full_access_staff
2026-09-24	create_interview_question_banks_table	2026-10-13	add_audience_to_hackathons
2026-09-24	create_interview_questions_table		

APPENDIX F

Glossary

Term	Meaning
Academic Enterprise / Pro Campus / Standard	The three institution plans a college is sold; Custom is a negotiated seat count.
Add-on	A `plans.kind = addon` row bought alongside a base plan; today only Project Lab.
Audience scope	`all` / `college` / `direct` — who a talent-pool contest or interview is aimed at.
Authorship guard	Editor-side script that refuses pasted text not copied inside the workspace and reports metadata-only events.
basePath	A prop that tells a shared frontend console which API prefix (and so which controller family) to call.
Catalog drive	A placement drive Ops schedules once for all colleges (`source = catalog`).
Coverage	`User::subscriptionCoverage()`: where a student's plan comes from — own plan or their college's.
Drive mapping	Row linking a drive to a college; needs `approved` + `is_active` to count.
Entitlements	Plan-derived limits resolved per user by `effectiveEntitlements()`.
Final Interview track	A three-round chained interview pipeline built from a role template.
Harness	Generated driver code appended to a student's solution that runs every test case and prints JSON.
Heartbeat	Per-minute ping from an open workspace page; both idle detector and hours meter.
Hiring tenant	A `companies` row with `account_type = hiring_tenant` and its own `admin_company` login.
Mellow Direct	A student with no college — treated as a sales lead owned by Marketing.
Mellow Ops / Internal / Marketing	Staff roles `admin_internal` and `admin_marketing`; "Internal" also names the post-conversion customer owner.
Mirror controllers	Near-identical Admin / Tpo / Company controllers, one per role, each with its own scope.
Piston	Open-source code-execution engine; each node runs sandboxed jobs in `isolate`.
Reaper	The `workspaces:reap` command that stops idle, lapsed or over-limit workspaces.
Readiness score	0–100 blend of interview, contest rating, practice, consistency and academics.
Role template	Reusable role definition whose `rounds_config` seeds a track's three rounds.
Section	A free-text class/section label on a student; a coordinator manages exactly one.
Slot	One concurrent execution a Piston node may run (set to physical cores).
SOW	Statement of Work — the markdown spec a Project Lab student builds against.
Static export	Next.js build mode that emits plain HTML/JS; no server runtime.
Talent Pool	Consent-based marketplace of students who passed Mellow scouting assessments.
TPO	Training & Placement Officer — the college's placement administrator (`admin_tpo`).
Tenant wall	The controller-level scoping that keeps one college/company away from another's rows.
Workspace host	A separate machine running Docker + Caddy that hosts student VS Code containers.